

Komponenta výukového serveru TI NP-úplné problémy 2

Component of Learning Server for
Theoretical Computer Science
NP-complete problems 2

Phat Tran Dai

Bakalářská práce

Vedoucí práce: Ing. Martin Kot, Ph.D.

Ostrava, 2026

Zadání bakalářské práce

Student:

Phat Dai Tran

Studijní program:

B0613A140014 Informatika

Téma:

Komponenta výukového serveru TI - NP-úplné problémy 2
Component of Learning Server for Theoretical Computer Science - NP-
complete problems 2

Jazyk vypracování:

čeština

Zásady pro vypracování:

V rámci diplomových a bakalářských prací vzniká výukový server pro předměty teoretické informatiky. Jedná se o sadu dynamických webových stránek umožňujících studentům pochopení různých typů úloh a problémů tím, že si mohou zadat na stránce libovolné zadání a zobrazí se jim řešení včetně postupu. Cílem této práce je vytvořit komponentu (tedy sadu webových stránek) pro výuku vybraných NP-úplných problémů a převodů mezi nimi.

1. Nastudujte si problematiku tříd složitosti problémů a s tím souvisejícím převodem mezi problémy.
2. Vytvořte dynamické webové stránky umožňující uživateli následující:
 - a) Nechat si zobrazit postupně po krocích postup algoritmu s polynomiální časovou složitostí převádějícího zadanou instanci jednoho problému na instanci jiného problému (budou implementovány alespoň 3 různé převody mezi problémy).
 - b) Zadat libovolnou instanci každého z problémů vyskytujících se v těchto převodech.
 - c) Zobrazit odpověď na otázku daného problému pro zadanou instanci, v případě kladné odpovědi i se zdůvodněním.
3. Není cílem mít co nejefektivněji implementován samotný převod, ale mít jej implementován tak, aby uživateli byla myšlenka tohoto převodu co nejsrozumitelněji ukázána.
4. Vytvořte i ukázkové vstupní instance pro implementované problémy tak, aby uživatel mohl vše vyzkoušet i bez zadávání vlastních vstupů (alespoň 5 instancí pro každý problém).

Studenti řešící toto zadání s rozdílným číslem v názvu mohou (ale nemusí) spolupracovat tak, že výsledek může mít společné uživatelské rozhraní apod. Ale každý bude implementovat jiné 3 převody mezi problémy.

Seznam doporučené odborné literatury:

- [1] Sipser, M.: Introduction to the Theory of Computation, PWS Publishing Company, 1997.
- [2] Papadimitriou, C.: Computational Complexity, Addison Wesley, 1993.
- [3] Sawa, Z.: Prezentace přednášek předmětu Teoretická informatika, dostupné online <https://www.cs.vsb.cz/sawa/ti/index.html>

Formální náležitosti a rozsah bakalářské práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí bakalářské práce: **Ing. Martin Kot, Ph.D.**

Datum zadání: 01.09.2025

Datum odevzdání: 30.04.2026

Garant studijního programu: Ing. Tomáš Fabián, Ph.D.

V IS EDISON zadáno: 14.10.2025 13:16:02

Abstrakt

Tato práce se zabývá teorií výpočetní složitosti algoritmů se zaměřením na NP-úplné problémy. Klíčovým nástrojem pro studium těchto problémů jsou polynomiální redukce, které umožňují převod instancí jednoho problému na instance jiného. Pochopení redukcí je často náročné kvůli jejich abstraktní povaze a nedostatku interaktivních nástrojů.

Cílem této práce je navrhnout a implementovat webovou aplikaci umožňující interaktivní vizualizaci polynomiálních redukcí mezi vybranými NP-úplnými problémy. Aplikace umožňuje uživatelům zadávat vlastní instance problémů, provádět redukce, zobrazovat kroky s vysvětlením a vizualizovat řešení.

Výsledkem je interaktivní výuková pomůcka spojující teoretické koncepty s praktickou implementací.

Klíčová slova: NP-úplné problémy, polynomiální redukce, výpočetní složitost, interaktivní vizualizace, výuková pomůcka

Abstract

This thesis deals with the complexity theory of algorithms with focus on NP-complete problems. Key tools for studying these problems are polynomial reductions, which allow transforming instances of one problem to instances of another. Understanding reductions is often challenging due to their abstract nature and the lack of interactive tools.

The goal of this thesis is to design and implement a web application that enables interactive visualization of polynomial reductions between selected NP-complete problems. The application allows users to input their own problem instances, perform reductions, view step-by-step explanations, and visualize solutions.

The result is an interactive educational tool that connects theoretical concepts with practical implementation.

Keywords: NP-complete problems, polynomial reductions, computational complexity, interactive visualization, educational tool

Poděkování

Rád bych poděkoval vedoucímu práce, Ing. Martinu Kotovi, Ph.D., za cenné rady a odborné vedení během procesu jejího zpracování.

Seznam použitých zkratek

EXP, EXPTIME	– Exponential Time
P, PTIME	– Polynomial Time
NP, NPTIME	– Non-deterministic Polynomial Time
3-SAT	– 3-Satisfiability
TSP	– Traveling Salesman Problem
HCYCLE	– Hamiltonian Cycle Problem
HCIRCUIT	– Hamiltonian Circuit Problem
SSP	– Subset Sum Problem
3-CG	– 3-Coloring Problem
DPLL	– Davis-Putnam-Logemann-Loveland
UI	– User Interface

Obsah

Úvod	14
1 Základní pojmy a terminologie	15
1.1 Problém a algoritmus	15
1.2 Rozhodovací problém	15
1.3 Redukce	15
1.4 NP-úplné problémy	16
2 Vybrané NP-úplné problémy	17
2.1 3-SAT	17
2.1.1 Vyjádření problému jako rozhodovacího	18
2.1.2 Instance s kladnou odpovědí	18
2.1.3 Instance se zápornou odpovědí	18
2.2 HCYCLE	20
2.2.1 Vyjádření problému jako rozhodovacího	20
2.2.2 Instance s kladnou odpovědí	20
2.2.3 Instance se zápornou odpovědí	21
2.3 HCIRCUIT	22
2.3.1 Vyjádření problému jako rozhodovacího	22
2.3.2 Instance s kladnou odpovědí	22
2.3.3 Instance se zápornou odpovědí	22
2.4 TSP	24
2.4.1 Vyjádření problému jako rozhodovacího	24
2.4.2 Příklady instancí problému	24
2.5 SSP	26
2.5.1 Vyjádření problému jako rozhodovacího	26
2.5.2 Instance s kladnou odpovědí	26
2.5.3 Instance se zápornou odpovědí	26
2.6 3-CG	27
2.6.1 Vyjádření problému jako rozhodovacího	27
2.6.2 Instance s kladnou odpovědí	27
2.6.3 Instance se zápornou odpovědí	28
3 Redukce mezi vybranými problémy	29
3.1 Redukce 3-SAT na HCYCLE	30
3.1.1 Konstrukční prvky proměnných	30
3.1.2 Konstrukční prvky klauzulí	31
3.2. Redukce 3-SAT na SSP	34
3.2.1. Reprezentace ohodnocení proměnných	34
3.2.2. Reprezentace splnění klauzulí	35

3.2.3. Vyrovnávací čísla	36
3.3 Redukce 3-SAT na 3-CG	37
3.3.1 Jádrový konstrukční prvek	37
3.3.2 Konstrukční prvek proměnné	37
3.3.3 Konstrukční prvek klauzule	38
3.4 Redukce HCYCLE na HCIRCUIT	41
3.5 Redukce HCIRCUIT na TSP	44
4. Návrh webové aplikace	46
4.1. Požadavky na systém	46
4.1.1. Funkční požadavky	46
4.1.2. Nefunkční požadavky	47
4.2. Architektura systému	48
4.2.1. Architektonický model	48
4.2.2. Zpracování dat	49
4.3. Návrh uživatelského rozhraní	50
4.3.1. Struktura rozhraní	50
4.3.2. Komponenty stránky redukce	50
4.3.2.1. Editační oblast	50
4.3.2.2. Vizualizace instance	51
4.3.2.3. Zobrazení řešení	52
4.3.2.4. Krokové zobrazení redukce	52
5 Implementace	53
5.1 Reprezentace instancí problémů	53
5.1.1 Reprezentace formule 3-SAT	54
5.1.2 Reprezentace grafů	55
5.1.3 Reprezentace problému SSP	58
5.1.4 Validace vstupu	59
5.2 Implementace redukcí	60
5.2.1 Redukce 3-SAT na HCYCLE	61
5.2.2 Redukce 3-SAT na SSP	61
5.2.3 Redukce 3-SAT na 3-CG	62
5.2.4 Redukce HCYCLE na HCIRCUIT	62
5.2.5 Redukce HCIRCUIT na TSP	62
5.2.6 Ukládání kroků redukce	62
5.3 Řešení problémů	63
5.3.1 Řešení problému 3-SAT	63
5.3.2 Řešení problému HCYCLE	63
5.3.3 Řešení problému HCIRCUIT	64
5.3.4 Řešení problému SSP	64
5.3.5 Řešení problému TSP	64
5.3.6 Řešení problému 3-CG	64
5.4 Dekódování řešení	65
5.4.1 Dekódování řešení HCYCLE na 3-SAT	65
5.4.2 Dekódování řešení SSP na 3-SAT	65
5.4.3 Dekódování řešení 3-CG na 3-SAT	66

5.4.4 Dekódování řešení HCIRCUIT na HCYCLE	66
5.4.5 Dekódování řešení TSP na HCIRCUIT	66
5.5 Použité technologie	66
6 Ukázkové instance a demonstrace	67
6.1 Redukce 3-SAT na HCYCLE	67
6.1.1 Instance s kladnou odpovědí	67
6.1.2 Instance se zápornou odpovědí	71
6.2 Redukce 3-SAT na SSP	73
6.2.1 Instance s kladnou odpovědí	73
6.2.2 Instance se zápornou odpovědí	77
6.3 Redukce 3-SAT na 3-CG	79
6.3.1 Instance s kladnou odpovědí	79
6.3.2 Instance se zápornou odpovědí	82
6.4 Redukce HCYCLE na HCIRCUIT	84
6.4.1 Instance s kladnou odpovědí	84
6.4.2 Instance se zápornou odpovědí	86
6.5 Redukce HCIRCUIT na TSP	88
6.5.1 Instance s kladnou odpovědí	88
6.5.2 Instance se zápornou odpovědí	91
7 Zhodnocení	93
7.1 Shrnutí výsledků	93
7.2 Výuková hodnota	93
7.3 Omezení	94
7.4 Možnosti rozšíření	94
Závěr	95
Zdroje	96
A Obsah přílohy	97

Seznam obrázků

1. Instance problému HCYCLE s kladnou odpovědí	20
2. Instance problému HCYCLE se zápornou odpovědí	21
3. Instance problému HCIRCUIT s kladnou odpovědí	22
4. Instance problému HCIRCUIT se zápornou odpovědí	23
5. Instance problému TSP - ohodnocený neorientovaný graf	24
6. Řešení problému TSP pro $k = 8$	25
7. Řešení problému TSP pro $k = 6$	25
8. Instance problému 3-CG s kladnou odpovědí	27
9. Validní obarvení grafu třemi barvami	28
10. Instance problému 3-CG se zápornou odpovědí	28
11. Graf redukcí mezi problémy	29
12. Konstrukční prvek proměnné x	30
13. Řetězec konstrukčních prvků proměnných x, y a z	31
14. Konstrukční prvek klauzulí $(x \vee \neg y \vee z)$ a $(\neg x \vee x \vee y)$	32
15. Průchod nevalidním konstrukčním prvkem proměnné x – přeskokování	33
16. Prevence přeskokování přidáním volného vrcholu	33
17. Jádrový konstrukční prvek instance problému 3-CG	37
18. Konstrukční prvky proměnných α, β a γ	38
19. Konstrukční prvky proměnných α, β a γ a klauzule $\kappa = \{\alpha, \neg\beta, \gamma\}$	39
20. Neexistence validního 3-obarvení v případě nesplnění klauzule	40
21. Konstrukční prvek jednoho vrcholu	41
22. Vstupní graf problému HCYCLE	42
23. Výstupní graf problému HCIRCUIT	42
24. Vstupní graf problému HCIRCUIT	44
25. Výsledný graf H – modře vyznačené hrany váhy 1, červeně váhy 2	45
26. Hamiltonovský cyklus v grafu H s váhou $k = 6$	45
27. Diagram případů užití zobrazující funkční požadavky systému z pohledu uživatele	47
28. Diagram komponent aplikace	48
29. Vývojový diagram toku dat	49
30. Wireframe model struktury rozhraní	50
31. Wireframe model editační oblasti	51
32. Wireframe model vizualizace instance	51
33. Wireframe model krokového zobrazení redukce	52
34. Třídní hierarchie datových struktur	54
35. Vizualizace grafu bez ohodnocení hran	56
36. Vizualizace grafu s ohodnocením hran	57
37. Vizualizace grafu s TeX názvy	58

38. Třídní hierarchie redukcí	61
39. Třídní hierarchie solverů a certifikátů	63
40. Třídní hierarchie dekoderů	65
41. Vstupní instance 3-SAT a výsledná instance HCYCLE (kladná odpověď) . . .	68
42. Krokové zobrazení redukce 3-SAT na HCYCLE (kladná odpověď)	69
43. Vstupní instance 3-SAT a výsledná instance HCYCLE (záporná odpověď) . .	72
44. Zadání instance 3-SAT a výsledná množina čísel SSP (kladná odpověď) . . .	74
45. Výsledná množina čísel SSP zobrazená s 3-SAT formátováním	75
46. Krokové zobrazení redukce 3-SAT na SSP (kladná odpověď)	76
47. Zadání instance 3-SAT a výsledná množina čísel SSP (záporná odpověď) . . .	78
48. Vstupní instance 3-SAT a výsledná grafová instance 3-CG (kladná odpověď) .	80
49. Krokové zobrazení redukce 3-SAT na 3CG (kladná odpověď)	81
50. Vstupní instance 3-SAT a výsledná grafová instance 3-CG (záporná odpověď)	83
51. Vstupní instance HCYCLE a výsledný graf HCIRCUIT (kladná odpověď) . .	85
52. Krokové zobrazení redukce HCYCLE na HCIRCUIT (kladná odpověď)	86
53. Vstupní instance HCYCLE a výsledný graf HCIRCUIT (záporná odpověď) .	87
54. Vstupní instance HCIRCUIT a výsledná instance TSP (kladná odpověď) . . .	89
55. Krokové zobrazení redukce HCIRCUIT na TSP (kladná odpověď)	90
56. Vstupní instance HCIRCUIT a výsledná instance TSP (záporná odpověď) . .	92

Seznam tabulek

1. Všechna možná ohodnocení proměnných formule 1 – vede ke kontradikci . . . 19

Seznam výpisů

1. Abstraktní třída <code>ProblemInstance</code>	53
2. Příklad vstupního formátu	55
3. Příklad vstupu s TeX názvy	55
4. Příklad vstupu grafu bez ohodnocení hran	55
5. Příklad vstupu grafu s ohodnocením hran	56
6. Příklad vstupu grafu s TeX názvy	57
7. Příklad vstupu SSP	58
8. Abstraktní třída <code>Reducer</code>	60
9. Rozhraní <code>Solver</code>	63
10. Rozhraní <code>Decoder</code>	65

Úvod

Tato práce se zabývá problematikou výpočetní složitosti algoritmů se zaměřením na třídu NP-úplných problémů. Výpočetní složitost představuje jednu ze základních disciplín informatiky, která studuje nároky na zdroje potřebné k vyřešení algoritmického problému. Mezi tyto zdroje patří především čas výpočtu a operační paměť pro uložení mezivýsledků.

Zvláštní postavení v teorii složitosti zaujímají NP-úplné problémy. Tato třída sdružuje problémy, pro které neexistuje známý algoritmus s polynomiální časovou složitostí. Zároveň však platí, že efektivní řešení libovolného z nich by umožnilo efektivně řešit všechny ostatní problémy ve třídě NP. Otázka, zda takový algoritmus existuje, zůstává jedním z nejvýznamnějších nevyřešených problémů současné informatiky.

Základním nástrojem pro zkoumání vztahů mezi problémy jsou polynomiální redukce. Ty umožňují transformovat instanci jednoho problému na instanci problému jiného tak, že odpověď pro druhý problém implikuje odpověď pro problém původní. Tento koncept tvoří základ definice NP-úplnosti a umožňuje srovnání relativní obtížnosti jednotlivých problémů.

Studium redukcí je však náročné, protože formální konstrukce bývají abstraktní a obtížně představitelné. Studenti se setkávají s bariérou mezi matematickým popisem a jeho konkrétní vizuální reprezentací. Navíc stávající výukové materiály a animace jsou zpravidla statické a pracují pouze s předem danými instancemi. Z tohoto důvodu je žádoucí poskytnout studentům nástroj, který by jim pomohl tyto principy lépe pochopit.

Cílem této práce je navrhnout a implementovat webovou aplikaci, která by tento nedostatek alespoň částečně odstranila. Aplikace slouží jako interaktivní výuková pomůcka, umožňující zadávat instance zdrojových problémů, realizovat pět polynomiálních redukcí a vizualizovat celý transformační proces včetně jeho jednotlivých kroků. Součástí řešení jsou rovněž algoritmy pro řešení vzniklých instancí a mechanismy pro dekodování výsledků zpět do kontextu vstupního problému.

Práce je rozdělena na teoretickou část (kap. 1 až 3) a praktickou část (kap. 4 až 6).

První kapitola definuje základní pojmy z teorie výpočetní složitosti potřebné pro pochopení následujícího textu. Druhá kapitola představuje šest NP-úplných problémů, se kterými aplikace pracuje, včetně jejich formálních definic a příkladů instancí. Třetí kapitola tvoří teoretické jádro práce a obsahuje detailní popis pěti polynomiálních redukcí mezi těmito problémy.

Ve čtvrté kapitole je popsán návrh systému z hlediska architektury, požadavků a uživatelského rozhraní. Pátá kapitola se věnuje technické implementaci, včetně reprezentace instancí, redukčních, řešících a dekodovacích modulů a použitých technologií. Šestá kapitola demonstruje funkčnost aplikace na ukázkových instancích s kladnou i zápornou odpovědí pro každou redukci.

Závěr shrnuje dosažené výsledky, diskutuje omezení navrženého řešení a nastiňuje možnosti budoucího rozšíření.

Kapitola 1

Základní pojmy a terminologie

Tato kapitola stručně zavádí základní pojmy z teorie výpočetní složitosti, které jsou nezbytné pro další části práce. Nejde o úplný teoretický přehled, ale o vymezení terminologie a konceptů, se kterými bude práce dále operovat.

1.1 Problém a algoritmus

Pod problémem rozumíme obecnou úlohu, kterou je třeba řešit, a to zpravidla pro různé vstupní instance. Instance problému je konkrétní zadání, tedy konkrétní vstupní data, pro která je požadováno rozhodnutí nebo řešení. Například u problému splnitelnosti booleovských formulí je instancí konkrétní logický výraz, zatímco u grafových problémů je instancí konkrétní graf se zadanými vlastnostmi.

Pod algoritmem rozumíme konečný a jednoznačně definovaný postup, který pro danou vstupní instanci produkuje výstup. Řekneme, že algoritmus řeší problém, pokud pro každou jeho vstupní instanci poskytne správnou odpověď.

1.2 Rozhodovací problém

V teorii výpočetní složitosti se obvykle pracuje s tzv. rozhodovacími problémy. Rozhodovací problém je formulován tak, že pro danou vstupní instanci je odpověď buď ano, nebo ne.

Optimalizační i vyhledávací problémy lze převést na rozhodovací problémy, například zavedením prahové hodnoty a otázky, zda existuje řešení splňující dané omezení. Dejme tomu optimalizační úlohu najít nejkratší cestu v grafu lze reformulovat zavedením prahové hodnoty k a otázky, zda existuje cesta s délkou nejvýše k .

1.3 Redukce

Redukci lze chápat jako algoritmickou transformaci jedné vstupní instance na jinou výstupní instanci. Formálněji, redukce problému A na problém B je převod instance problému A na instanci problému B tak, že odpověď pro instanci problému B lze využít k získání odpovědi pro instanci problému A . U rozhodovacích problémů se odpověď

zachovává: instance problému A má kladnou odpověď právě tehdy, když má kladnou odpověď i odpovídající instance problému B .

V kontextu této práce uvažujeme pouze polynomiální redukce, tedy takové, které lze provést v polynomiálním čase. Redukce slouží k porovnávání obtížnosti problémů a tvoří základ definice NP-úplnosti.

1.4 NP-úplné problémy

Problém označíme jako NP-těžký, pokud na něj lze polynomiálně redukovat každý problém ze třídy NP. Pokud je problém NP-těžký a zároveň náleží do třídy NP, nazývá se NP-úplný. NP-úplné problémy jsou tedy nejobtížnějšími problémy třídy NP. To znamená, že existence polynomiálního algoritmu pro jediný NP-úplný problém by implikovala existenci polynomiálního algoritmu pro všechny problémy třídy NP, a tím pádem rovnost $P = NP$.

Kapitola 2

Vybrané NP-úplné problémy

Tato kapitola představuje vybrané NP-úplné problémy, se kterými pracuje navržený systém. Pro každý problém je uvedena jeho formální definice, popis vstupu a formulace rozhodovací otázky. Ukážeme si také konkrétní instance těchto problémů s kladnou odpovědí i instance se zápornou odpovědí.

V této práci se zaměříme na následující NP-úplné problémy:

- 3-SAT – splnitelnost booleovské formule v 3-konjunktivní normální formě,
- HCYCLE – existence hamiltonovského cyklu v orientovaném grafu,
- HCIRCUIT – existence hamiltonovské kružnice¹ v neorientovaném grafu,
- TSP – problém obchodního cestujícího (v rozhodovací verzi),
- SSP – problém podmnožinového součtu,
- 3-CG – 3-obarvitelnost vrcholů grafu.

Tyto problémy slouží jako základní stavební kameny pro demonstraci redukcí a jejich vizualizaci.

2.1 3-SAT

Problém splnitelnosti booleovské formule v 3-konjunktivní normální formě (3-Satisfiability, 3-SAT) je definován následovně.

Uvažujeme booleovskou formuli:

$$\Phi = (\mathcal{V}, \mathcal{K}),$$

kde $\mathcal{V}$ je množina booleovských proměnných a $\mathcal{K}$ je množina klauzulí, z nichž každá obsahuje právě tři literály².

Každou klauzuli $\kappa \in \mathcal{K}$ lze chápat jako množinu obsahující právě 3 literály. Klauzuli budeme tedy značit:

$$\kappa = \{\alpha, \beta, \gamma\},$$

¹Obecně se pojem hamiltonovský cyklus vztahuje jak na orientované, tak na neorientované grafy. Pro zachování přehlednosti a terminologické jednoznačnosti však budeme v této práci používat následující konvenci:

- pojem hamiltonovský cyklus bude používán výhradně pro orientované grafy,
- pojem hamiltonovská kružnice bude používán výhradně pro neorientované grafy.

V obou případech budeme o grafu říkat, že je hamiltonovský, pokud obsahuje hamiltonovský cyklus (v případě orientovaného grafu), resp. hamiltonovskou kružnici (v případě grafu neorientovaného).

²Literál představuje výskyt booleovské proměnné ve formuli. Proměnná se v ní může vyskytovat buď v neznegované, nebo v znegované podobě.

kde α , β a γ jsou literály odpovídající booleovským proměnným a , b a c v tomto pořadí, přičemž každý z literálů může být případně znegován.

Pro ilustraci uvažujme následující instanci problému:

$$(a \vee b \vee c) \wedge (x \vee y \vee z) \wedge (\neg x \vee \neg b \vee c).$$

V tomto případě mají množiny $\mathcal{V}$ a $\mathcal{K}$ následující podobu:

$$\begin{aligned}\mathcal{V} &= \{a, b, c, x, y, z\} \\ \mathcal{K} &= \{\{a, b, c\}, \{x, y, z\}, \{\neg x, \neg b, c\}\}\end{aligned}$$

Zaměřme se na poslední klauzuli $(\neg x \vee \neg b \vee c)$. Tato klauzule odpovídá prvku $\{\neg x, \neg b, c\}$ v množině $\mathcal{K}$, kde literály $\neg x$, $\neg b$ a c představují výskyty booleovských proměnných x , b a c v dané klauzuli.

Zápis $\mathcal{V}(\Phi)$ označuje množinu všech booleovských proměnných formule Φ , zatímco $\mathcal{K}(\Phi)$ označuje množinu jejích klauzulí.

Pravdivostní hodnoty jsou v této práci vyjadřovány zpravidla jako T pro pravdivou hodnotu a F pro nepravdivou hodnotu.

Formule Φ se nazývá splnitelná, pokud existuje ohodnocení booleovských proměnných, při němž se celá formule vyhodnotí jako pravdivá. Ekvivalentně lze říci, že splnitelnost nastává tehdy, je-li splněna každá klauzule z množiny $\mathcal{K}$.

2.1.1 Vyjádření problému jako rozhodovacího

Vstup Booleovská formule $\Phi = (\mathcal{V}, \mathcal{K})$ ve 3-KNF.

Otázka Je formule Φ splnitelná?

2.1.2 Instance s kladnou odpovědí

Mějme booleovský výraz ve 3-KNF:

$$(x \vee y \vee z) \wedge (\neg x \vee \neg y \vee \neg z),$$

který v množinovém zápise vypadá následovně:

$$\Phi = (\mathcal{V}, \mathcal{K}) \quad \mathcal{V} = \{x, y, z\} \quad \mathcal{K} = \{\{x, y, z\}, \{\neg x, \neg y, \neg z\}\}.$$

Jedno z možných ohodnocení, které splňuje formuli Φ , je:

$$\begin{aligned}x &= T \\ y &= F \\ z &= F.\end{aligned}$$

2.1.3 Instance se zápornou odpovědí

Mějme booleovský výraz:

$$(\neg x \vee \neg x \vee \neg x) \wedge (\neg y \vee \neg y \vee x) \wedge (\neg x \vee \neg x \vee y) \wedge (x \vee x \vee x). \quad (1)$$

Jeho podoba v množinovém zápise je:

$$\Phi = (\mathcal{V}, \mathcal{K})$$

$$\mathcal{V} = \{x, y\}$$

$$\mathcal{K} = \{\{\neg x, \neg x, \neg x\}, \{\neg y, \neg y, x\}, \{\neg x, \neg x, y\}, \{x, x, x\}\}.$$

Tento výraz je kontradikcí, protože pro libovolné ohodnocení jeho proměnných nebude nikdy splněn. Výraz Φ je nesplnitelný.

x	y	Φ
F	F	F
F	T	F
T	F	F
T	T	F

Tabulka 1: Všechna možná ohodnocení proměnných formule 1 – vede ke kontradikci

2.2 HCYCLE

Uvažujme orientovaný graf:

$$G = (V, E),$$

kde V je množina vrcholů a $E \subseteq V \times V$ je množina orientovaných hran.

Problém hamiltonovského cyklu (Hamiltonian Cycle Problem, HCYCLE) spočívá v určení, zda graf G obsahuje hamiltonovský cyklus, tj. posloupnost vrcholů

$$P = (v_1, v_2, \dots, v_{|V|-1}, v_{|V|}, v_1)$$

která splňuje následující podmínky:

- každý vrchol z množiny V se v posloupnosti P vyskytuje právě jednou s výjimkou počátečního a koncového vrcholu v_1 ,
- platí $(v_i, v_{i+1}) \in E$ pro $1 \leq i < |V|$ a $(v_{|V|}, v_1) \in E$.

Jinými slovy, hamiltonovský cyklus je orientovaná uzavřená cesta, která prochází všemi vrcholy grafu právě jednou. Graf, který hamiltonovský cyklus obsahuje, nazýváme hamiltonovský.

2.2.1 Vyjádření problému jako rozhodovacího

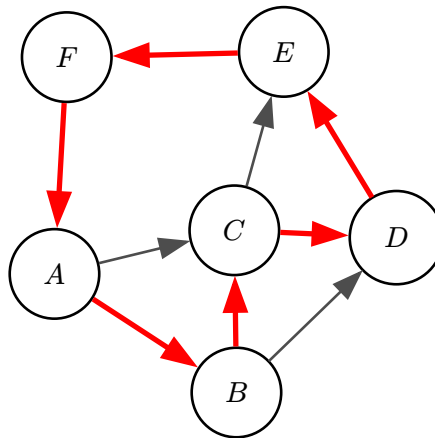
Vstup Orientovaný graf $G = (V, E)$.

Otázka Obsahuje graf G hamiltonovský cyklus?

2.2.2 Instance s kladnou odpovědí

Mějme graf $G = (V, E)$, kde:

$$\begin{aligned} V &= \{A, B, C, D, E, F\} \\ E &= \{\{A, C\}, \{B, D\}, \{C, E\}, \{A, B\}, \{B, C\}, \\ &\quad \{C, D\}, \{D, E\}, \{E, F\}, \{F, A\}\}. \end{aligned}$$



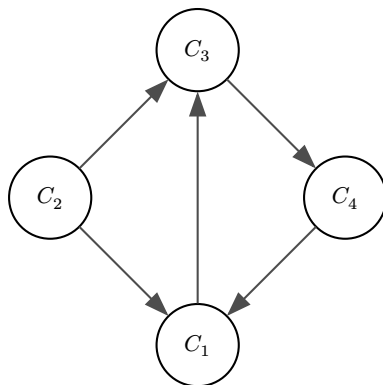
Obrázek 1: Instance problému HCYCLE s kladnou odpovědí

Graf G je hamiltonovský, protože obsahuje hamiltonovský cyklus daný uzavřenou cestou:

$$P = (A, B, C, D, E, F, A).$$

2.2.3 Instance se zápornou odpovědí

Mějme graf:



Obrázek 2: Instance problému HCYCLE se zápornou odpovědí

Tento graf neobsahuje hamiltonovský cyklus. Vrchol C_2 nemá žádnou vstupní hranu, a proto se do něj není možné vrátit poté, co jej cyklus opustí.

2.3 HCIRCUIT

Problém hamiltonovské kružnice (Hamiltonian Circuit, HCIRCUIT) je obdobou problému HCYCLE definovaný pro neorientované grafy.

Uvažujme neorientovaný graf:

$$G = (V, E),$$

kde V je množina vrcholů a $E \subseteq \{\{u, v\} \mid u, v \in V \wedge u \neq v\}$ je množina neorientovaných hran, přičemž každá hrana spojuje dva různé vrcholy.

Řekneme, že graf G obsahuje hamiltonovskou kružnici, pokud v něm existuje uzavřená cesta, která prochází každým vrcholem grafu právě jednou.

2.3.1 Vyjádření problému jako rozhodovacího

Vstup Neorientovaný graf $G = (V, E)$.

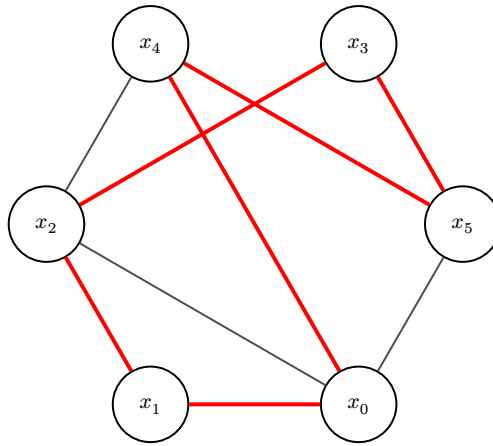
Otázka Obsahuje graf G hamiltonovskou kružnici?

2.3.2 Instance s kladnou odpovědí

Mějme graf $G = (V, E)$, kde:

$$V = \{x_0, x_1, x_2, x_3, x_4, x_5\}$$

$$E = \{\{x_0, x_1\}, \{x_4, x_2\}, \{x_5, x_3\}, \{x_3, x_2\}, \{x_5, x_0\}, \{x_2, x_1\}, \{x_4, x_5\}, \{x_0, x_2\}, \{x_4, x_0\}\}.$$



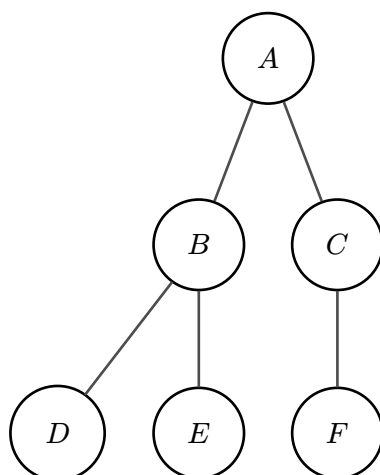
Obrázek 3: Instance problému HCIRCUIT s kladnou odpovědí

Graf G je hamiltonovský, protože obsahuje hamiltonovskou kružnici danou uzavřenou cestou:

$$P = (x_0, x_4, x_5, x_3, x_2, x_1, x_0).$$

2.3.3 Instance se zápornou odpovědí

Mějme graf strom:



Obrázek 4: Instance problému HCIRCUIT se zápornou odpovědí

Tento graf neobsahuje hamiltonovskou kružnici, protože je acyklický.

2.4 TSP

Uvažujme ohodnocený neorientovaný graf:

$$G = (V, E),$$

kde V je množina vrcholů a $E \subseteq \{\{u, v\} \mid u, v \in V \wedge u \neq v\}$ je množina neorientovaných hran. Každé hraně je přiřazena nezáporná váha pomocí funkce:

$$w : E \rightarrow \mathbb{N}.$$

Problém obchodního cestujícího (Traveling Salesman Problem, TSP) je optimalizační problém, jehož cílem je nalézt hamiltonovskou kružnici v grafu G s minimální celkovou cenou, kde cena kružnice je dána součtem vah jejích hran.

Stejně jako každý optimalizační problém lze i TSP převést na odpovídající problém rozhodovací.

Řekneme, že graf G obsahuje hamiltonovskou kružnici s cenou nejvýše $k \in \mathbb{N}$, pokud existuje posloupnost vrcholů:

$$P = (v_1, v_2, \dots, v_{|V|-1}, v_{|V|}, v_1),$$

která splňuje následující podmínky:

- každý vrchol z množiny V se v posloupnosti P vyskytuje právě jednou s výjimkou počátečního a koncového vrcholu v_1 ,
- platí $\{v_i, v_{i+1}\} \in E$ pro $1 \leq i < |V|$ a $\{v_{|V|}, v_1\} \in E$,
- celková cena kružnice splňuje $\sum_i w(\{v_i, v_{i+1}\}) \leq k$.

Abychom získali optimální řešení, kterým je uzavřená cesta s nejnižší cenou splňující zmíněné podmínky, budeme hodnotu k snižovat do té doby, než bude odpověď záporná. V tu chvíli víme, že cena optimální cesty je $k+1$ a žádná jiná cesta s nižší cenou neexistuje.

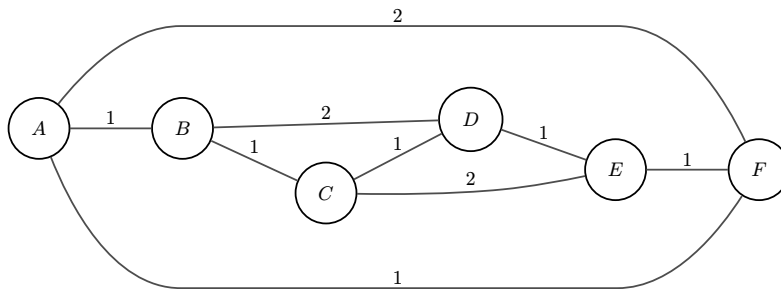
2.4.1 Vyjádření problému jako rozhodovacího

Vstup Ohodnocený neorientovaný graf G a konstanta $k \in \mathbb{N}$.

Otázka Obsahuje graf G hamiltonovskou kružnici s cenou nejvýše k ?

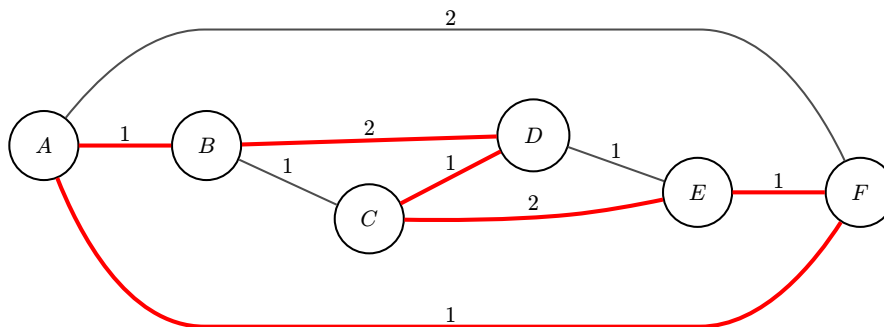
2.4.2 Příklady instancí problému

Mějme graf $G = (V, E)$:



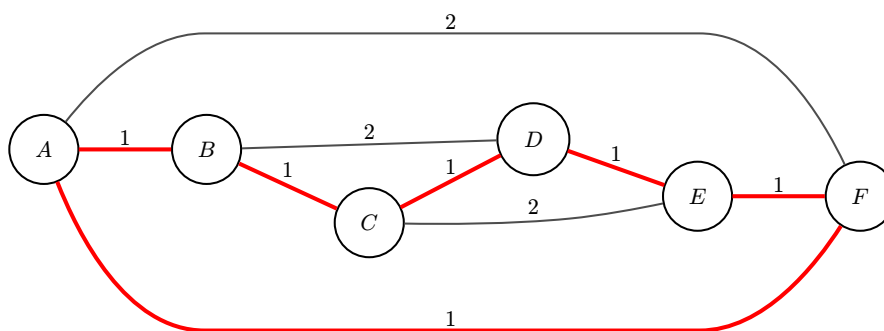
Obrázek 5: Instance problému TSP - ohodnocený neorientovaný graf

Pro $k = 8$ existuje možné řešení a odpověď na rozhodovací otázku je kladná.



Obrázek 6: Řešení problému TSP pro $k = 8$

Pro $k = 6$ existuje také možné řešení. Odpověď na rozhodovací otázku je kladná.



Obrázek 7: Řešení problému TSP pro $k = 6$

Pro $k = 5$ žádné řešení neexistuje, tudíž odpověď na rozhodovací otázku je záporná a nejnižší dosažitelná cena je $k + 1 = 5 + 1 = 6$. Cesta (A, B, C, D, E, F, A) zobrazená na obrázku 7 o celkové ceně 6 je optimálním řešením.

2.5 SSP

Problém SSP (Subset Sum Problem, SSP) spočívá v určení, zda pro danou konečnou množinu přirozených čísel:

$$S \subseteq \mathbb{N}$$

a cílovou hodnotu $\tau \in \mathbb{N}$ existuje podmnožina $S' \subseteq S$, jejíž součet prvků je roven dané cílové hodnotě τ :

$$\sum_{s \in S'} s = \tau.$$

2.5.1 Vyjádření problému jako rozhodovacího

Vstup Konečná množina přirozených čísel $S \subseteq \mathbb{N}$ a cílová hodnota $\tau \in \mathbb{N}$.

Otázka Existuje podmnožina $S' \subseteq S$ taková, že součet jejích prvků je roven τ ?

2.5.2 Instance s kladnou odpovědí

Mějme množinu čísel:

$$S = \{3, 7, 10, 5\}$$

a cílovou hodnotu $\tau = 15$.

Validním řešením je $S' = \{10, 5\}$, protože:

$$\sum_{s \in S'} s = 10 + 5 = 15 = \tau.$$

Dalším možným řešením je $S'' = \{3, 7, 5\}$, protože:

$$\sum_{s \in S''} s = 3 + 7 + 5 = 15 = \tau.$$

2.5.3 Instance se zápornou odpovědí

Mějme množinu čísel:

$$S = \{3, 5, 11\}$$

a cílovou hodnotu $\tau = 10$.

Tato instance nemá řešení. Ať už vybereme jakoukoli kombinaci prvků z množiny S , nikdy nebude součet těchto prvků roven 10.

$$S' \subseteq S$$

$$S' \in \{\emptyset, \{3\}, \{5\}, \{11\}, \{3, 5\}, \{3, 11\}, \{5, 11\}, \{3, 5, 11\}\}$$

$$\sum_{s \in S'} s \neq 10.$$

2.6 3-CG

Uvažujme neorientovaný graf $G = (V, E)$, kde V je množina vrcholů a $E \subseteq \{\{u, v\} \mid u, v \in V \wedge u \neq v\}$. Necht' K je množina tří barev, například červené, zelené a modré:

$$K = \{R, G, B\}$$

Problém barvení grafu třemi barvami (3-Coloring Graph Problem, 3-CG) spočívá v určení, zda existuje zobrazení:

$$\kappa : V \rightarrow K,$$

které každému vrcholu přiřadí právě jednu barvu tak, aby žádné dva sousední vrcholy neměly stejnou barvu.

Formálně požadujeme, aby pro každou hranu $\{u, v\} \in E$ platilo $\kappa(u) \neq \kappa(v)$.

2.6.1 Vyjádření problému jako rozhodovacího

Vstup Neorientovaný graf $G = (V, E)$.

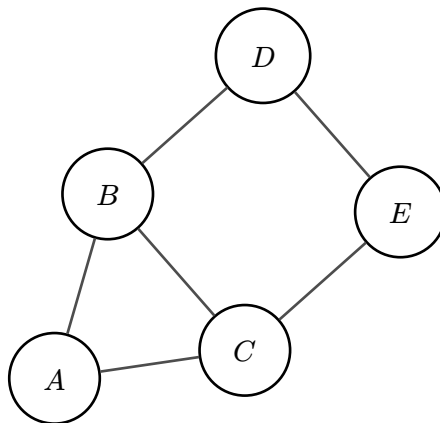
Otázka Lze vrcholy grafu obarvit 3 barvami tak, aby žádné dva sousední vrcholy neměly stejnou barvu?

2.6.2 Instance s kladnou odpovědí

Mějme neorientovaný graf $G = (V, E)$, kde:

$$V = \{A, B, C, D, E\}$$

$$E = \{\{A, B\}, \{A, C\}, \{B, C\}, \{B, D\}, \{C, E\}, \{D, E\}\}.$$



Obrázek 8: Instance problému 3-CG s kladnou odpovědí

Pro tento graf existuje validní předpis funkce $\kappa : V \rightarrow K$:

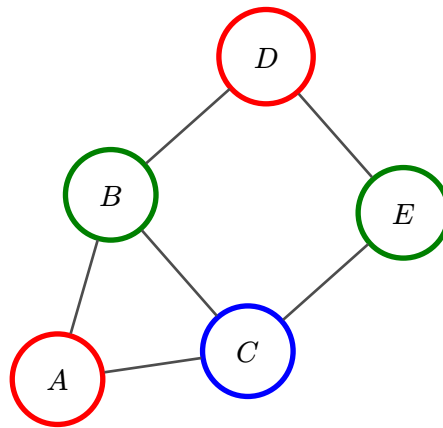
$$A \rightarrow R$$

$$B \rightarrow G$$

$$C \rightarrow B$$

$$D \rightarrow R$$

$$E \rightarrow G.$$



Obrázek 9: Validní obarvení grafu třemi barvami

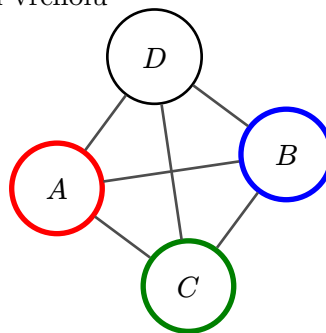
2.6.3 Instance se zápornou odpovědí

Mějme neorientovaný graf $G = (V, E)$, kde:

$$V = \{A, B, C, D\}$$

$$E = \{\{A, B\}, \{A, C\}, \{A, D\}, \{D, C\}, \{B, D\}, \{B, C\}\}.$$

nezbývá žádná barva
pro obarvení vrcholu



Obrázek 10: Instance problému 3-CG se zápornou odpovědí

Pokud obarvíme vrcholy A , B a C každý jinou barvou, nezůstane žádná barva pro vrchol D . Ten totiž nemůžeme obarvit žádnou ze tří použitých barev, protože je incidentní ke všem ostatním vrcholům, které už tyto barvy mají.

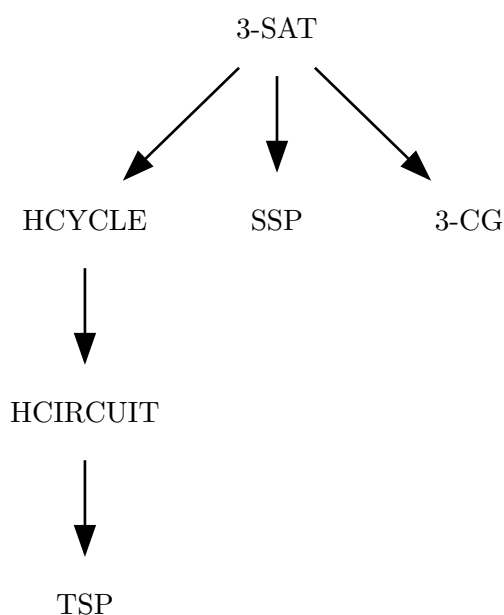
Kapitola 3

Redukce mezi vybranými problémy

Tato kapitola tvoří teoretické jádro práce a je věnována redukcím mezi vybranými NP-úplnými problémy. Cílem je nejen formálně popsat jednotlivé převody, ale především objasnit jejich princip a usnadnit jejich pochopení.

V této práci se zaměřujeme na následující redukce:

- redukci problému 3-SAT na problémy HCYCLE, SSP a 3-CG,
- redukci problému HCYCLE na problém HCIRCUIT,
- redukci problému HCIRCUIT na problém TSP.



Obrázek 11: Graf redukcí mezi problémy

Pro každý převod si popíšeme polynomiální algoritmus, který převede libovolnou instanci problému A na instanci problému B tak, aby byla zachována odpověď na otázku rozhodovacího problému.

Nejprve uvedeme základní myšlenku převodu, následně konceptuálně popíšeme jednotlivé kroky algoritmu a poté zdůvodníme jeho korektnost, tj. že původní instance má kladnou odpověď právě tehdy, když má kladnou odpověď i instance převedená.

Ve většině popisovaných redukcí využíváme tzv. konstrukční prvky (v anglické literatuře často označované jako *gadgets*). Konstrukční prvek je vymezená část instance cílového problému, která svým chováním simuluje určitý prvek nebo vlastnost instance původního problému. Tyto konstrukční prvky umožňují převést logickou strukturu jedné úlohy do struktury jiné úlohy tak, aby byly zachovány podstatné vlastnosti řešení. V případě redukcí na grafové problémy mají konstrukční prvky podobu vhodně navržených podgrafů. Při redukcích na problém SSP jsou konstrukčními prvky jednotlivá čísla vstupní množiny a cílová hodnota.

3.1 Redukce 3-SAT na HCYCLE

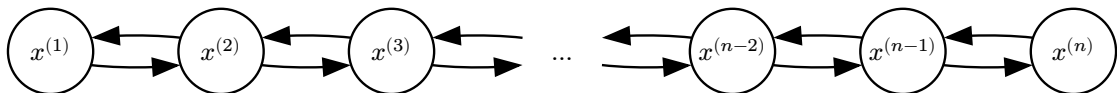
Tato redukce vychází z myšlenek prezentovaných v [1].

Algoritmus pro převod instance problému 3-SAT na instanci problému HCYCLE spočívá ve vytvoření speciálních konstrukčních prvků pro proměnné a klauzule vstupní formule. Tyto konstrukční prvky mají podobu vhodně navržených podgrafů, které jsou následně propojeny tak, aby byla zachována odpověď původní instance.

Výsledkem konstrukce je orientovaný graf, který obsahuje hamiltonovský cyklus právě tehdy, je-li vstupní instance problému 3-SAT splnitelná.

3.1.1 Konstrukční prvky proměnných

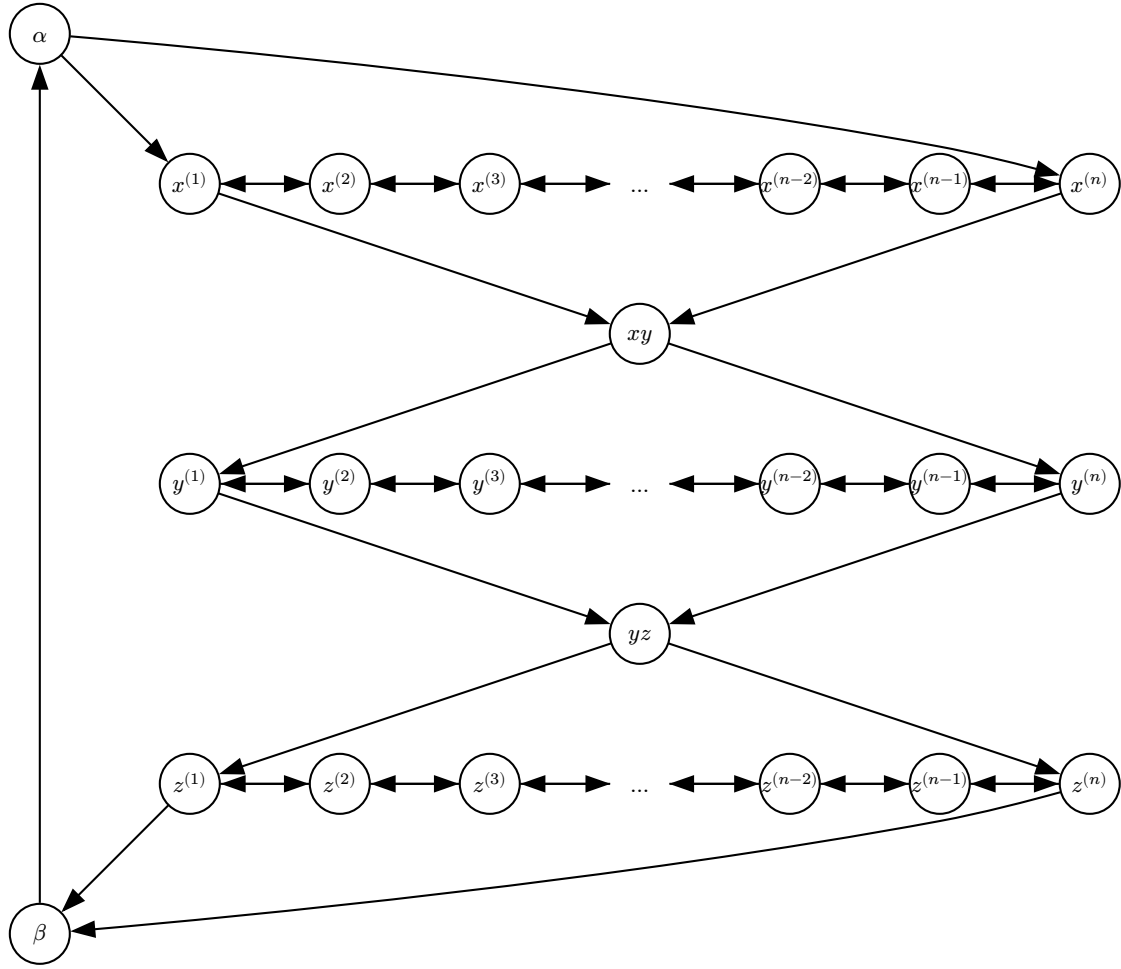
Konstrukční prvek odpovídající jedné booleovské proměnné má podobu grafu cesty, který je průchozí v obou směrech. Strukturálně tedy odpovídá neorientované cestě, avšak vzhledem k tomu, že výsledný graf je orientovaný, jsou jednotlivé hrany nahrazeny dvojicemi protisměrných orientovaných hran.



Obrázek 12: Konstrukční prvek proměnné x

Počet vrcholů tohoto konstrukčního prvku je zvolen tak, aby umožňoval jeho korektní propojení s konstrukčními prvky klauzulí. Tento prvek obsahuje alespoň dva vrcholy, přičemž jeho krajní vrcholy slouží k reprezentaci ohodnocení příslušné booleovské proměnné.

Konstrukční prvky proměnných jsou ve výsledném grafu uspořádány sériově do jednoho řetězce. Na jeho začátku, mezi jednotlivými konstrukčními prvky i na jeho konci se nacházejí speciální vrcholy, které budeme dále označovat jako *mezi-vrcholy*. Tyto mezi-vrcholy slouží jako spojovací uzly a jsou vždy napojeny na krajní vrcholy příslušných konstrukčních prvků proměnných, přičemž tyto krajní vrcholy jsou propojeny s následujícím mezi-vrcholem. Poslední mezi-vrchol je spojen s prvním mezi-vrcholem řetězce, čímž se uzavře cyklus.

Obrázek 13: Řetězec konstrukčních prvků proměnných x , y a z

Průchod tímto řetězcem jednoznačně odpovídá volbě ohodnocení jednotlivých proměnných. Lze například stanovit, že průchod levou větví konstrukčního prvku reprezentuje přiřazení pravdivostní hodnoty T dané proměnné, zatímco průchod pravou větví odpovídá přiřazení hodnoty F . Tato konvence bude uplatňována i v následujících podkapitolách.

Jakmile je při průchodu zvolena jedna z větví, není již možné toto rozhodnutí změnit. Pokud jsme do konstrukčního prvku vstoupili levou větví, jedinou možností dalšího postupu je projít celý prvek směrem doprava. Naopak, pokud jsme vstoupili pravou větví, můžeme pokračovat pouze směrem doleva. Tím je zajištěno, že každá proměnná je v rámci libovolného hamiltonovského cyklu ohodnocena právě jednou pravdivostní hodnotou.

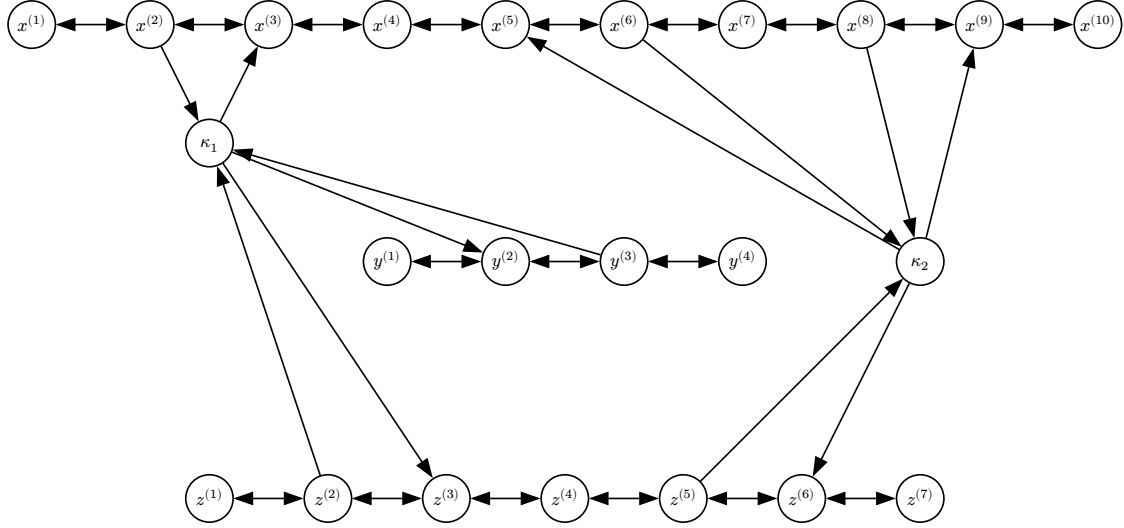
3.1.2 Konstrukční prvky klauzulí

Konstrukční prvek odpovídající jedné klauzuli je tvořen jediným vrcholem. Tento vrchol je propojen s příslušnými konstrukčními prvky proměnných, a to vždy mezi dvěma sousedními vrcholy jejich cesty. Konkrétní způsob propojení závisí na literálech, které se v dané klauzuli vyskytují.

Jako příklad uvažujme booleovskou formuli:

$$(x \vee \neg y \vee z) \wedge (\neg x \vee x \vee y).$$

Pro každou klauzuli vytvoříme konstrukční prvek – klauzuli $(x \vee \neg y \vee z)$ přiřadíme vrchol κ_1 , a klauzuli $(\neg x \vee x \vee y)$ vrchol κ_2 . Tyto vrcholy budou propojeny s příslušnými konstrukčními prvky proměnných podle výskytu literálů. Jak by takové propojení mohlo vypadat, je znázorněno na obrázku 14.

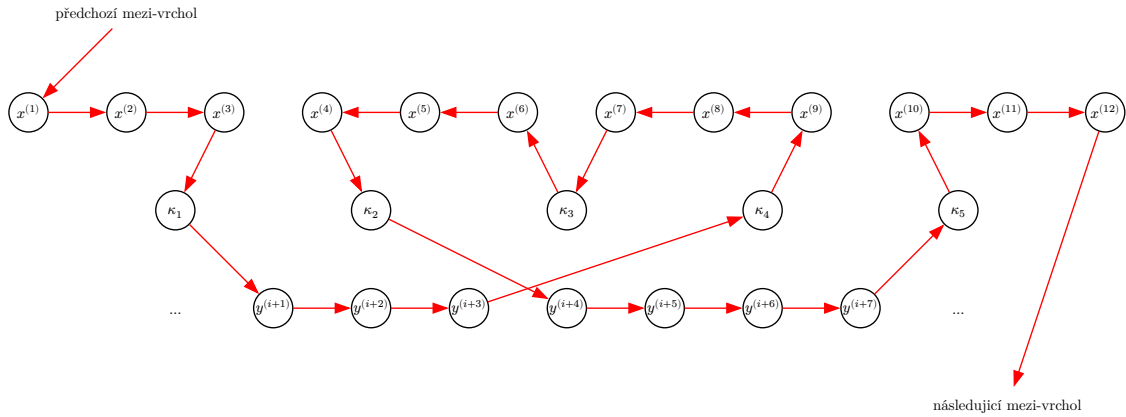


Obrázek 14: Konstrukční prvek klauzulí $(x \vee \neg y \vee z)$ a $(\neg x \vee x \vee y)$

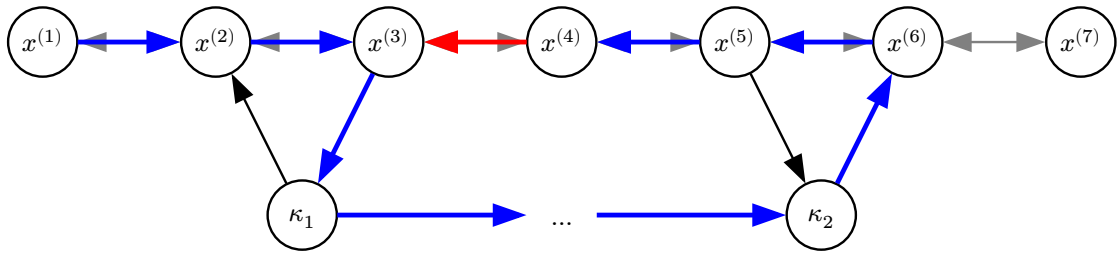
Pokud se v klauzuli vyskytuje neznegovaný literál, jsou hrany mezi konstrukčním prvkem proměnné a vrcholem klauzule přidány tak, že hrana směřující do vrcholu klauzule leží vlevo od hrany směřující z něj. V případě negovaného literálu jsou hrany přidány opačně – hrana směřující do vrcholu klauzule leží vpravo od hrany směřující z něj.

Platí, že mezi každou dvojicí vrcholů konstrukčního prvku proměnné propojených s vrcholem klauzule musí zůstat alespoň jeden vrchol volný, a samotné koncové vrcholy konstrukčního prvku proměnné nesmí být propojeny s vrcholem klauzule. Tím je zajištěno, že v rámci hamiltonovského cyklu nemůže docházet k „přeskakování“ mezi konstrukčními prvky proměnných, což by narušilo jednoznačné přiřazení pravdivostní hodnoty proměnné.

Příklad tohoto jevu je znázorněn na obrázku 15, kde dochází k přeskoku z konstrukčního prvku proměnné x na konstrukční prvek proměnné y . Současně dochází ke změně směru průchodu konstrukčním prvkem proměnné, a tím se stává dosažitelným i vrchol klauzule, který by za korektního průchodu dosažitelný být neměl.

Obrázek 15: Průchod nevalidním konstrukčním prvkem proměnné x – přeskokování

Tomuto jevu zabráníme vložení alespoň jednoho volného vrcholu mezi každou dvojici vrcholů konstrukčního prvku proměnné, jež jsou propojeny s vrcholem klauzule. Pokus o přeskok by totiž vedl k tomu, že tento vložený vrchol zůstane nenavštíven, což znemožní existenci hamiltonovského cyklu. Jakýkoli následný pokus o jeho dodatečné navštívení by nutně vedl do již navštíveného vrcholu.



Obrázek 16: Prevence přeskokování přidáním volného vrcholu

Takto navržené propojení zaručuje, že vrchol klauzule je dosažitelný v rámci hamiltonovského cyklu právě tehdy, pokud je alespoň jeden z jejích literálů splněn. Pokud je proměnným přiřazena hodnota T , procházíme konstrukčním prvkem proměnné zleva doprava, což umožňuje navštívit vrchol klauzule obsahující neznegovaný literál a vrátit se zpět. Přiřazením hodnoty F procházíme konstrukčním prvkem zprava doleva, čímž lze navštívit vrcholy klauzulí s negovaným literálem.

Tím je zachována ekvivalence mezi splnitelností původní formule a existencí hamiltonovského cyklu ve výsledném grafu.

3.2. Redukce 3-SAT na SSP

Tato redukce vychází z myšlenek prezentovaných v [2].

V této redukci převádíme booleovskou formuli Φ ve 3-KNF na instanci problému SSP, tedy na množinu čísel S a cílovou hodnotu τ .

Nechť $|\mathcal{V}|$ značí počet proměnných a $|\mathcal{K}|$ počet klauzulí formule $\Phi = (\mathcal{V}, \mathcal{K})$. Maximální počet číslic výsledných čísel i jejich celkový počet závisí právě na těchto dvou parametrech.

Maximální počet číslic každého čísla označme jako k , přičemž:

$$k = |\mathcal{V}| + |\mathcal{K}|,$$

a velikost množiny S je:

$$|S| = 2|\mathcal{V}| + 2|\mathcal{K}|.$$

Cílovou hodnotu τ zvolíme jako číslo, jehož:

- $|\mathcal{V}|$ nejvyšších číslic (odpovídajících proměnným) je rovno jedné,
- $|\mathcal{K}|$ nejnižších číslic (odpovídajících klauzulím) je rovno třem.

Formálně tedy:

$$\tau = \sum_{i=0}^{|\mathcal{V}|-1} 1 \cdot 10^{|\mathcal{K}|+i} + \sum_{j=0}^{|\mathcal{K}|-1} 3 \cdot 10^j.$$

Například pro $|\mathcal{V}| = 5$ a $|\mathcal{K}| = 3$ dostáváme:

$$\tau = 11111333.$$

3.2.1. Reprezentace ohodnocení proměnných

Každé proměnné $x_i \in \mathcal{V}(\Phi)$, kde $0 \leq i < |\mathcal{V}|$, přiřadíme dvě čísla, $x_i^{(T)}$ a $x_i^{(F)}$. Výběr právě jednoho z těchto čísel do finální podmnožiny S' reprezentuje přiřazení pravdivostní hodnoty proměnné x_i – výběr čísla $x_i^{(T)}$ odpovídá ohodnocení proměnné x_i hodnotou T a výběr čísla $x_i^{(F)}$ odpovídá ohodnocení proměnné x_i hodnotou F .

Obě tato čísla mají na i -té pozici³ číslici rovnou jedné, zatímco všechny ostatní číslice jsou prozatím nulové.

Tím je zajištěno, že v libovolné podmnožině $S' \subseteq S$, jejíž součet má být roven τ , musí být pro každou proměnnou vybráno právě jedno z čísel $x_i^{(T)}$ nebo $x_i^{(F)}$. Kdyby nebylo vybráno žádné, příslušná číslice by byla v součtu množiny S' nulová. Kdyby byla vybrána obě, vznikla by číslice 2. V obou případech by se součet lišil od τ , kde je na dané pozici právě 1.

Výběr podmnožiny tedy jednoznačně odpovídá volbě pravdivostního ohodnocení proměnných.

Jako příklad uvažujme formuli:

³V této práci jsou číslice čísel indexovány zleva doprava od nuly. Například pro číslo 0003682 je číslice na indexu 1 rovna 0, číslice na indexu 3 je rovna 3, číslice na indexu 4 je 6, číslice na indexu 6 je 2 atd. Nuly na začátku čísel slouží pouze pro přehlednost a nemají vliv na jejich hodnotu – například 00100 je totéž číslo jako 100.

$$(\alpha \vee \beta \vee \gamma) \wedge (\neg\alpha \vee \neg\beta \vee \gamma),$$

kterou lze v množinovém zápisu vyjádřit jako:

$$\begin{aligned}\mathcal{V}(\Phi) &= \{\alpha, \beta, \gamma\} \\ \mathcal{K}(\Phi) &= \{\{\alpha, \beta, \gamma\}, \{\neg\alpha, \neg\beta, \gamma\}\}.\end{aligned}$$

Klauzule si označme jako $\kappa_0 = \{\alpha, \beta, \gamma\}$ a $\kappa_1 = \{\neg\alpha, \neg\beta, \gamma\}$.

Maximální počet číslic každého čísla bude:

$$k = |\mathcal{V}(\Phi)| + |\mathcal{K}(\Phi)| = 3 + 2 = 5.$$

První tři číslice odpovídají proměnným α, β a γ (poslední dvě odpovídají klauzulím κ_0 a κ_1 , těm věnujeme pozornost v podkapitole 3.2.2).

Čísla reprezentující ohodnocení proměnných mají podobu:

$$\begin{aligned}\alpha^{(T)} &= 10000 \\ \alpha^{(F)} &= 10000 \\ \beta^{(T)} &= 01000 \\ \beta^{(F)} &= 01000 \\ \gamma^{(T)} &= 00100 \\ \gamma^{(F)} &= 00100.\end{aligned}$$

První tři číslice cílové hodnoty $\tau = 11133$ odpovídají proměnným α, β, γ a jsou rovny jedné. To vynucuje, aby bylo do podmnožiny S' vybráno právě jedno číslo z každé dvojice $x_i^{(T)}$ a $x_i^{(F)}$, tedy právě jedno číslo odpovídající každé proměnné.

3.2.2. Reprezentace splnění klauzulí

Nechť $\kappa_j \in \mathcal{K}(\Phi)$, kde $0 \leq j < |\mathcal{K}|$, je klauzule formule. Pro každou klauzuli rezervujeme číslici na pozici $|\mathcal{V}| + j$.

Pokud se proměnná $x_i \in \mathcal{V}(\Phi)$ vyskytuje v klauzuli κ_j :

- jako neznegovaný literál, nastavíme číslici na pozici $|\mathcal{V}| + j$ rovnu 1 v čísle $x_i^{(T)}$,
- jako negovaný literál, nastavíme tutéž číslici rovnu 1 v čísle $x_i^{(F)}$.

Například pro klauzuli $\kappa_0 = \{\alpha, \beta, \gamma\}$ nastavíme příslušnou číslici v číslech $\alpha^{(T)}$, $\beta^{(T)}$ a $\gamma^{(T)}$, zatímco pro klauzuli $\kappa_1 = \{\neg\alpha, \neg\beta, \gamma\}$ ji nastavíme v číslech $\alpha^{(F)}$, $\beta^{(F)}$ a $\gamma^{(T)}$.

$$\begin{aligned}\alpha^{(T)} &= 10010 \\ \alpha^{(F)} &= 10001 \\ \beta^{(T)} &= 01010 \\ \beta^{(F)} &= 01001 \\ \gamma^{(T)} &= 00111 \\ \gamma^{(F)} &= 00100.\end{aligned}$$

Nyní výběr čísel s nenulovou číslicí na pozici $|\mathcal{V}| + j$ zajišťuje, že součet podmnožiny S' bude mít na této pozici kladnou hodnotu (větší než nula), a klauzule κ_j bude tedy reprezentována jako splnění.

3.2.3. Vyrovnávací čísla

Cílová hodnota τ však vyžaduje, aby každá klauzulová číslice byla rovna třem. Výběrem ohodnocení proměnných může být na dané pozici dosaženo hodnoty 0, 1, 2 nebo 3 – podle toho, kolik literálů klauzule je splněno.

Klauzule se považuje za splněnou, pokud je na této pozici dosažena hodnota alespoň 1, což odpovídá tomu, že je splněna alespoň jedním svým literálem. Aby bylo v těchto případech možné vždy dosáhnout přesně hodnoty 3, přidáme pro každou klauzuli κ_j dvě tzv. *vyrovnávací čísla*.

Každé z těchto čísel má na pozici $|\mathcal{V}| + j$ číslici rovnu 1 a na všech ostatních pozicích číslici 0.

Pokud je tedy součet na pozici $|\mathcal{V}| + j$ po výběru proměnných roven 1 nebo 2, lze pomocí těchto vyrovnávacích čísel dosáhnout přesně hodnoty 3. Jestliže je součet na příslušné pozici roven nule, což znamená, že klauzule není splněna žádným literálem, nelze ani s využitím vyrovnávacích čísel dosáhnout požadované hodnoty 3.

Pro klauzuli κ_0 zavedeme dvě vyrovnávací čísla:

$$\begin{aligned}\kappa_{0,0} &= 00010 \\ \kappa_{0,1} &= 00010,\end{aligned}$$

která mají číslo 1 na pozici odpovídající této klauzuli a na ostatních pozicích nuly.

Analogicky pro klauzuli κ_1 definujeme:

$$\begin{aligned}\kappa_{1,0} &= 00001 \\ \kappa_{1,1} &= 00001\end{aligned}$$

opět s jedinou nenulovou číslicí na pozici příslušné klauzule.

Tato konstrukce tedy zajišťuje, že existuje podmnožina $S' \subseteq S$ se součtem rovným τ právě tehdy, když je původní formule Φ splnitelná.

3.3 Redukce 3-SAT na 3-CG

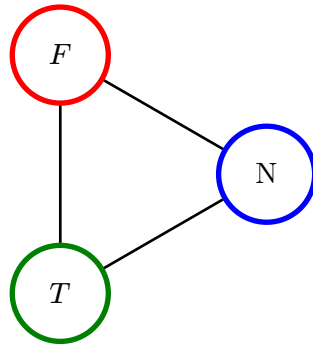
Tato redukce vychází z myšlenek prezentovaných v [3].

Redukce vstupní instance problému 3-SAT na instanci problému 3-CG, tedy na neorientovaný graf, je založena na konstrukci několika typů speciálních konstrukčních prvků:

- jádrového konstrukčního prvku,
- konstrukčních prvků proměnných a
- konstrukčních prvků klauzulí.

3.3.1 Jádrový konstrukční prvek

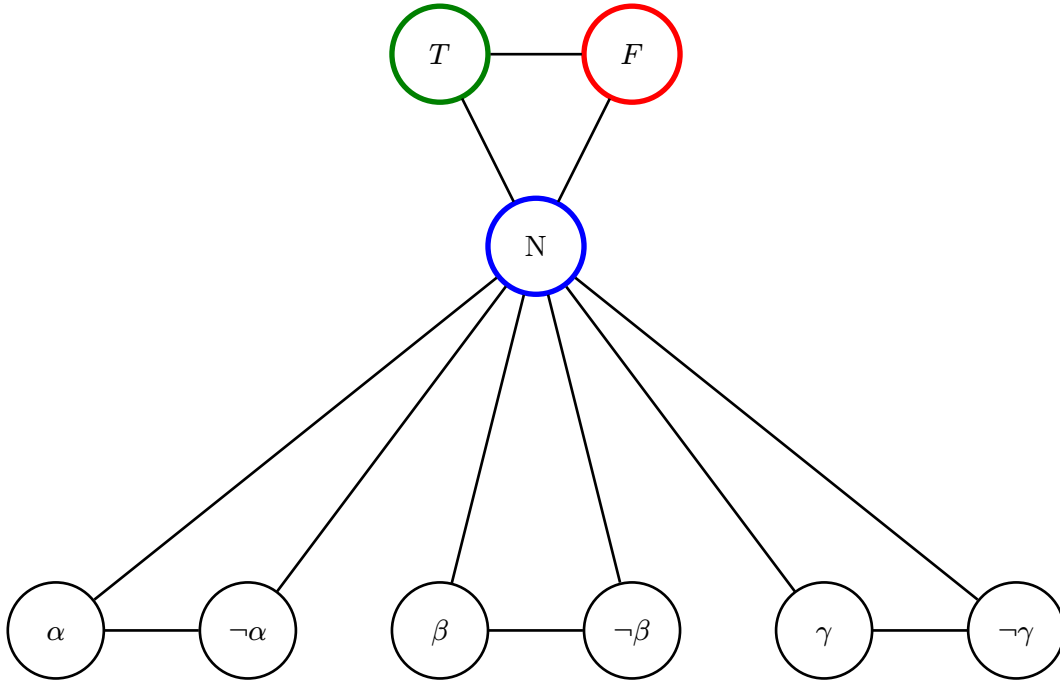
Jádrový konstrukční prvek tvoří tři vrcholy – T , F a N . Tyto vrcholy jsou navzájem propojeny tak, že vytvářejí cyklus, konkrétně hranami $\{T, N\}$, $\{F, N\}$ a $\{T, F\}$. Obarvení vrcholů tohoto podgrafu je jednoznačně dané, vrchol T je nabarven zeleně (G), vrchol F červeně (R) a vrchol N modře (B).



Obrázek 17: Jádrový konstrukční prvek instance problému 3-CG

3.3.2 Konstrukční prvek proměnné

Pro každou proměnnou $x \in \mathcal{V}(\Phi)$ se vytvoří odpovídající konstrukční prvek proměnné. Ten je tvořen trojicí vrcholů: x , $\neg x$ a vrcholem N z jádrového konstrukčního prvku. Mezi těmito vrcholy jsou zavedeny hrany $\{x, \neg x\}$, $\{x, N\}$ a $\{\neg x, N\}$, čímž opět vzniká cyklus.

Obrázek 18: Konstrukční prvky proměnných α, β a γ

Tento prvek reprezentuje přiřazení pravdivostní hodnoty dané proměnné. Vrchol N je již pevně obarven modře, a protože jsou vrcholy x a $\neg x$ s vrcholem N sousední, nemohou být obarveny modře. Jejich obarvení je tedy omezeno na dvě zbývající barvy, a to zelenou a červenou.

Zároveň jsou vrcholy x a $\neg x$ spojeny hranou, a proto nemohou mít stejnou barvu. Právě jeden z nich tedy musí být zelený a druhý červený.

Zvolíme-li interpretaci, že zelená barva reprezentuje hodnotu pravda a červená hodnotu nepravda, potom obarvení jednoznačně určuje pravdivostní hodnotu proměnné:

- je-li vrchol x zelený, potom $x = T$,
- je-li zelený vrchol $\neg x$, potom $x = F$.

3.3.3 Konstrukční prvek klauzule

Pro každou klauzuli $\kappa \in \mathcal{K}(\Phi)$ se sestrojí samostatný konstrukční prvek klauzule. Necht:

$$\kappa = \{A, B, \Gamma\},$$

kde A , B a Γ představují literály proměnných α , β a γ v daném pořadí.

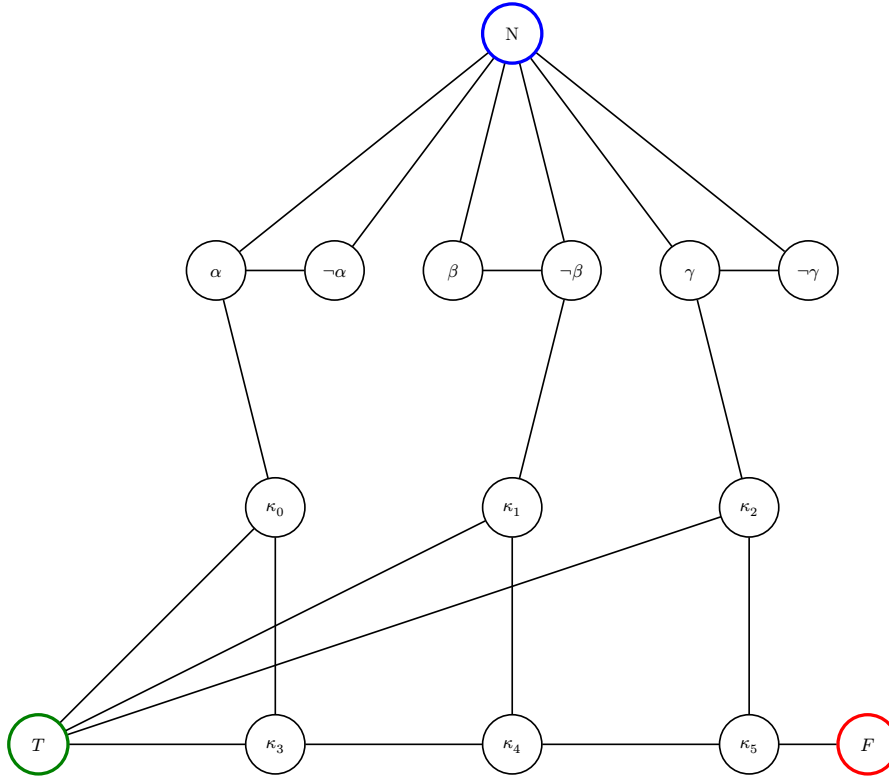
Literály, značené velkými řeckými písmeny, mohou vystupovat buď v neznegované, nebo v negované podobě. Například literál A může odpovídat buď vrcholu α , nebo vrcholu $\neg\alpha$ v konstrukčním prvku příslušné proměnné. Stejná konvence platí i pro literály B a Γ .

Konstrukční prvek klauzule obsahuje 6 vlastních vrcholů $\kappa_0, \kappa_1, \kappa_2, \kappa_3, \kappa_4, \kappa_5$, 3 vrcholy převzaté z konstrukčních prvků proměnných A, B, Γ a 2 vrcholy z jádrového konstrukčního prvku T a F .

Hrany mezi těmito vrcholy jsou definovány následovně:

$$\{A, \kappa_0\}, \{B, \kappa_1\}, \{\Gamma, \kappa_2\},$$

$$\{\kappa_0, \kappa_3\}, \{\kappa_1, \kappa_4\}, \{\kappa_2, \kappa_5\}, \{\kappa_3, \kappa_4\}, \{\kappa_4, \kappa_5\}, \\ \{T, \kappa_0\}, \{T, \kappa_1\}, \{T, \kappa_2\}, \{T, \kappa_3\}, \{\kappa_5, F\}.$$

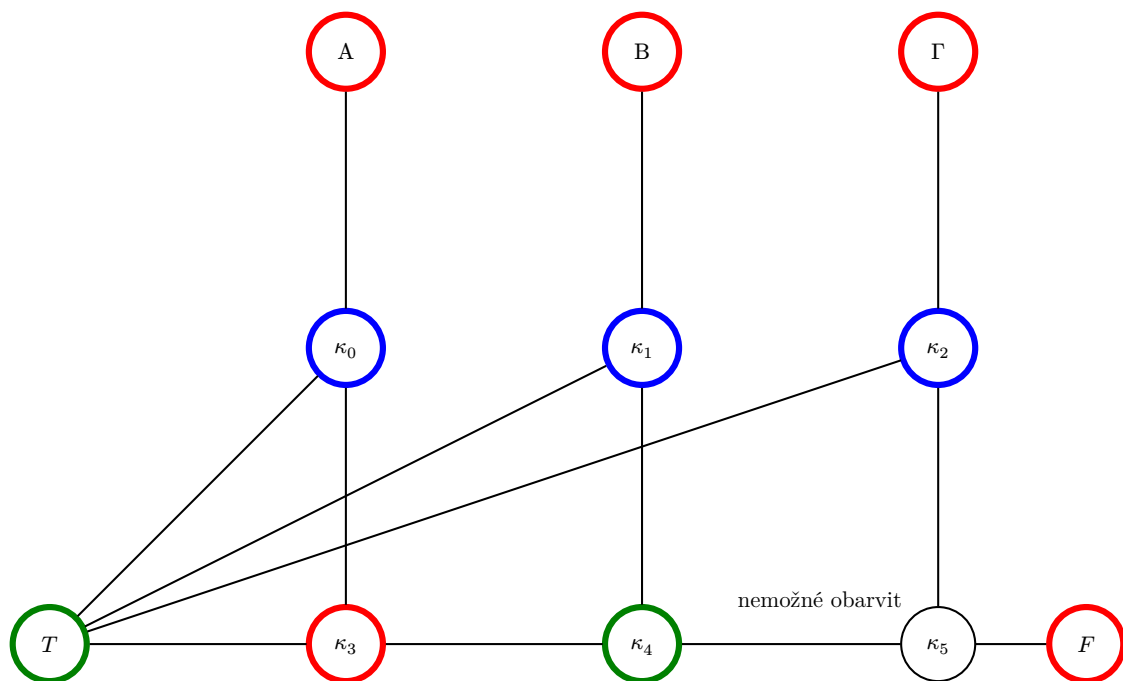


Obrázek 19: Konstrukční prvky proměnných α, β a γ a klauzule $\kappa = \{\alpha, \neg\beta, \gamma\}$

Toto uspořádání hran zajišťuje, že konstrukční prvek klauzule je 3-obarvitelný právě tehdy, když je daná klauzule splněna alespoň jedním svým literálem.

Celá struktura je navržena tak, že pokud je alespoň jeden z literálů A, B, Γ pravdivý (zelený), existuje způsob, jak vhodně dobarvit vrcholy $\kappa_0, \dots, \kappa_5$ tak, aby byly splněny všechny podmínky správného 3-obarvení.

Avšak jsou-li všechny tři literály nepravdivé (červené), dojde k propagaci omezení obarvení přes hrany mezi vrcholy κ_i , která vynutí konflikt – některé dva sousední vrcholy by musely mít stejnou barvu nebo by některý vrchol neměl k dispozici žádnou přípustnou barvu. V takovém případě tedy validní 3-obarvení neexistuje.



Obrázek 20: Neexistence validního 3-obarvení v případě nesplnění klauzule

Celý graf je tedy 3-obarvitelný právě tehdy, když existuje ohodnocení proměnných, které splňuje všechny klauzule formule Φ .

3.4 Redukce HCYCLE na HCIRCUIT

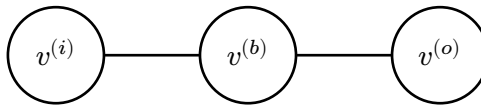
Tato redukce vychází z [4].

V této redukci převádíme instanci problému HCYCLE, tedy orientovaný graf, na instanci problému HCIRCUIT, tedy neorientovaný graf. Je založena na konstrukci konstrukčních prvků pro vrcholy.

Pro každý vrchol $v \in V(G)$ vstupního orientovaného grafu G vytvoříme konstrukční prvek skládající se ze tří vrcholů:

$$v^{(i)}, v^{(b)}, v^{(o)},$$

spojených do cesty:



Obrázek 21: Konstrukční prvek jednoho vrcholu

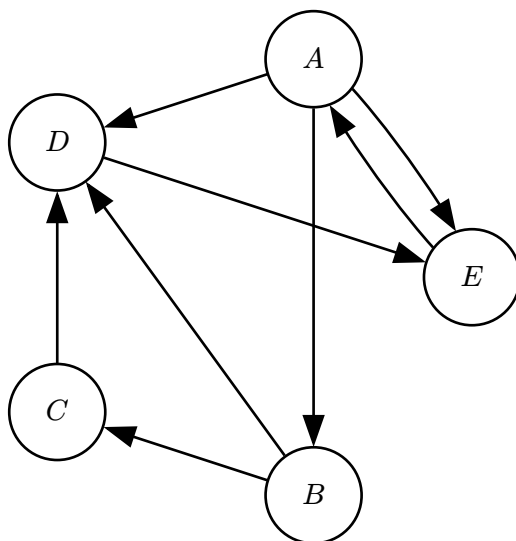
Význam jednotlivých vrcholů konstrukčního prvku je následující:

- $v^{(i)}$ – vstupní vrchol, slouží jako konec pro přicházející hrany. Návštěva tohoto vrcholu odpovídá vstupu do vrcholu v ve vstupním grafu G .
- $v^{(b)}$ – střední („bridge“) vrchol, který zajišťuje, že pokud projdeme vstupním vrcholem konstrukčního prvku, musíme zároveň navštívit všechny jeho vrcholy. Jinými slovy, jakmile vstoupíme do v_i , cyklus musí pokračovat přes v_b a následně v_o . Tudíž návštěva v_i ekvivalentně znamená navštívení vrcholu v ve vstupním grafu G .
- $v^{(o)}$ – výstupní vrchol, slouží jako počátek odchozí hrany. Z tohoto vrcholu odcházejí hrany do vstupních vrcholů dalších konstrukčních prvků, a to na základě původních hran grafu G .

Tento princip zajišťuje, že v převedeném grafu HCIRCUIT je každý vrchol původního grafu reprezentován trojicí, jejíž návštěva je povinná a odpovídá skutečné návštěvě vrcholu ve vstupním grafu.

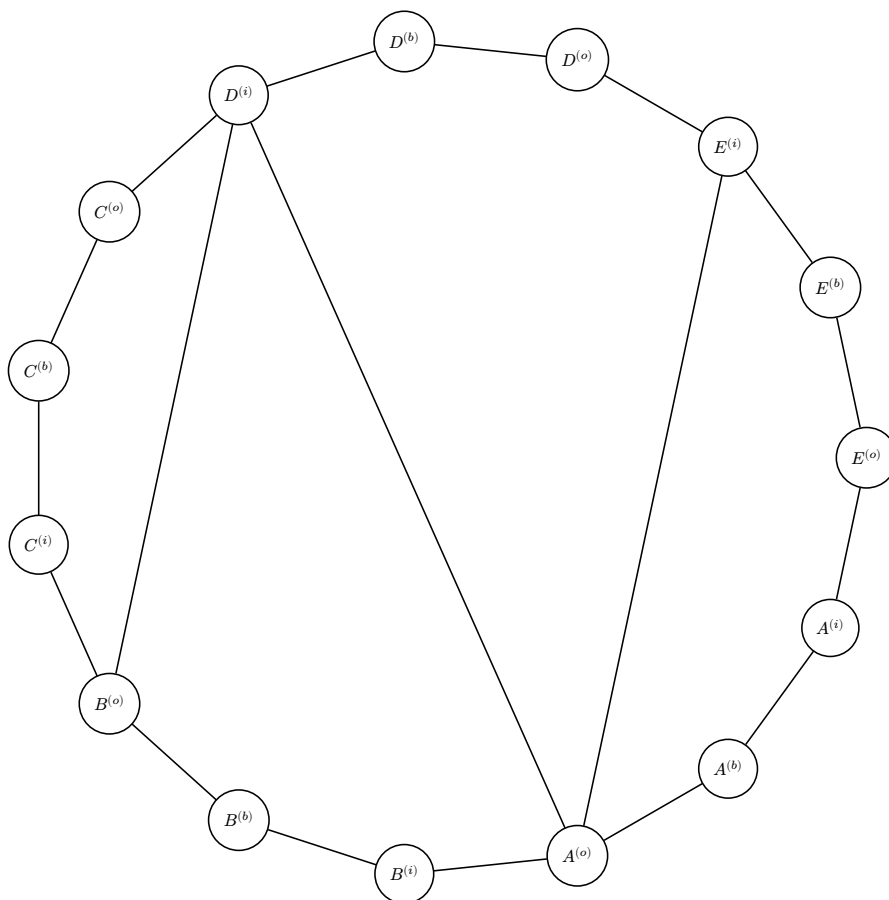
Pro každou hranu $(x, y) \in E(G)$ spojíme výstupní vrchol $x^{(o)}$ se vstupním vrcholem $y^{(i)}$ v novém grafu. Takto se zachovává orientace původního grafu v novém neorientovaném grafu problému HCIRCUIT.

Jako konkrétní příklad uvažujme graf G na obrázku 22:



Obrázek 22: Vstupní graf problému HCYLE

Transformace tohoto grafu do instance problému HCIRCUIT probíhá tak, že každý vrchol je nahrazen trojicí $v^{(i)}, v^{(b)}, v^{(o)}$ a hrany jsou propojeny podle výše uvedeného pravidla. Výsledný graf je zobrazen na obrázku 23.



Obrázek 23: Výstupní graf problému HCIRCUIT

Tento postup zaručuje, že existuje hamiltonovský cyklus ve vstupním grafu HCYCLE právě tehdy, když existuje hamiltonovská kružnice v převedeném grafu HCIRCUIT.

3.5 Redukce HCIRCUIT na TSP

V této redukci převádíme vstupní neorientovaný graf G problému HCIRCUIT na ohodnocený neorientovaný graf H problému TSP.

Všechny vrcholy z grafu G zkopírujeme do nového grafu H . Mezi každou dvojici vrcholů v H přidáme hranu, čímž vznikne kompletní graf.

Každé hraně $e = \{x, y\} \in E(H)$ přiřadíme váhu podle pravidla:

- Pokud hrana e existuje ve vstupním grafu, $\{x, y\} \in E(G)$, nastavíme její váhu na 1.
- Pokud hrana e ve vstupním grafu neexistuje, nastavíme její váhu na hodnotu větší než 1, v našem případě 2.

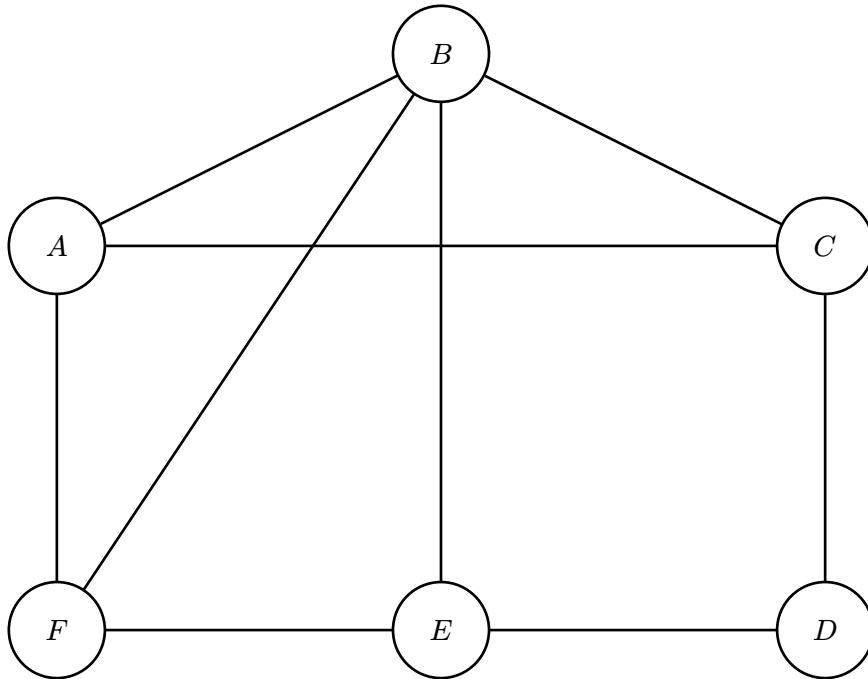
Cílovou hodnotu k pro problém TSP stanovíme rovnou počtu vrcholů grafu:

$$k = |V(G)| = |V(H)|.$$

Tento postup zajišťuje, že ve vstupním grafu G existuje hamiltonovský cyklus právě tehdy, pokud v kompletním grafu H existuje hamiltonovský cyklus s celkovou cenou k .

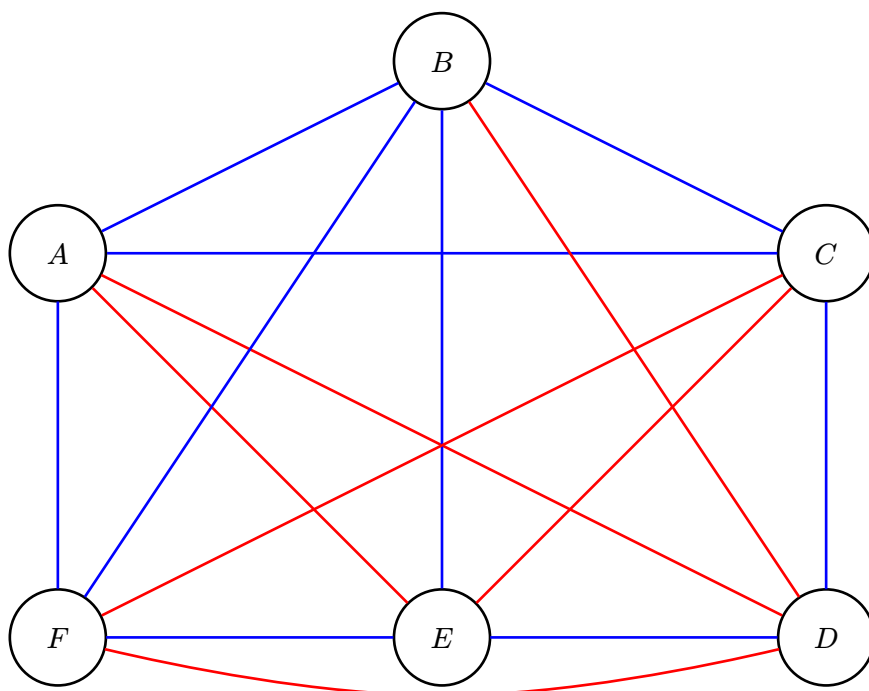
Redukce je korektní, protože jakákoli hamiltonovská cesta s celkovou cenou k ve výsledném grafu H musí využívat pouze hrany s vahou 1, tedy přesně ty, které existují ve vstupním grafu G .

Jako příklad uvažujme graf G na obrázku 24.



Obrázek 24: Vstupní graf problému HCIRCUIT

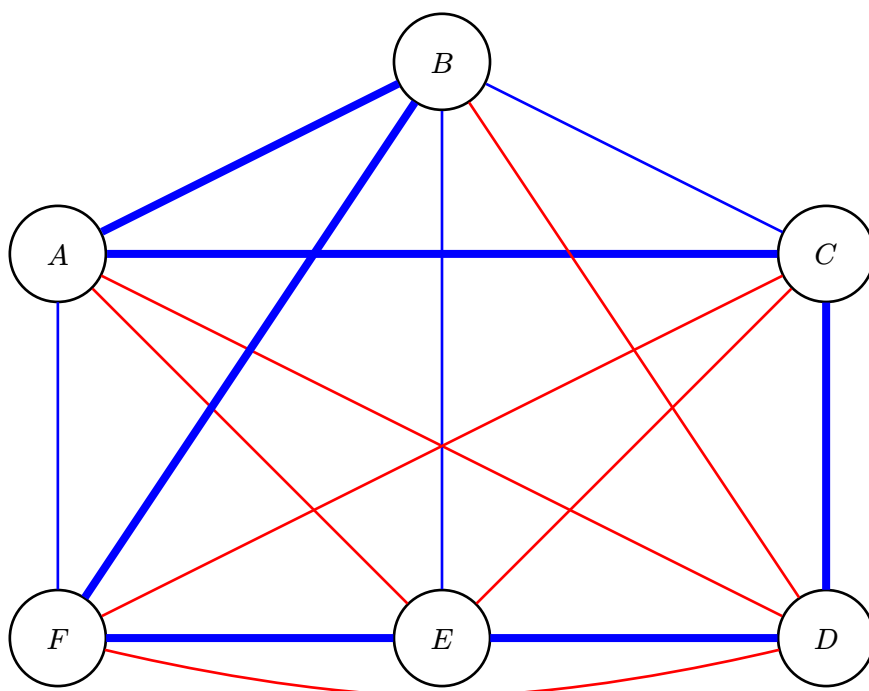
Graf G převedeme podle popsané redukce na graf H , který je znázorněn na obrázku 25. Graf H je kompletní graf nad stejnou množinou vrcholů. Hrany, které existovaly již v grafu G , mají váhu 1 (značeny modře), zatímco nově přidané hrany mají váhu 2 (značeny červeně).



Obrázek 25: Výsledný graf H – modře vyznačené hrany váhy 1, červeně váhy 2

Na obrázku 26 je vyznačen hamiltonovský cyklus nalezený v grafu H . Zvýrazněné hrany tvoří cyklus délky $k = 6$, tedy rovné počtu vrcholů grafu.

Protože cyklus používá pouze hrany s váhou 1, odpovídá tento cyklus přímo hamiltonovskému cyklu ve vstupním grafu G .



Obrázek 26: Hamiltonovský cyklus v grafu H s váhou $k = 6$

Kapitola 4.

Návrh webové aplikace

Cílem navrhované webové aplikace je převést teoretické poznatky z oblasti výpočetní složitosti, konkrétně problematiku polynomiálních redukcí mezi NP-úplnými problémy, do podoby interaktivního výukového nástroje.

Navržený systém umožňuje nejen pasivní studium formálních důkazů, ale především aktivní práci s konkrétními instancemi problémů. Uživatel může zadávat vlastní vstupy, sledovat konstrukci redukce krok za krokem a vizuálně porovnávat vstupní a výslednou instanci včetně jejich odpovědí.

Tím dochází k propojení formální matematické teorie s názornou vizualizací, což přispívá k hlubšímu porozumění probírané problematice.

4.1. Požadavky na systém

Požadavky na systém lze rozdělit na funkční a nefunkční. Funkční požadavky definují, jaké operace musí aplikace umožňovat, zatímco nefunkční požadavky specifikují její kvalitativní vlastnosti.

4.1.1. Funkční požadavky

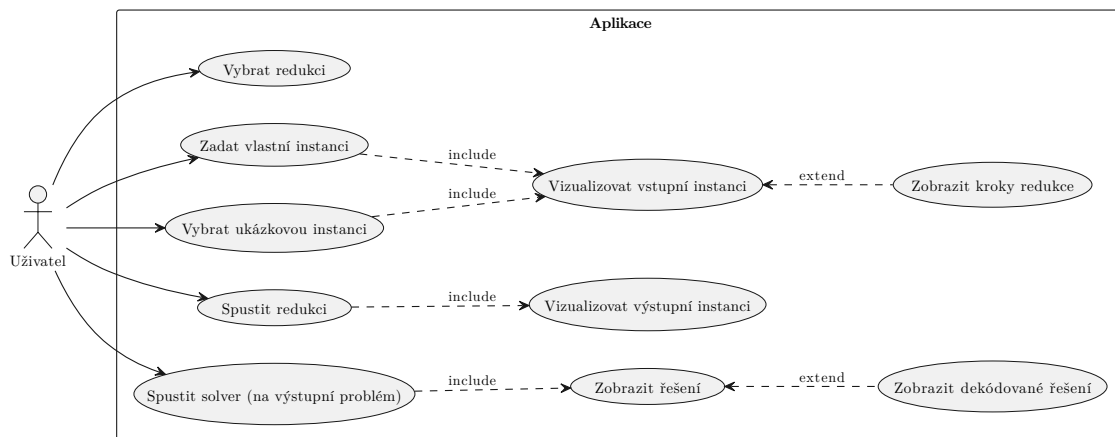
Aplikace musí uživateli umožnit výběr konkrétní redukce mezi problémy a zadání libovolné instance vstupního problému, a to jak formou vlastního vstupu, tak prostřednictvím předdefinovaných ukázkových instancí.

Na základě zadané instance systém provede její transformaci na instanci cílového problému. Vstupní i výsledná instance jsou následně vizualizovány, aby bylo možné jejich přímé porovnání.

Součástí funkcionality je rovněž možnost zobrazit proces redukce krokově, včetně textového vysvětlení jednotlivých transformačních kroků.

V případě, že je odpověď na rozhodovací otázku kladná, aplikace zobrazí nalezené řešení jak pro vstupní, tak pro cílový problém.

Systém dále validuje vstupní data a upozorňuje uživatele na případné syntaktické chyby. Uživatel může upravovat již zadanou instanci bez nutnosti opětovného načtení aplikace.



Obrázek 27: Diagram případů užití zobrazující funkční požadavky systému z pohledu uživatele

4.1.2. Nefunkční požadavky

Nefunkční požadavky se zaměřují na kvalitu uživatelského rozhraní, výkon aplikace a její rozšiřitelnost.

Aplikace musí zajistit plynulou odezvu bez znatelného zpoždění, a to i při práci s rozsáhlejšími instancemi. Důraz je kladen také na přehlednost vizualizace, zejména na zvýraznění konstrukčních prvků v cílových instancích.

Uživatel musí mít možnost interaktivně manipulovat se vstupním grafem u problémů, kde je vstupní instancí graf.

4.2. Architektura systému

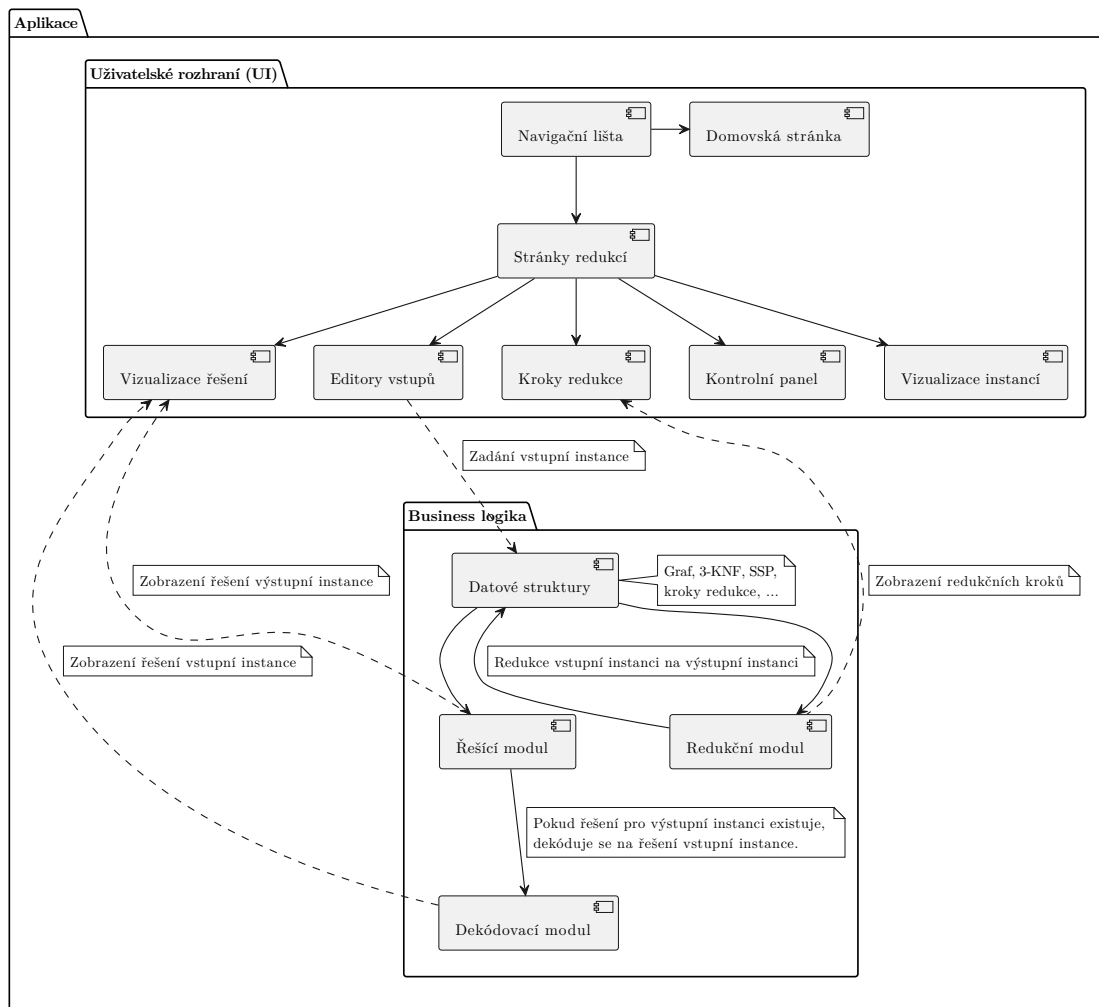
4.2.1. Architektonický model

Aplikace je navržena jako čistě klientská webová aplikace bez serverové části. Veškerá aplikační logika je vykonávána na straně klienta v prostředí webového prohlížeče. Aplikace je distribuována ve formě statických souborů.

Aplikace je rozdělena na uživatelské rozhraní (dále jen UI) a business logiku.

Kód UI zahrnuje komponenty pro domovskou stránku, jednotlivé stránky redukci, navigační lištu, editory vstupních instancí, kontrolní panel, grafickou reprezentaci vstupních a výstupních instancí, grafickou reprezentaci řešení a karty pro jednotlivé kroky redukce. UI aplikace je dále rozebráno v kapitole 4.3.

Kód business logiky se skládá z datových struktur reprezentujících instance jednotlivých problémů a z modulů. V této aplikaci se vyskytují tři typy modulů: redukční (reducer), řešící (solver) a dekódovací (decoder). Redukční modul převádí instance jednoho problému na instance jiného problému. Řešící modul se pokouší nalézt řešení zadané instance. Dekódovací modul převádí nalezené řešení výstupního problému na řešení vstupního problému, pokud je řešení v očekávaném formátu.

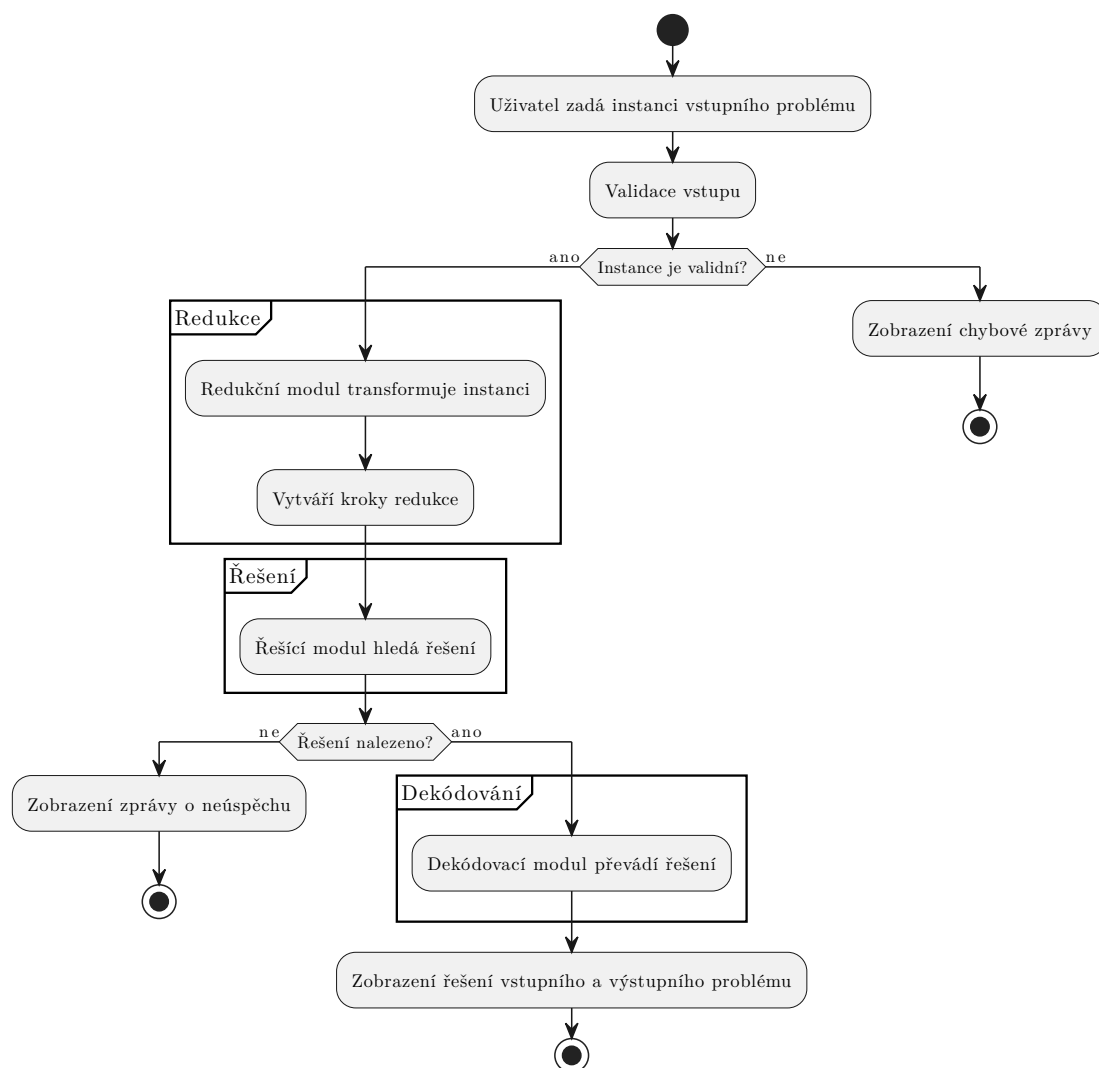


Obrázek 28: Diagram komponent aplikace

4.2.2. Zpracování dat

Zpracování dat v systému probíhá v následujících krocích:

1. Uživatel zadá instanci vstupního problému.
2. Instance je převedena do interní reprezentace systému.
3. Redukční modul transformuje vstupní instanci na instanci cílového problému a vytváří krokový průběh redukce.
4. Řešící modul se pokusí nalézt řešení cílové instance.
5. V případě úspěchu dekódovací modul převede nalezené řešení zpět na řešení původní instance.



Obrázek 29: Vývojový diagram toku dat

4.3. Návrh uživatelského rozhraní

Tato kapitola se věnuje návrhu uživatelského rozhraní aplikace. Nejprve je popsána celková struktura rozhraní a následně jednotlivé komponenty.

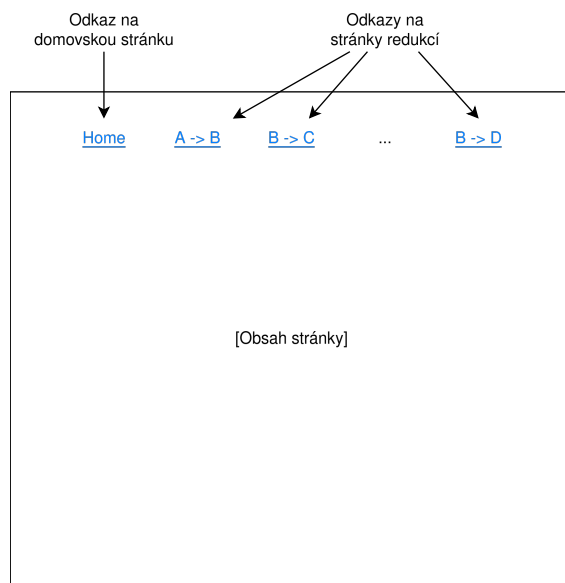
4.3.1. Struktura rozhraní

Webová aplikace má navigační lištu, v níž jsou umístěny odkazy na domovskou stránku a na jednotlivé stránky redukci. Z navigační lišty má uživatel přístup ke všem součástem aplikace. Domovská stránka obsahuje obecné informace o aplikaci a stručné definice problémů vyskytujících se v převodech.

Každá stránka redukce se skládá ze tří hlavních částí:

- editační oblasti pro zadávání vstupní instance,
- vizualizace vstupní instance a
- vizualizace výstupní instance.

Toto rozdělení umožňuje uživateli sledovat celý proces převodu instance z jednoho problému na jiný.



Obrázek 30: Wireframe model struktury rozhraní

4.3.2. Komponenty stránky redukce

Následující podkapitoly podrobněji popisují jednotlivé komponenty stránky redukce.

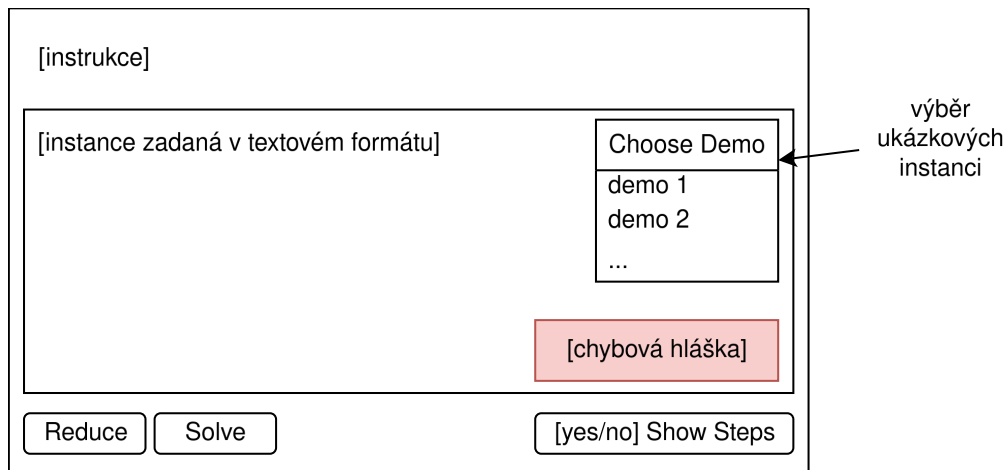
4.3.2.1. Editační oblast

Editační oblast slouží k zadávání vstupní instance. Obsahuje textové vstupní pole a kontrolní panel. V horní části se nachází výběr ukázkových instancí. Při syntaktické chybě vstupu je uživatel informován chybovou hláškou.

Kontrolní panel obsahuje tři hlavní prvky. Tlačítko „Reduce“ zahajuje převod vstupní instance na výstupní instanci. Tlačítko „Solve“ spouští hledání řešení výstupní instance

a v případě úspěchu provádí dekódování řešení zpět na řešení vstupního problému. Přepínač „Show Steps“ zobrazuje jednotlivé kroky redukce.

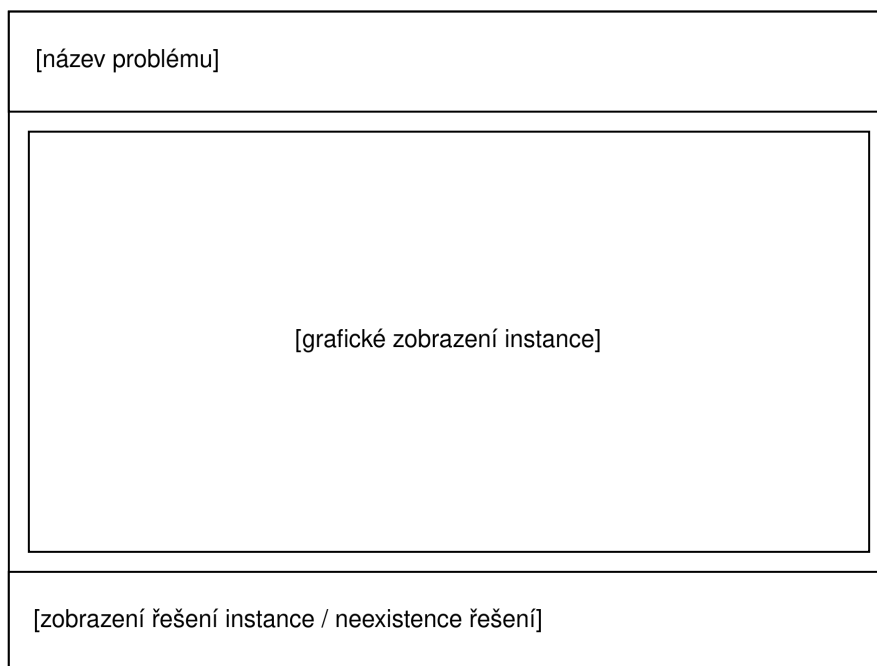
Editor dále obsahuje instrukce a nápovědu k formátu vstupních dat, aby uživatel věděl, jak správně zadat instanci problému.



Obrázek 31: Wireframe model editační oblasti

4.3.2.2. Vizualizace instance

Vizualizační oblast zobrazuje grafickou reprezentaci instance. Obě vizualizace, vstupní i výstupní instance, mají shodné rozložení. Hlavička obsahuje název problému. Hlavní část obsahuje grafickou reprezentaci instance. Zápatí obsahuje nalezené řešení nebo informaci o jeho neexistenci. Zadaná instance je průběžně vizualizována, což umožňuje okamžitou kontrolu správnosti vstupu.



Obrázek 32: Wireframe model vizualizace instance

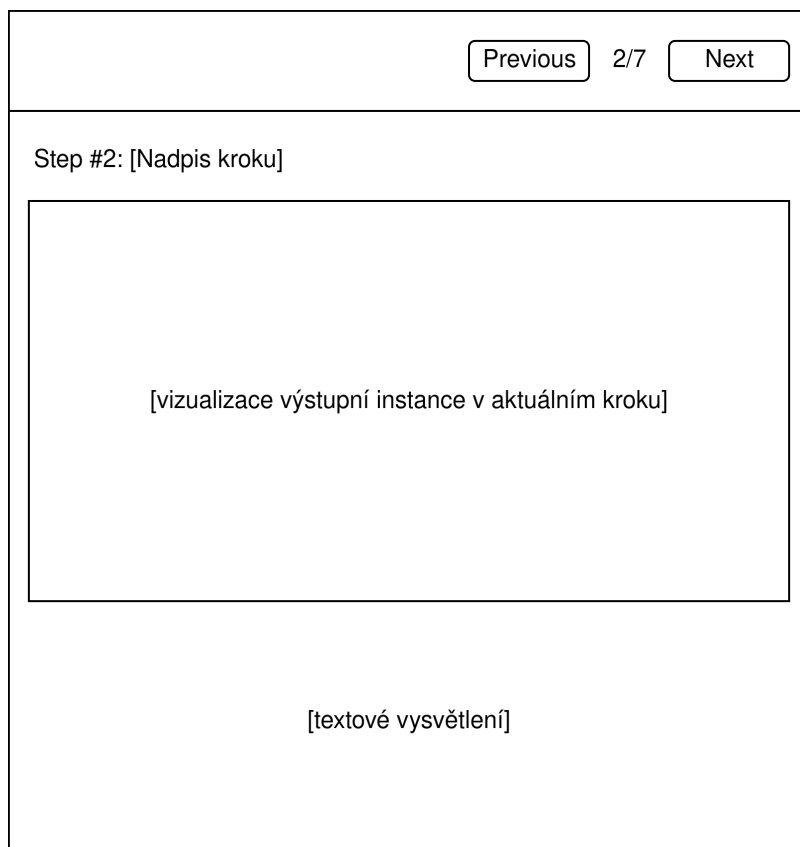
4.3.2.3. Zobrazení řešení

Po aktivaci funkce „Solve“ se aplikace pokusí nalézt řešení cílového problému. V případě úspěchu je nalezené řešení zobrazeno a následně dekodováno na řešení vstupního problému, které je rovněž prezentováno uživateli. Obě řešení jsou zobrazena v zápatí příslušných sekcí a v případě vhodnosti jsou také graficky zvýrazněna.

U problému obchodního cestujícího jsou zvýrazněny hrany Hamiltonovského cyklu. U problému barvení grafu jsou vrcholy obarveny třemi barvami. U problému 3-SAT je řešení prezentováno textově.

4.3.2.4. Krokové zobrazení redukce

Přepínač „Show Steps“ umožňuje zobrazit jednotlivé kroky redukce. Proces redukce je prezentován sekvenčně, přičemž každý krok transformace je graficky znázorněn a doplněn textovým vysvětlením. Uživatel může mezi jednotlivými kroky přecházet pomocí tlačítek „Next“ a „Previous“. Tento přístup umožňuje uživateli pochopit samotný konstrukční postup.



Obrázek 33: Wireframe model krokového zobrazení redukce

Kapitola 5

Implementace

Tato kapitola se věnuje implementaci navrženého systému. Nejprve je popsána vnitřní reprezentace instancí problémů a validace vstupních dat. Následně je detailně rozebrána implementace jednotlivých redukcí. Kapitola dále popisuje, jakým způsobem systém řeší problémy a dekoduje řešení.

Cílem bylo vytvořit přehledný a srozumitelný kód, který co nejpřesněji reflektuje teoretický popis redukcí. Optimalizace výkonu byly prováděny pouze v nezbytných případech.

5.1 Reprezentace instancí problémů

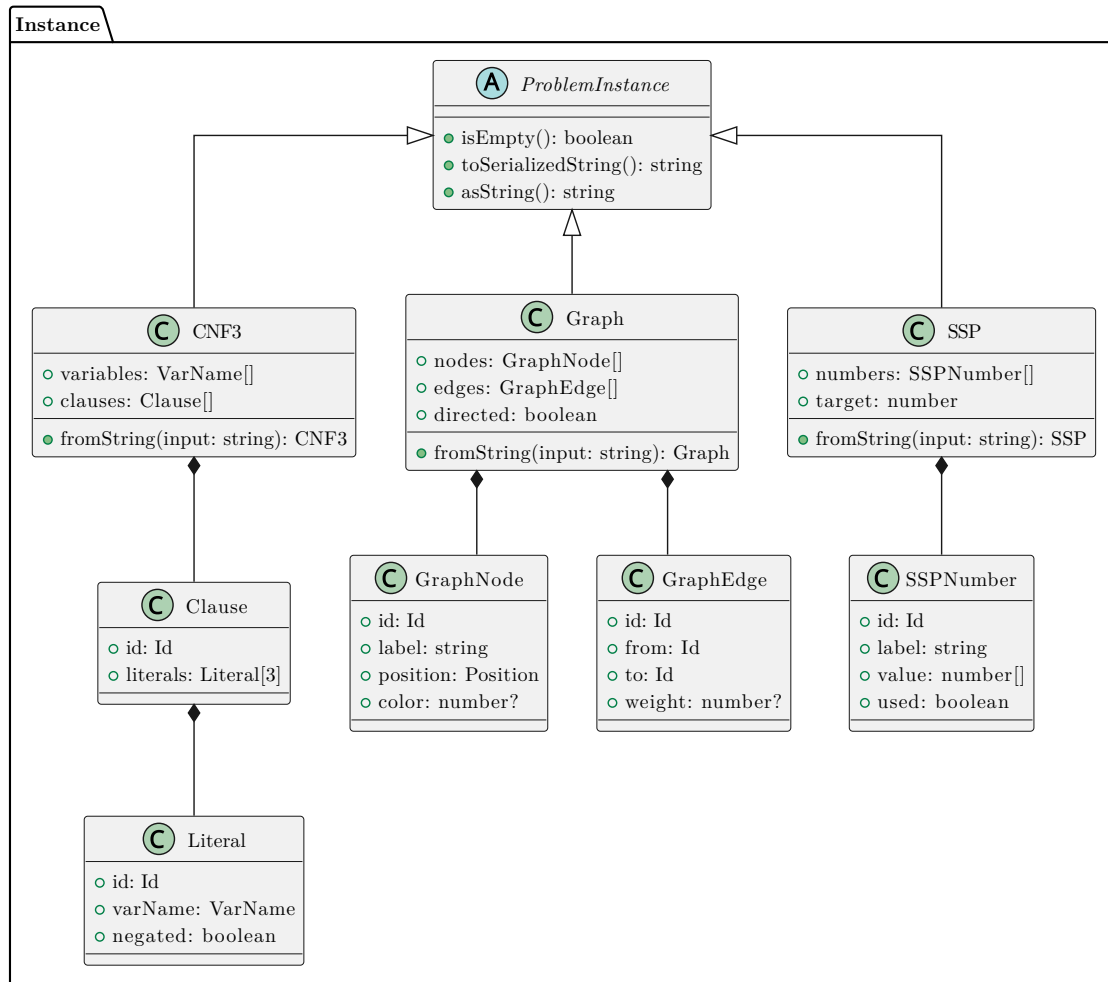
Interní reprezentace instancí je založena na abstraktní třídě `ProblemInstance`, která definuje společné rozhraní pro všechny typy problémů. Každá konkrétní reprezentace rozšiřuje tuto abstraktní třídu a implementuje specifické metody pro práci s daným typem problému.

```
1  abstract class ProblemInstance {
2      /*
3       * Problem specific. Each problem has an empty state.
4       */
5      public abstract isEmpty(): boolean;
6
7      /*
8       * Serializes the problem instance.
9       *
10      * Each child class has a corresponding:
11      * public static fromSerializedString(serialized: string): T;
12      */
13      public abstract toSerializedString(): string;
14
15      /*
16      * Formats the instance into a format that
17      * the editor component accepts.
18      */
19      public abstract asString(): string;
20 }
```

Výpis 1: Abstraktní třída `ProblemInstance`

Toto řešení umožňuje jednotné zpracování různých typů problémů v rámci systému redukci, přičemž každá instance si zachovává specifické vlastnosti svého typu.

Následující diagram znázorňuje třídní hierarchii použitou pro reprezentaci instancí problémů.



Obrázek 34: Třídní hierarchie datových struktur

5.1.1 Reprezentace formule 3-SAT

Booleovská formule v 3-KNF je reprezentována třídou `CNF3`. Tato třída obsahuje množinu proměnných a množinu klauzulí. Každá klauzule je reprezentována třídou `Clause`, která obsahuje trojici literálů. Literál je reprezentován třídou `Literal`, jež obsahuje název proměnné a příznak negace.

Třída `CNF3` poskytuje statickou metodu `fromString` pro parsování textového vstupu. Tato metoda očekává vstup ve formě víceřádkového textu, kde každý řádek reprezentuje jednu klauzuli. Literály jsou odděleny mezerou a negace je označena prefixem `!` (vykřičník).

Následující příklad ukazuje vstupní formát a odpovídající matematickou reprezentaci:

```

1  a !b c
2  !x y z
3  x !b !c
4  a y z

```

Výpis 2: Příklad vstupního formátu

Matematická reprezentace:

$$(a \vee \neg b \vee c) \wedge (\neg x \vee y \vee z) \wedge (x \vee \neg b \vee \neg c) \wedge (a \vee y \vee z)$$

Je také možné zadávat literály ve zjednodušené⁴ syntaxi TeX:

```

1  \alpha \beta \gamma
2  \alpha_1 \alpha_2 \beta_3

```

Výpis 3: Příklad vstupu s TeX názvy

Matematická reprezentace:

$$(\alpha \vee \beta \vee \gamma) \wedge (\alpha_1 \vee \alpha_2 \vee \beta_3)$$

5.1.2 Reprezentace grafů

Grafové problémy využívají třídu `Graph`, která poskytuje reprezentaci orientovaných i neorientovaných grafů. Graf je reprezentován jako množina vrcholů a množina hran. Každý vrchol obsahuje identifikátor, popis, pozici pro vizualizaci a volitelnou barvu. Hrana obsahuje identifikátor, počáteční vrchol, cílový vrchol a volitelnou váhu.

Pro parsování vstupu poskytuje třída `Graph` statickou metodu `fromString`. Vstupní formát umožňuje specifikovat vrcholy a hrany. Jednotlivé řádky mohou obsahovat:

- pouze název vrcholu pro definici izolovaného vrcholu,
- dva názvy vrcholů oddělené mezerou pro definici hrany,
- dva názvy vrcholů a celé číslo pro definici hrany s váhou.

Následující příklad ukazuje vstupní formát bez ohodnocení hran:

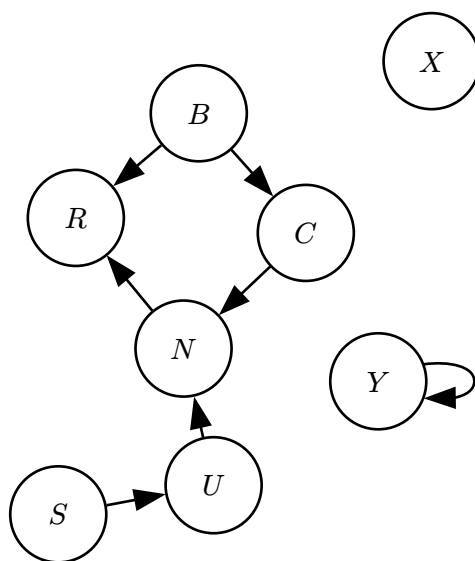
```

1  X
2  Y Y
3  S U
4  U N
5  B C
6  C N
7  N R
8  B R

```

Výpis 4: Příklad vstupu grafu bez ohodnocení hran

⁴Validator pouze kontroluje, že vstup obsahuje povolené znaky. Nejedná se o plnohodnotnou podporu TeX – systém pouze kontroluje, že text obsahuje pouze alfanumerické a povolené speciální znaky jako `\`, `_`, `{`, `}`, `(`, `)`. Jelikož mezi povolenými znaky není znak `^`, superskripty nejsou podporovány.

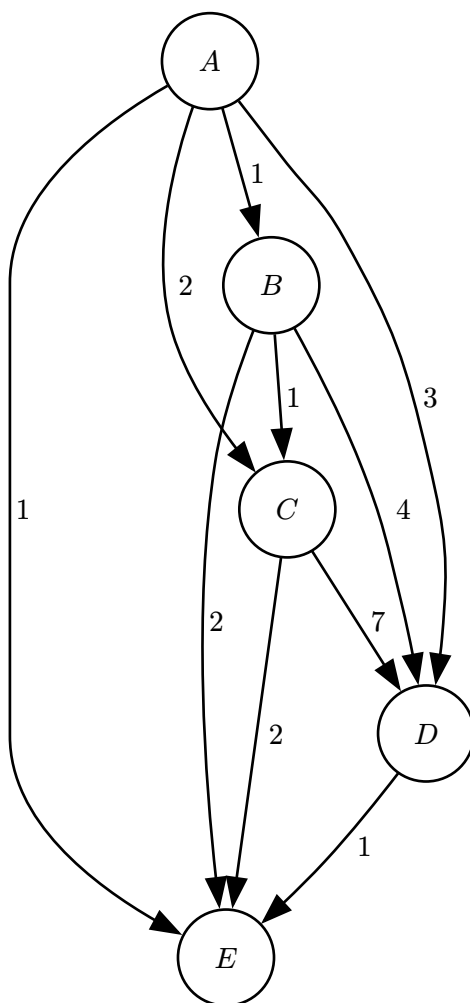


Obrázek 35: Vizualizace grafu bez ohodnocení hran

Následující příklad ukazuje vstupní formát s ohodnocením hran:

```
1 A B 1
2 A C 2
3 A D 3
4 A E 1
5 B C 1
6 B D 4
7 B E 2
8 C D 7
9 C E 2
10 D E 1
```

Výpis 5: Příklad vstupu grafu s ohodnocením hran



Obrázek 36: Vizualizace grafu s ohodnocením hran

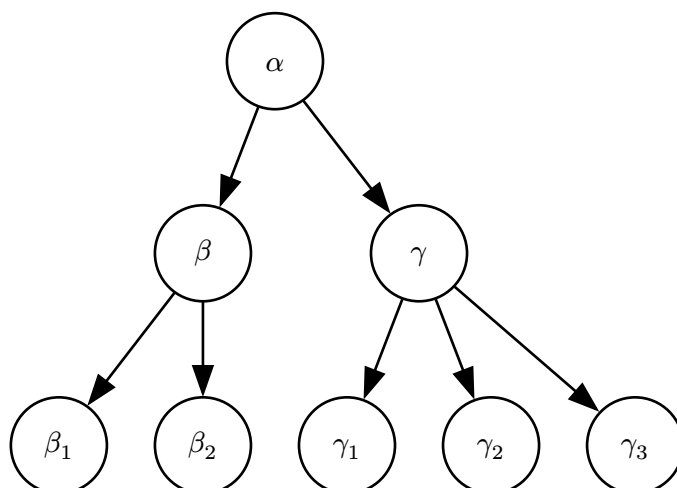
Je také možné zadávat názvy vrcholů ve zjednodušené syntaxi TeX:

```

1 \alpha \beta
2 \alpha \gamma
3 \beta \beta_1
4 \beta \beta_2
5 \gamma \gamma_1
6 \gamma \gamma_2
7 \gamma \gamma_3

```

Výpis 6: Příklad vstupu grafu s TeX názvy



Obrázek 37: Vizualizace grafu s TeX názvy

5.1.3 Reprezentace problému SSP

Problém podmnožinového součtu je reprezentován třídou `SSP`. Tato třída obsahuje seznam čísel a cílovou hodnotu. Každé číslo je reprezentováno jako pole cifer, což umožňuje práci s velkými čísly, která by přesáhla maximální hodnotu standardního celočíselného typu.

Parsování vstupu funguje následovně:

- Každý řádek vstupu reprezentuje jedno číslo.
- První řádek obsahuje cílovou hodnotu.
- Následující řádky obsahují jednotlivá čísla množiny.

Následující příklad ukazuje vstupní formát a odpovídající matematickou reprezentaci:

```

1  11111133
2
3  10000010
4  10000000
5  1000010
6  1000000
7  100010
8  100000
9  10001
10 10000
11 1001
12 1000
13 101
14 100
15 10
16 10
17 1
18 1
  
```

Výpis 7: Příklad vstupu SSP

Matematická reprezentace:

$$\begin{aligned}\tau &= 11111133 \\ S &= \{10000010, 10000000, 1000010, 1000000, 100010, 100000, \\ &\quad 10001, 10000, 1001, 1000, 101, 100, 10, 10, 1, 1\}\end{aligned}$$

5.1.4 Validace vstupu

Validace vstupních dat probíhá ve dvou fázích. První fáze kontroluje syntaktickou správnost vstupu během parsování. V této fázi jsou odmítnuty vstupy obsahující neplatné znaky nebo porušující základní syntaktická pravidla.

Druhá fáze validace probíhá po úspěšném parsování vstupu prostřednictvím metody `isEmpty`. Tato metoda kontroluje, zda instance obsahuje všechny potřebné prvky. Například pro problém 3-SAT kontroluje, zda formule obsahuje alespoň jednu proměnnou a jednu klauzuli. Tato fáze je důležitá zejména v redukčním modulu, kde zajišťuje validitu instancí před dalším zpracováním.

Chybové zprávy jsou vráceny jako typ `ErrorMessage`, který v případě chyby obsahuje textový popis problému.

5.2 Implementace redukci

Každá redukce je implementována jako samostatná třída rozšiřující abstraktní třídu `Reducer`. Třída `Reducer` definuje společné rozhraní pro všechny redukce:

```
1  abstract class Reducer<
2      I extends ProblemInstance,
3      O extends ProblemInstance,
4  > {
5      constructor(public inInstance: I) {}
6
7      public reduce(): ReductionResult<I, O> {
8          if (this.inInstance.isEmpty()) {
9              throw "Call to reduce failed. Input instance is empty.";
10         }
11         const result = this.doReduce();
12         return result;
13     }
14
15     /*
16     * Implemented by derived classes.
17     */
18     protected abstract doReduce(): ReductionResult<I, O>;
19 }
```

Výpis 8: Abstraktní třída `Reducer`

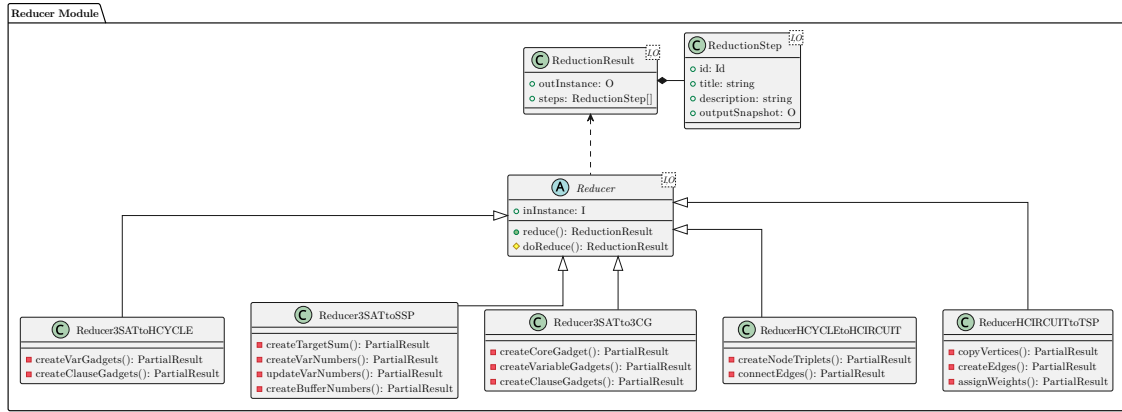
Metoda `reduce` kontroluje, zda vstupní instance není prázdná, a následně volá abstraktní metodu `doReduce`, která je implementována v každé konkrétní redukci.

Výsledkem redukce je datová struktura `ReductionResult` obsahující výstupní instanci a seznam kroků redukce. Každý krok redukce je reprezentován strukturou `ReductionStep`, která obsahuje:

- jedinečný identifikátor kroku,
- nadpis popisující prováděnou operaci,
- podrobný popis v HTML formátu a
- snímek (snapshot) výstupní instance v aktuálním kroku.

Tato struktura umožňuje zpětnou rekonstrukci procesu redukce a vizualizaci jednotlivých kroků.

Následující diagram znázorňuje třídní hierarchii redukci.



Obrázek 38: Třídní hierarchie redukcí

5.2.1 Redukce 3-SAT na HCYCLE

Implementace redukce `Reducer3SATtoHCYCLE` konstruuje orientovaný graf z konstrukčních prvků pro proměnné a klauzule vstupní formule. Metoda `doReduce` volá postupně dvě pomocné metody:

1. `createVarGadgets` vytváří pro každou proměnnou formule řadu vrcholů reprezentující konstrukční prvek proměnné. Součástí je vytvoření mezi-vrcholů, které spojují jednotlivé řady proměnných do jednoho cyklu.
2. `createClauseGadgets` vytváří pro každou klauzuli jediný vrchol. Na základě literálů v klauzuli jsou přidány hrany mezi vrcholem klauzule a příslušnými vrcholy v řadách proměnných. Směr připojení hran závisí na tom, zda je literál negován či nikoli.

Podrobnosti tohoto převodu jsou popsány v kapitole 3.1.

5.2.2 Redukce 3-SAT na SSP

Implementace redukce `Reducer3SATtoSSP` využívá maticovou reprezentaci čísel. Metoda `doReduce` volá postupně čtyři pomocné metody:

1. `createTargetSum` nastaví cílovou hodnotu τ , která se skládá z v jedniček a c trojek, kde v je počet proměnných a c počet klauzulí.
2. `createVarNumbers` vytváří dvojici čísel $x^{(T)}$ a $x^{(F)}$ pro každou proměnnou x . Diagonála matice je nastavena na hodnotu 1, což zajišťuje, že v konečné podmnožině může být zvolena buď $x^{(T)}$ nebo $x^{(F)}$, nikoli obě.
3. `updateVarNumbers` aktualizuje čísla proměnných na základě struktury klauzulí. Pro každý výskyt proměnné v klauzuli je příslušná pozice v matici nastavena na 1.
4. `createBufferNumbers` přidává dvojici vyrovnávacích čísel pro každou klauzuli. Tato čísla umožňují dosáhnout cílové hodnoty 3 v příslušné pozici, pokud je klauzule splněna.

Podrobnosti tohoto převodu jsou popsány v kapitole 3.2.

5.2.3 Redukce 3-SAT na 3-CG

Implementace redukce `Reducer3SATto3CG` konstruuje neorientovaný graf z jádrového prvku, prvků proměnných a prvků klauzulí. Metoda `doReduce` volá postupně tři pomocné metody:

1. `createCoreGadget` vytváří základní trojici vzájemně propojených vrcholů T , F a B , které reprezentují pravdu, nepravdu a vyrovnávací barvu.
2. `createVariableGadgets` vytváří pro každou proměnnou trojici vrcholů ν , $\neg\nu$ a B . Vrcholy ν a $\neg\nu$ jsou spojeny s vrcholem B , což zajišťuje, že mohou být obarveny pouze barvami pravdy a nepravdy.
3. `createClauseGadgets` vytváří pro každou klauzuli šest vrcholů a příslušné hrany. Klauzule je připojena k vrcholům odpovídajícím jejím literálům.

Podrobnosti tohoto převodu jsou popsány v kapitole 3.3.

5.2.4 Redukce HCYCLE na HCIRCUIT

Implementace redukce `ReducerHCYCLEtoHCIRCUIT` převádí orientovaný graf na neorientovaný. Metoda `doReduce` volá postupně dvě pomocné metody:

1. `createNodeTriplets` vytváří pro každý vrchol orientovaného grafu trojici vrcholů neorientovaného grafu: v_i pro vstup, v_b jako propojovací uzel a v_o pro výstup. Tyto tři vrcholy jsou spojeny do řetězce.
2. `connectEdges` propojuje výstupní vrcholy zdrojových uzlů se vstupními vrcholy cílových uzlů podle hran původního grafu.

Podrobnosti tohoto převodu jsou popsány v kapitole 3.4.

5.2.5 Redukce HCIRCUIT na TSP

Implementace redukce `ReducerHCIRCUITtoTSP` převádí neorientovaný graf na ohodnocený úplný graf. Metoda `doReduce` volá postupně tři pomocné metody:

1. `copyVertices` přenáší všechny vrcholy ze vstupní instance `HCIRCUIT` do nového grafu.
2. `createEdges` přidává hrany mezi každou dvojicí vrcholů, čímž vzniká úplný graf.
3. `assignWeights` přiřazuje váhu 1 hranám, které odpovídají hranám původního grafu `HCIRCUIT`, a váhu 2 ostatním hranám. Cílová hodnota k je nastavena na počet vrcholů grafu.

Podrobnosti tohoto převodu jsou popsány v kapitole 3.5.

5.2.6 Ukládání kroků redukce

Každá implementace redukce ukládá mezikroky do seznamu `ReductionStep[]`. Každý krok obsahuje popis prováděné operace v kombinaci HTML a TeX, který je následně vykreslen pomocí knihovny KaTeX. Pro vizualizaci změn mezi kroky jsou pořizovány snímky instancí pomocí metody `copy` příslušné třídy.

5.3 Řešení problémů

Pro nalezení odpovědi na rozhodovací otázku je využito rozhraní `Solver`, které definuje metodu `solve` vracející buď certifikát řešení, nebo konstantu `Unsolvable`.

Každý solver je navržen jako samostatná třída implementující toto rozhraní:

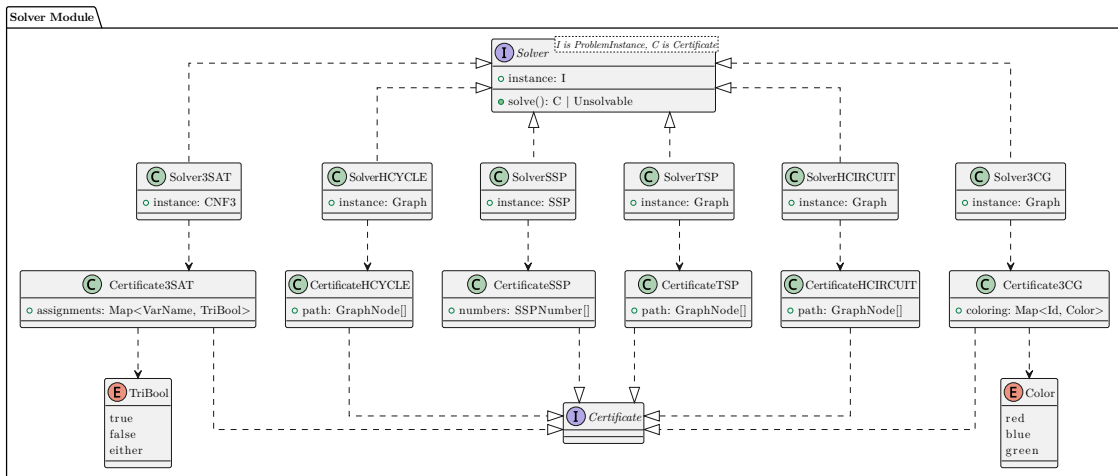
```

1 interface Solver<
2     I extends ProblemInstance,
3     C extends Certificate
4 > {
5     instance: I;
6     solve(): C | Unsolvable;
7 }

```

Výpis 9: Rozhraní `Solver`

Následující diagram znázorňuje třídní hierarchii solverů a certifikátů.



Obrázek 39: Třídní hierarchie solverů a certifikátů

5.3.1 Řešení problému 3-SAT

Solver `Solver3SAT` využívá algoritmus zpětného vyhledávání DPLL⁵.

5.3.2 Řešení problému HCYCLE

Solver `SolverHCYCLE` využívá backtrackingový algoritmus procházející možné orientované cesty v grafu.

Algoritmus nejprve sestaví seznam sousednosti pro rychlé vyhledávání. Počáteční vrchol je libovolně zvolen jako první vrchol grafu. Algoritmus se rekurzivně pokouší rozšířit aktuální cestu o nepoužité sousední vrcholy. Po návratu z rekurze algoritmus backtrackuje, tj. odstraňuje poslední vrchol z cesty a zkouší jiného souseda. Po úspěšném

⁵https://en.wikipedia.org/wiki/DPLL_algorithm

nalezení cesty obsahující všechny vrcholy algoritmus kontroluje, zda poslední vrchol má hranu zpět do počátečního vrcholu.

5.3.3 Řešení problému HCIRCUIT

Solver `SolverHCIRCUIT` využívá modifikovaný backtrackingový algoritmus pro neorientované grafy. Algoritmus se od řešení `HCYCLE` liší pouze tím, že hrany jsou neorientované, tj. je možné jimi procházet oběma směry.

5.3.4 Řešení problému SSP

Solver `SolverSSP` využívá dynamické programování.

Algoritmus nejprve inicializuje tabulku `dp` mapující dosažitelné součty na indexy použitých čísel. Základní případ nastavuje součet 0 jako dosažitelný. V každé iteraci algoritmus prochází aktuálně dosažitelné součty a zkouší přidat aktuální číslo. Pokud nový součet ještě není dosažitelný, je přidán do tabulky. Po vyčerpání všech čísel algoritmus kontroluje, zda je cílový součet dosažitelný. Pokud ano, rekonstruuje nalezenou podmnožinu zpětným procházením tabulky.

5.3.5 Řešení problému TSP

Solver `SolverTSP` využívá bitmaskové dynamické programování.

Algoritmus nejprve sestaví matici vzdáleností z vah hran grafu. Následně inicializuje tabulku `dp` pro dynamické programování, přičemž `dp[mask][last]` reprezentuje minimální cenu cesty procházející vrcholy specifikované maskou `mask` a končící ve vrcholu `last`.

Algoritmus iterativně rozšiřuje cesty přidáváním nových vrcholů. Po naplnění tabulky nalezne minimální cenu uzavřené cesty a rekonstruuje nalezenou cestu pomocí tabulky rodičů.

5.3.6 Řešení problému 3-CG

Solver `Solver3CG` využívá backtrackingový algoritmus přiřazující barvy vrcholům.

Algoritmus nejprve sestaví seznam sousednosti grafu. Následně rekurzivně prochází vrcholy a pro každý vrchol zkouší tři barvy. Před přiřazením barvy kontroluje, zda žádný sousední vrchol nemá stejnou barvu.

Po neúspěchu algoritmus backtrakuje a zkouší jinou barvu.

5.4 Dekódování řešení

Dekódování řešení slouží k převodu certifikátu výstupní instance zpět na certifikát vstupní instance. Každý dekoder implementuje rozhraní `Decoder` definované jako:

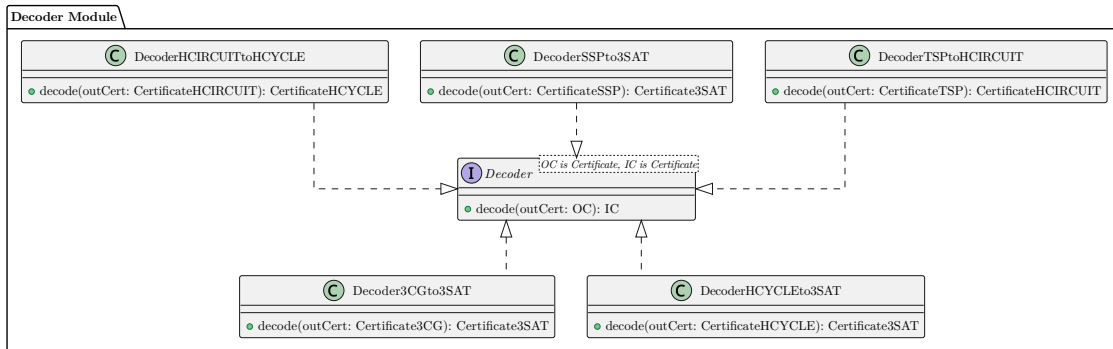
```

1 interface Decoder<
2     OC extends Certificate,
3     IC extends Certificate
4 > {
5     decode(outCert: OC): IC;
6 }

```

Výpis 10: Rozhraní `Decoder`

Následující diagram znázorňuje třídní hierarchii dekoderů.



Obrázek 40: Třídní hierarchie dekoderů

5.4.1 Dekódování řešení HCYCLE na 3-SAT

Dekoder `DecoderHCYCLEto3SAT` analyzuje nalezený hamiltonovský cyklus a rekonstruuje ohodnocení booleovských proměnných.

Algoritmus prochází vrcholy v nalezené cestě. Pokud identifikátor vrchol začíná prefixem `NODE_ID_PREFIX_TRUE`, je příslušná proměnná nastavena na hodnotu pravda. Naopak, pokud identifikátor vrchol začíná prefixem `NODE_ID_PREFIX_FALSE`, je příslušná proměnná nastavena na hodnotu nepravda.

Název proměnné je extrahován odstraněním příslušného prefixu a následné části identifikátoru obsahující pořadové číslo vrcholu.

5.4.2 Dekódování řešení SSP na 3-SAT

Dekoder `DecoderSSPto3SAT` analyzuje nalezenou podmnožinu čísel a rekonstruuje ohodnocení booleovských proměnných.

Proměnná je nastavena na pravdu, pokud je v podmnožině přítomno odpovídající číslo $x^{(T)}$. Proměnná je nastavena na nepravdu, pokud je v podmnožině přítomno odpovídající číslo $x^{(F)}$.

5.4.3 Dekódování řešení 3-CG na 3-SAT

Dekoder `Decoder3CGto3SAT` analyzuje nalezené obarvení grafu a rekonstruuje ohodnocení booleovských proměnných.

Proměnná je nastavena na pravdu, pokud je vrchol x obarven zeleně. Proměnná je nastavena na nepravdu, pokud je vrchol x obarven červeně.

5.4.4 Dekódování řešení HCIRCUIT na HCYCLE

Dekoder `DecoderHCIRCUITtoHCYCLE` převádí nalezenou hamiltonovskou kružnici v neorientovaném grafu na hamiltonovský cyklus v orientovaném grafu.

Algoritmus prochází nalezenou cestu, odebírá prefixy identifikátorů vrcholů, které označují typ uzlu (vstupní, výstupní nebo propojovací), a odstraňuje po sobě jdoucí duplicity. Vrcholy, které zbydou, odpovídají hamiltonovskému cyklu v orientovaném grafu.

5.4.5 Dekódování řešení TSP na HCIRCUIT

Dekoder `DecoderTSPtoHCIRCUIT` převádí nalezenou optimální cestu v TSP na hamiltonovskou kružnici v neorientovaném grafu.

Jelikož vrcholy grafů TSP a HCIRCUIT jsou totožné a řešení má shodný formát, dekodování je triviální – nalezená cesta je přímo vrácena jako řešení HCIRCUIT.

5.5 Použité technologie

Aplikace je implementována pomocí frameworku Svelte⁶, který poskytuje komponentově orientovaný přístup k tvorbě uživatelského rozhraní.

Vývoj probíhal v jazyce TypeScript, který rozšiřuje jazyk JavaScript o statické typování. To umožňuje lepší kontrolu nad strukturou dat, zvyšuje čitelnost kódu a snižuje pravděpodobnost vzniku chyb z důsledků dynamického typování.

Pro vizualizaci grafových struktur je využita knihovna Cytoscape⁷, která představuje jedním ze standardních nástrojů pro práci s grafy ve webovém prostředí.

Pro vykreslování matematických výrazů v TeX syntaxi je použita knihovna KaTeX⁸, která převádí TeX zápis do HTML reprezentace vhodné pro zobrazení v prohlížeči.

Pro sestavení aplikace je využit nástroj Vite⁹.

⁶<https://svelte.dev/>

⁷<https://js.cytoscape.org/>

⁸<https://katex.org/>

⁹<https://vite.dev/>

Kapitola 6

Ukázkové instance a demonstrace

Tato kapitola představuje vybrané ukázkové instance pro všechny implementované redukce. Uživatelé mohou samozřejmě zadávat vlastní instance, ale systém jim zároveň umožňuje vybírat z předdefinovaných ukázkových příkladů. Díky tomu se mimo jiné mohou naučit, v jakém formátu editor očekává zadání instancí. Na konkrétních příkladech je demonstrováno, jak aplikace vizualizuje vstupní i výstupní instance po redukci včetně jejich řešení (pokud existují) a jak zobrazuje proces redukce krok za krokem.

Pro každou redukci jsou v kapitole uvedeny dva vybrané příklady¹⁰: jedna s kladnou odpovědí a jedna se zápornou odpovědí. U každé instance je vysvětleno, proč byla zvolena, a jsou u ní uvedeny i snímky obrazovky aplikace.

6.1 Redukce 3-SAT na HCYCLE

6.1.1 Instance s kladnou odpovědí

Jako ukázková instance byla zvolena formule:

$$(x \vee y \vee z) \wedge (\neg x \vee y \vee \neg z) \wedge (x \vee \neg y \vee z)$$

Tato instance byla vybrána z následujících důvodů:

- obsahuje dostatečný počet proměnných pro demonstraci řetězce konstrukčních prvků,
- kombinuje neznegované i negované literály,
- je snadno ověřitelná ručním výpočtem.

¹⁰V samotné aplikaci jsou pro každou redukci k dispozici minimálně 4 ukázkové instance (2 kladné, 2 záporné), přičemž v této kapitole jsou detailněji rozebrány pouze dvě z nich.

Editor

Each line defines a clause. A clause consists of three variable names separated by spaces.
A variable name is a single word containing only letters (a-z, A-Z), digits (0-9) and the symbols `_`, `!`, `\`, `(`, `)`, `{`, `}`.
The symbol `\` must be followed by letters (e.g. `\alpha`, `\beta_{\gamma_5}`).
Variable names must not include spaces.
A variable is negated by prefixing it with `!`.
Any duplicate clauses will be removed automatically.

```
x y z
!x y z
x !y z
```

Yes(2) ▾

Click "Reduce" to reduce the input instance to the output instance.
Click "Solve" to run an solver the output problem instance.
If a solution is found, it is then decoded to the input problem as well.

Reduce

Solve

Shows how the reduction is done step by step.

☐ Show steps

Input 3-SAT Instance

☐ View as column

$$(x \vee y \vee z) \wedge (\neg x \vee y \vee z) \wedge (x \vee \neg y \vee z)$$

Certificate

$$\begin{aligned}
x &= T \\
y &= T \\
z &= T
\end{aligned}$$

Output HCYCLE Instance

Copy to clipboard

Certificate

☐ Show as list

$$\begin{aligned}
P = & (\alpha^{(source)}, x^{(1)}, x^{(2)}, x^{(3)}, x^{(4)}, x^{(5)}, x^{(6)}, x^{(7)}, x^{(8)}, x^{(9)}, x^{(10)}, x^{(11)}, x^{(12)}, (x, y), y^{(1)}, y^{(2)}, y^{(3)}, y^{(4)}, y^{(5)}, y^{(6)}, \\
& y^{(7)}, y^{(8)}, y^{(9)}, y^{(10)}, y^{(11)}, y^{(12)}, (y, z), z^{(1)}, z^{(2)}, z^{(3)}, \kappa^{(1)}, z^{(4)}, z^{(5)}, z^{(6)}, \kappa^{(2)}, z^{(7)}, z^{(8)}, z^{(9)}, \kappa^{(3)}, z^{(10)}, z^{(11)}, z^{(12)}, \\
& \beta^{(target)}, \alpha^{(source)})
\end{aligned}$$

Obrázek 41: Vstupní instance 3-SAT a výsledná instance HCYCLE (kladná odpověď)

Steps

☐ Show all
Showing all steps might take a while for bigger instances. Previous 1/6 Next

Use arrow keys to move to the next and previous steps.

Step #1: Create variable gadgets

Let $\Phi = (\mathcal{V}, \mathcal{K})$ be the input boolean formula, where $\mathcal{V}$ is the set of variables and $\mathcal{K}$ is the set of clauses.

$$\mathcal{V} = \{x, y, z\}$$

$$\mathcal{K} = \{(x, y, z), (\neg x, y, z), (x, \neg y, z)\}$$

For every variable $v \in \mathcal{V}$, we create a row variable gadget $G_v = (V_v, E_v)$. This gadget consists of $k = 12$ row nodes.

$$V_v = \{v^{(1)}, v^{(2)}, \dots, v^{(k-12)}\}$$

They are connected bidirectionally creating a path graph.

$$E_v = \{\{v^{(i)}, v^{(i+1)}\}, \{v^{(i+1)}, v^{(i)}\} \mid 0 \leq i \leq k-1\}$$

There will be $|\mathcal{V}| = 3$ of these variable gadgets.

The number k is derived as follows:

For every clause $\kappa \in \mathcal{K}$, we need 2 nodes - an incoming and outgoing node. Each of these 2 nodes must be padded by at least one pad node, we choose to have 1 pad node. The rows themselves also need two nodes for the *True* and *False* ends.

- $2|\mathcal{K}|$ incoming and outgoing nodes
- $|\mathcal{K}| + 1$ pad nodes
- 2 nodes for *True* and *False* ends

$$k = (2|\mathcal{K}|) + (|\mathcal{K}| + 1) + 2 = 3|\mathcal{K}| + 3$$

$$|\mathcal{K}| = 3$$

$$k = 3 \cdot 3 + 3$$

$$k = 12$$

☐ Show all Previous 1/6 Next

Steps

☐ Show all
Showing all steps might take a while for bigger instances. Previous 2/6 Next

Use arrow keys to move to the next and previous steps.

Step #2: Create inbetween nodes

Create the source node $\alpha^{(source)}$, target node $\beta^{(target)}$ and the inbetween nodes:

$$(x, y), (y, z)$$

that lie between the variable rows.

Connect the source node $\alpha^{(source)}$ to the row ends $x^{(1)}$ and $x^{(12)}$ of the first variable x . Then, connect these row ends to the inbetween or target node below. Repeat for the rest of the graph.

Finally connect the target node $\beta^{(target)}$ to source node $\alpha^{(source)}$ to close the loop.

Why did we do this?

Going from the source node $\alpha^{(source)}$ or an inbetween node $(v^{(i-1)}, v^{(i)})$ to one of the row end nodes of a variable v is equivalent to assigning a boolean value that corresponding variable v .

Here,

- going through the **left edge** means $v = \text{True}$
- and going through the **right edge** means $v = \text{False}$.

Notice that we can only choose going through either the left (*True*), or the right (*False*) edge. Backtracking is impossible.

After going through either the right or left edge, we must visit the row nodes, for the final cycle to be Hamiltonian.

☐ Show all Previous 2/6 Next

Steps

☐ Show all
Showing all steps might take a while for bigger instances. Previous 3/6 Next

Use arrow keys to move to the next and previous steps.

Step #3: Create clause nodes

For every clause $\kappa \in \mathcal{K}$, create one clause node κ' . This node must be visited exactly once. Visiting this node corresponds to the clause κ being satisfied. There are $|\mathcal{K}| = 3$ clause nodes.

For each clause node κ' , representing some clause $\kappa = (X, Y, Z) \in \mathcal{K}$, where X, Y and Z are its literals (they can be negated) and x, y and z are the variables, create edges to and from the variable row gadgets G_x, G_y and G_z as follows:

For each literal X of the clause κ pick a free row node $x^{(i)}$ (one that hasn't been used yet in this step) and connect it to κ' . This selected node is the **outgoing** node. If the literal is negated:

- connect κ' back to row node $x^{(i-1)}$ (adjacent, on the left of $x^{(i)}$),
- otherwise connect κ' back to row node $x^{(i+1)}$ (adjacent, on the right of $x^{(i)}$).

This node, be it $x^{(i-1)}$ or $x^{(i+1)}$, is the **incoming** node.

This way, we guarantee for each clause node κ' that:

- if some literal X of variable x in the clause κ **isn't negated**, then we can reach it from some node $x^{(i)}$ of the variable gadget G_x and come back to $x^{(i+1)}$ (on the right), if we approach it from the left (we assigned x to be *True*).
- Otherwise, the literal X is **negated** and we can reach it from some node $x^{(i)}$ of the variable gadget G_x and come back to $x^{(i-1)}$ (on the left), if we approach it from the right (we assigned x to be *False*).

Note: In the graph, the clause nodes are named $\kappa^{(i)}$, not κ' .

☐ Show all Previous 3/6 Next

Steps

☐ Show all
Showing all steps might take a while for bigger instances. Previous 4/6 Next

Use arrow keys to move to the next and previous steps.

Step #4: Connect clause node κ_1 to variable row nodes

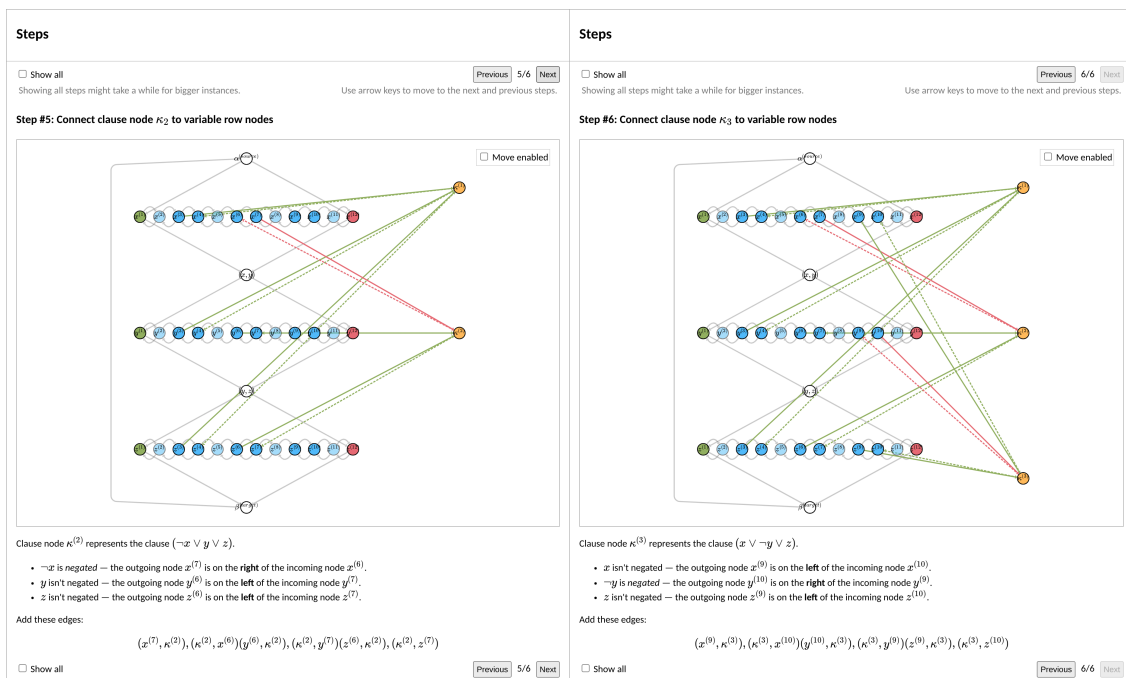
Clause node $\kappa^{(1)}$ represents the clause $(x \vee y \vee z)$.

- x isn't negated — the outgoing node $x^{(3)}$ is on the **left** of the incoming node $x^{(4)}$.
- y isn't negated — the outgoing node $y^{(3)}$ is on the **left** of the incoming node $y^{(4)}$.
- z isn't negated — the outgoing node $z^{(3)}$ is on the **left** of the incoming node $z^{(4)}$.

Add these edges:

$$(x^{(3)}, \kappa^{(1)}), (\kappa^{(1)}, x^{(4)}), (y^{(3)}, \kappa^{(1)}), (\kappa^{(1)}, y^{(4)}), (z^{(3)}, \kappa^{(1)}), (\kappa^{(1)}, z^{(4)})$$

☐ Show all Previous 4/6 Next



Obrázek 42: Krokové zobrazení redukce 3-SAT na HCYLE (kladná odpověď)

6.1.2 Instance se zápornou odpovědí

Ukázková instance:

$$(a \vee a \vee a) \wedge (\neg a \vee \neg a \vee b) \wedge (\neg b \vee \neg b \vee \neg b)$$

Tato instance je nesplnitelná, což demonstruje, že výsledný graf neobsahuje hamiltonovský cyklus. Instance je užitečná pro ilustraci toho, jak aplikace zpracovává instance se zápornou odpovědí.

Editor

Each line defines a clause. A clause consists of three variable names separated by spaces.
A variable name is a single word containing only letters (a-z, A-Z), digits (0-9) and the symbols `_`, `.`, `\`, `(`, `)`, `{`, `}`.
The symbol `\` must be followed by letters (e.g. `\alpha`, `\beta`, `\gamma_5`).
Variable names must not include spaces.
A variable is negated by prefixing it with `!`.
Any duplicate clauses will be removed automatically.

```
a a a
!a !a b
!b !b !b
```

No(2) ▾

Click "Reduce" to reduce the input instance to the output instance.
Click "Solve" to run an solver the output problem instance.
If a solution is found, it is then decoded to the input problem as well.

Reduce Solve

☐ Show steps

Input 3-SAT Instance

☐ View as column

$$(a \vee a \vee a) \wedge (\neg a \vee \neg a \vee b) \wedge (\neg b \vee \neg b \vee \neg b)$$

Certificate

The 3CNF formula is unsatisfiable.

Output HCYCLE Instance

Copy to clipboard

☐ Move enabled

Certificate

The graph doesn't contain a Hamiltonian cycle.

Obrázek 43: Vstupní instance 3-SAT a výsledná instance HCYCLE (záporná odpověď)

6.2 Redukce 3-SAT na SSP

6.2.1 Instance s kladnou odpovědí

Ukázková instance:

$$(\alpha \vee \beta \vee \gamma) \wedge (\neg\alpha \vee \beta \vee \gamma) \wedge (\alpha \vee \neg\beta \vee \gamma)$$

Instance demonstruje:

- reprezentaci proměnných pomocí dvojic čísel,
- vliv klauzulí na jednotlivé číslice,
- funkci vyrovnávacích čísel.

Editor

Each line defines a clause. A clause consists of three variable names separated by spaces.
A variable name is a single word containing only letters (a-z, A-Z), digits (0-9) and the symbols `_`, `.`, `\`, `(`, `)`, `{`, `}`.
The symbol `\` must be followed by letters (e.g. `\alpha`, `\beta_{\gamma_5}`).
Variable names must not include spaces.
A variable is negated by prefixing it with `!`.
Any duplicate clauses will be removed automatically.

`\alpha \beta \gamma`
`!\alpha \beta \gamma`
`\alpha !\beta \gamma`

Yes-TeX(1) ▼

Click "Reduce" to reduce the input instance to the output instance.
Click "Solve" to run an solver the output problem instance.
If a solution is found, it is then decoded to the input problem as well.

Reduce Solve

Shows how the reduction is done step by step. ☐ Show steps

Input 3-SAT Instance

☐ View as column

$(\alpha \vee \beta \vee \gamma) \wedge (\neg \alpha \vee \beta \vee \gamma) \wedge (\alpha \vee \neg \beta \vee \gamma)$

Certificate

$$\begin{aligned}\alpha &= T \\ \beta &= F \\ \gamma &= T\end{aligned}$$

Output SSP Instance

Copy to clipboard

☐ Use 3-SAT format

Target: 111333

Name	Value
$\alpha^{(T)}$	100101
$\alpha^{(F)}$	100010
$\beta^{(T)}$	10110
$\beta^{(F)}$	10001
$\gamma^{(T)}$	1111
$\gamma^{(F)}$	1000
$\kappa_{0,0}$	100
$\kappa_{0,1}$	100
$\kappa_{1,0}$	10
$\kappa_{1,1}$	10
$\kappa_{2,0}$	1
$\kappa_{2,1}$	1

Certificate

☐ Display as table

Subset of numbers summing to the target of 111333:

$$S' = \{10, 10, 100, 1111, 10001, 100101\}$$

Obrázek 44: Zadání instance 3-SAT a výsledná množina čísel SSP (kladná odpověď)

74

Output SSP Instance

Copy to clipboard

☒ Use 3-SAT format

- $\kappa_0 = (\alpha \vee \beta \vee \gamma)$
- $\kappa_1 = (\neg\alpha \vee \beta \vee \gamma)$
- $\kappa_2 = (\alpha \vee \neg\beta \vee \gamma)$

	α	β	γ	κ_0	κ_1	κ_2
$\alpha^{(T)}$	1	0	0	1	0	1
$\alpha^{(F)}$	1	0	0	0	1	0
$\beta^{(T)}$		1	0	1	1	0
$\beta^{(F)}$		1	0	0	0	1
$\gamma^{(T)}$			1	1	1	1
$\gamma^{(F)}$			1	0	0	0
$\kappa_{0,0}$				1	0	0
$\kappa_{0,1}$				1	0	0
$\kappa_{1,0}$					1	0
$\kappa_{1,1}$					1	0
$\kappa_{2,0}$						1
$\kappa_{2,1}$						1
Target:	1	1	1	3	3	3

Certificate

☐ Display as table

Subset of numbers summing to the target of 111333:

$$S' = \{10, 10, 100, 1111, 10001, 100101\}$$

Obrázek 45: Výsledná množina čísel SSP zobrazená s 3-SAT formátováním

Steps

☐ Show all
Showing all steps might take a while for bigger instances.

Previous

1/4

Next

Step #1: Create target sum

☐ Use 3-SAT format

Target: 111333

We start out by defining the target sum τ . Then we define the numbers.

All these numbers including the target sum τ are $k = |\mathcal{V}| + |\mathcal{K}|$ digits long, where $|\mathcal{V}|$ is the number of variables and $|\mathcal{K}|$ is the number of clauses in the 3-CNF formula $\Phi = (\mathcal{V}, \mathcal{K})$. In this case, $k = |\mathcal{V}| + |\mathcal{K}| = 3 + 3 = 6$.

The target sum τ is defined as:

$$\tau = \sum_{i=0}^{|\mathcal{V}|-1} (1 \cdot 10^{k-i}) + \sum_{i=0}^{|\mathcal{K}|-1} (3 \cdot 10^i)$$

In this case, $\tau = 111333$.

☐ Show all

Previous

1/4

Next

Steps

☐ Show all
Showing all steps might take a while for bigger instances.

Previous

2/4

Next

Step #2: Create variable numbers

☐ Use 3-SAT format

Target: 111333

Name	Value
$\alpha^{(T)}$	100000
$\alpha^{(F)}$	100000
$\beta^{(T)}$	010000
$\beta^{(F)}$	010000
$\gamma^{(T)}$	001000
$\gamma^{(F)}$	001000

Create two, at most k digit long, numbers $v^{(T)}$ and $v^{(F)}$ for every variable $v \in \mathcal{V}(\Phi)$. These numbers enforce boolean assignment for every variable. The final subset of numbers S that sums up to τ must include either $v^{(T)}$, or $v^{(F)}$. Choosing $v^{(T)}$ means assigning $v := T$ while choosing $v^{(F)}$ means $v := F$.

In this case we have $|\mathcal{V}| = 3$ variables:

$$\alpha, \beta, \gamma$$

Therefore we will create $2|\mathcal{V}| = 6$ numbers and for now fill them with $k = 6$ zeros.

$$\begin{aligned} \alpha^{(T)} &= 000000 \\ \alpha^{(F)} &= 000000 \\ \beta^{(T)} &= 000000 \\ \beta^{(F)} &= 000000 \\ \gamma^{(T)} &= 000000 \\ \gamma^{(F)} &= 000000 \end{aligned}$$

To enforce that we can choose either $v_i^{(T)}$, or $v_i^{(F)}$ of variable $v_i \in \mathcal{V}$ for $i \in [0, |\mathcal{V}|)$, we set the i -th digit (indexing from 0, left to right) in the numbers $v_i^{(T)}$ and $v_i^{(F)}$ to 1.

$$\begin{aligned} \alpha^{(T)} &= 100000 \\ \alpha^{(F)} &= 100000 \\ \beta^{(T)} &= 010000 \\ \beta^{(F)} &= 010000 \\ \gamma^{(T)} &= 001000 \\ \gamma^{(F)} &= 001000 \end{aligned}$$

The first $|\mathcal{V}|$ digits in the target sum $\tau = 111333$ can be achieved, iff either $v^{(T)}$, or $v^{(F)}$ of every variable v is present in the final subset S .

☐ Show all

Previous

2/4

Next

Steps

☐ Show all
Showing all steps might take a while for bigger instances.

Previous

3/4

Next

Step #3: Update variable numbers

☐ Use 3-SAT format

Target: 111333

Name	Value
$\alpha^{(T)}$	100101
$\alpha^{(F)}$	100010
$\beta^{(T)}$	010110
$\beta^{(F)}$	010001
$\gamma^{(T)}$	001111
$\gamma^{(F)}$	001000

In Φ , there are $|\mathcal{K}| = 3$ clauses:

- $(\alpha \vee \beta \vee \gamma)$
- $(\neg\alpha \vee \beta \vee \gamma)$
- $(\alpha \vee \neg\beta \vee \gamma)$

To model these clauses being satisfied by choosing either the variable number $v^{(T)}$, or $v^{(F)}$, we must update these variable numbers we just created.

For every clause $\kappa_i \in \mathcal{K}$ for $i \in [0, |\mathcal{K}|)$ we will proceed like this:
If v appears in the clause κ_i as a literal, we set the $(|\mathcal{V}| + i)$ -th digit to 1

- for the number $v^{(F)}$ if the literal is negated (appears in the clause as $\neg v$),
- otherwise set the digit for the number $v^{(T)}$.

$$\begin{aligned} \alpha^{(T)} &= 100101 \\ \alpha^{(F)} &= 100010 \\ \beta^{(T)} &= 010110 \\ \beta^{(F)} &= 010001 \\ \gamma^{(T)} &= 001111 \\ \gamma^{(F)} &= 001000 \end{aligned}$$

Now, choosing $v^{(T)}$ and $v^{(F)}$ will also affect the last $|\mathcal{K}|$ digits, at indices $|\mathcal{V}|, k$, of the target sum τ . Namely, it will cause these digits to be greater than zero. This corresponds to clauses being satisfied. The $(|\mathcal{V}| + i)$ -th digit of τ for $i \in [0, |\mathcal{K}|)$ being greater than zero, means the clause κ_i is satisfied.

☐ Show all

Previous

3/4

Next

Steps

☐ Show all
Showing all steps might take a while for bigger instances.

Previous

4/4

Next

Step #4: Create buffer numbers

☐ Use 3-SAT format

Target: 111333

Name	Value
$\alpha^{(T)}$	100101
$\alpha^{(F)}$	100010
$\beta^{(T)}$	010110
$\beta^{(F)}$	010001
$\gamma^{(T)}$	001111
$\gamma^{(F)}$	001000
$\kappa_{0,0}$	000100
$\kappa_{0,1}$	000100
$\kappa_{1,0}$	000010
$\kappa_{1,1}$	000010
$\kappa_{2,0}$	000001
$\kappa_{2,1}$	000001

The j -th digits for $j \in [|\mathcal{V}|, k)$ of the target sum $\tau = 111333$, are all 3. With our current set of numbers, a j -th digit can be 3 iff we choose the numbers $v^{(T)}$ and $v^{(F)}$ in such a way that a clause $\kappa_{j-|\mathcal{V}|}$ has all 3 of its literals be evaluated to *True*.

We know, however, that a clause $\kappa_{j-|\mathcal{V}|}$ is satisfied, if the j -th digit is greater than zero. It doesn't have to be 3.

Therefore, we add *filler numbers* in such a way, that all j -th digits greater than zero can achieve being 3. On the otherhand, we must prohibit j -th digits that are currently zero from being able to become 3.

To achieve this, we add two filler numbers, $\kappa_{i,0}$ and $\kappa_{i,1}$, for each clause κ_i .

$$\kappa_{i,0} = \kappa_{i,1} = 10^i$$

In this case:

$$\begin{aligned} \kappa_{0,0} &= 000100 \\ \kappa_{0,1} &= 000100 \\ \kappa_{1,0} &= 000010 \\ \kappa_{1,1} &= 000010 \\ \kappa_{2,0} &= 000001 \\ \kappa_{2,1} &= 000001 \end{aligned}$$

Now, if the j -th digit is greater than zero, we can choose the filler numbers $\kappa_{j,0}$ and $\kappa_{j,1}$ to add to our subset S to achieve the target value of 3 in that position. However, if the j -th digit is zero, adding these filler numbers won't help since the maximum we could achieve is 2, not 3.

☐ Show all

Previous

4/4

Next

Obrázek 46: Krokové zobrazení redukce 3-SAT na SSP (kladná odpověď)

6.2.2 Instance se zápornou odpovědí

Ukázková instance:

$$(x \vee x \vee x) \wedge (\neg x \vee \neg x \vee \neg x)$$

Demonstruje případ, kdy nelze dosáhnout cílové hodnoty τ , protože žádná podmnožina neumožňuje splnění všech klauzulí.

Editor

Each line defines a clause. A clause consists of three variable names separated by spaces.

A variable name is a single word containing only letters (a-z, A-Z), digits (0-9) and the symbols `_`, `x`, `\`, `(`, `)`, `{`, `}`.

The symbol `\` must be followed by letters (e.g. `\alpha`, `\beta_{\gamma_5}`).

Variable names must not include spaces.

A variable is negated by prefixing it with `!`.

Any duplicate clauses will be removed automatically.

x x x

!x !x !x

No(1) ▾

Click "Reduce" to reduce the input instance to the output instance.

Click "Solve" to run an solver the output problem instance.

If a solution is found, it is then decoded to the input problem as well.

Shows how the reduction is done step by step.

Reduce

Solve

☐ Show steps

Input 3-SAT Instance

☐ View as column

$$(x \vee x \vee x) \wedge (\neg x \vee \neg x \vee \neg x)$$

Certificate

The 3CNF formula is unsatisfiable.

Output SSP Instance

Copy to clipboard

☐ Use 3-SAT format

Target: 133

Name	Value
$x^{(T)}$	110
$x^{(F)}$	101
$\kappa_{0,0}$	10
$\kappa_{0,1}$	10
$\kappa_{1,0}$	1
$\kappa_{1,1}$	1

Certificate

The SSP instance doesn't have a solution.

Obrázek 47: Zadání instance 3-SAT a výsledná množina čísel SSP (záporná odpověď)

6.3 Redukce 3-SAT na 3-CG

6.3.1 Instance s kladnou odpovědí

Ukázková instance:

$$(\alpha \vee \beta \vee \gamma) \wedge (\neg\alpha \vee \beta \vee \neg\gamma)$$

Instance ukazuje:

- jádrový konstrukční prvek a jeho obarvení,
- konstrukční prvky proměnných,
- propojení s konstrukčními prvky klauzulí.

Editor

Each line defines a clause. A clause consists of three variable names separated by spaces.
A variable name is a single word containing only letters (a-z, A-Z), digits (0-9) and the symbols `_`, `.`, `\`, `{`, `}`, `[`, `]`.
The symbol `\` must be followed by letters (e.g. `\alpha`, `\beta_{\gamma_5}`).
Variable names must not include spaces.
A variable is negated by prefixing it with `!`.
Any duplicate clauses will be removed automatically.

```
\alpha !\alpha \beta
\beta \alpha !\beta
```

Yes-TeX(2) ▼

Click "Reduce" to reduce the input instance to the output instance.
Click "Solve" to run an solver the output problem instance.
If a solution is found, it is then decoded to the input problem as well.

Reduce Solve

Shows how the reduction is done step by step. ☐ Show steps

Input 3-SAT Instance

☐ View as column

$$(\alpha \vee \neg\alpha \vee \beta) \wedge (\beta \vee \alpha \vee \neg\beta)$$

Certificate

$$\alpha = F$$

$$\beta = F$$

Output 3-CG Instance

Copy to clipboard

☐ Move enabled

Certificate

☒ Display as sets

$$R = \{F, \alpha, \beta, \kappa_{0,1}, \kappa_{0,3}, \kappa_{1,2}, \kappa_{1,3}\}$$

$$G = \{T, \neg\alpha, \neg\beta, \kappa_{0,5}, \kappa_{1,4}\}$$

$$B = \{B, \kappa_{0,0}, \kappa_{0,2}, \kappa_{0,4}, \kappa_{1,0}, \kappa_{1,1}, \kappa_{1,5}\}$$

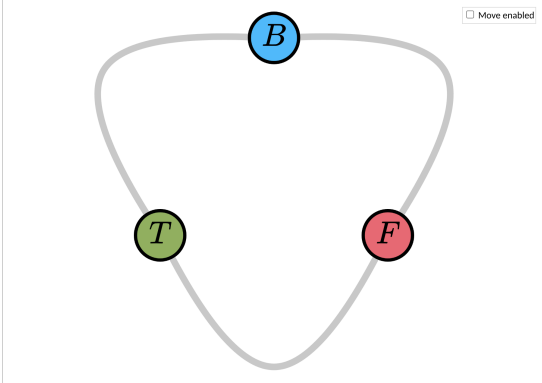
Obrázek 48: Vstupní instance 3-SAT a výsledná grafová instance 3-CG (kladná odpověď)

Steps

☐ Show all
Showing all steps might take a while for bigger instances. Previous 1/3 Next

Use arrow keys to move to the next and previous steps.

Step #1: Create the core gadget



Let's assume that the 3 colors are *red*, *green* and *blue*.

We start by creating the core gadget $G_C = (V_C, E_C)$. The core gadget has three nodes and they are all connected to one another.

$$V_C = \{T, F, B\}$$

$$E_C = \{\{T, F\}, \{F, B\}, \{B, T\}\}$$

The coloring for these nodes is the following:

$$\begin{aligned} \text{color}(T) &= \text{green} \\ \text{color}(F) &= \text{red} \\ \text{color}(B) &= \text{blue} \end{aligned}$$

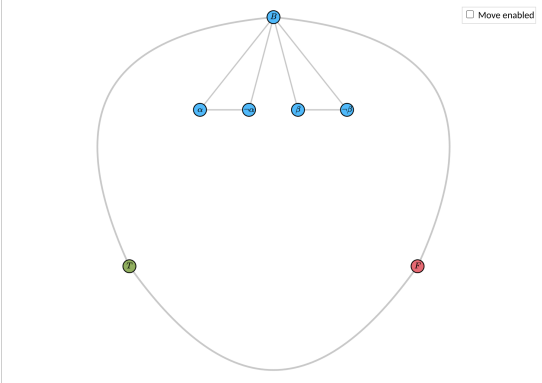
☐ Show all Previous 1/3 Next

Steps

☐ Show all
Showing all steps might take a while for bigger instances. Previous 2/3 Next

Use arrow keys to move to the next and previous steps.

Step #2: Create variable gadgets



Create a variable gadget for each variable $v \in \mathcal{V}(\Phi)$ of the boolean formula $\Phi = (\mathcal{V}, \mathcal{K})$, where $\mathcal{V}$ is the set of variables:

$$\mathcal{V} = \{\alpha, \beta\}$$

and $\mathcal{K}$ is the set of clauses:

$$\mathcal{K} = \{(\alpha, \neg\alpha, \beta), (\beta, \alpha, \neg\beta)\}$$

There will be $|\mathcal{V}| = 2$ variable gadgets.

A variable gadget $G_v = (V_v, E_v)$ for a variable v consists of three nodes,

$$V_v = \{v, \neg v, B\}$$

where the node B is the blue node from the core gadget G_C . These nodes are each connected to one another:

$$E_v = \{\{v, \neg v\}, \{v, B\}, \{\neg v, B\}\}$$

Since the nodes v and $\neg v$ are connected to the B node that is colored *blue*, they themselves can only be colored either *green*, or *red*. This encodes the truth assignment for the variable v , as only these 2 cases can occur:

- The node v is *green* and the node $\neg v$ is *red*. This means $v \models \text{True}$.
- The node v is *red* and the node $\neg v$ is *green*. This means $v \models \text{False}$.

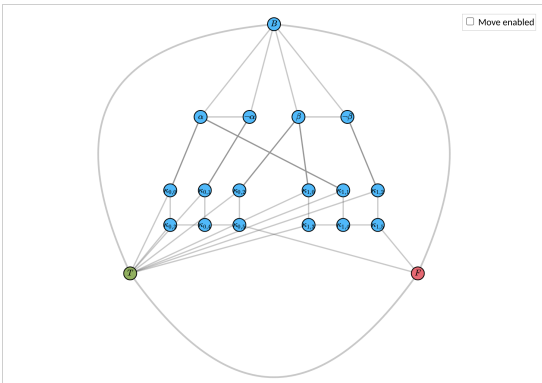
☐ Show all Previous 2/3 Next

Steps

☐ Show all
Showing all steps might take a while for bigger instances. Previous 3/3 Next

Use arrow keys to move to the next and previous steps.

Step #3: Add clause gadgets



Create a clause gadget for each clause $\kappa \in \mathcal{K}(\Phi)$:

$$\begin{aligned} \kappa_0 &= (\alpha, \neg\alpha, \beta) \\ \kappa_1 &= (\beta, \alpha, \neg\beta) \end{aligned}$$

There will be $|\mathcal{K}| = 2$ clause gadgets.

A clause gadget $G_\kappa = (V_\kappa, E_\kappa)$ for some clause $\kappa = \{A, B, \Gamma\}$, where A, B and Γ are its literals (they can be negated) and α, β and γ are the respective variables, is defined as follows:

$$\begin{aligned} V_\kappa &= \{A, B, \Gamma, \kappa_0, \kappa_1, \kappa_2, \kappa_3, \kappa_4, \kappa_5, T, F\} \\ E_\kappa &= \{ \{A, \kappa_0\}, \{B, \kappa_1\}, \{\Gamma, \kappa_2\} \\ &\quad \{\kappa_0, \kappa_3\}, \{\kappa_1, \kappa_4\}, \{\kappa_2, \kappa_5\} \\ &\quad \{T, \kappa_0\}, \{T, \kappa_1\}, \{T, \kappa_2\} \\ &\quad \{T, \kappa_3\}, \{\kappa_4, \kappa_5\}, \{\kappa_4, \kappa_2\}, \{\kappa_5, F\} \} \end{aligned}$$

where the nodes T and F come from the core gadget G_C .

Note that the nodes $\kappa_0, \kappa_1, \dots, \kappa_5$ are unique for each clause gadget.

A valid 3-coloring for the gadget G_κ exist, iff at least one of the literal nodes A, B or Γ is *green*. This would correspond to the clause κ being satisfied, because having at least one of the literal nodes being *green* means that at least one of its literals is evaluated to *True*.

However, if all the 3 literal nodes are *red*, then a valid 3-coloring for the gadget G_κ doesn't exist. Since all 3 literal nodes are *red*, this means that all the literals evaluate to *False* and the clause κ is not satisfied.

☐ Show all Previous 3/3 Next

Obrázek 49: Krokové zobrazení redukce 3-SAT na 3CG (kladná odpověď)

6.3.2 Instance se zápornou odpovědí

Ukázková instance:

$$(x \vee x \vee x) \wedge (\neg x \vee \neg x \vee \neg x)$$

Demonstruje případ, kdy graf není 3-obarvitelný, protože konflikt v klauzulích vynutí protichůdné podmínky pro obarvení.

Editor

Each line defines a clause. A clause consists of three variable names separated by spaces.
A variable name is a single word containing only letters (a-z, A-Z), digits (0-9) and the symbols `_`, `.`, `\`, `(`, `)`, `{`, `}`.
The symbol `\` must be followed by letters (e.g. `\alpha`, `\beta_{\gamma_5}`).
Variable names must not include spaces.
A variable is negated by prefixing it with `!`.
Any duplicate clauses will be removed automatically.

```
x x x
!x !x !x
```

No(1) ▾

Click "Reduce" to reduce the input instance to the output instance.
Click "Solve" to run an solver the output problem instance.
If a solution is found, it is then decoded to the input problem as well.

Reduce Solve

☐ Show steps

Input 3-SAT Instance

☐ View as column

$$(x \vee x \vee x) \wedge (\neg x \vee \neg x \vee \neg x)$$

Certificate

The 3CNF formula is unsatisfiable.

Output 3-CG Instance

Copy to clipboard

☐ Move enabled

Certificate

The graph can't be colored by 3 colors.

Obrázek 50: Vstupní instance 3-SAT a výsledná grafová instance 3-CG (záporná odpověď)

6.4 Redukce HCYCLE na HCIRCUIT

6.4.1 Instance s kladnou odpovědí

Ukázkovou instancí je orientovaný graf s hamiltonovským cyklem:

$$V = \{C_1, C_2, C_3, C_4\}$$
$$E = \{(C_1, C_3), (C_3, C_2), (C_2, C_1), (C_1, C_2), (C_2, C_3), (C_3, C_4), (C_4, C_1)\}$$

Editor

Write each entry on a new line. Each entry defines either a node, or an edge.
 A node is a single word containing only letters (a-z, A-Z), digits (0-9) and the symbols $_$, $\%$, $\>$, $\<$, $\{$, $\}$, $\{$, $\}$.
 The symbol $\%$ must be followed by letters (e.g. $\%alpha$, $\%beta_gamma\delta$).
 Node names must not include spaces.
 To define an edge, write two node names separated by a space.
 Any duplicate entries will be removed automatically.

C_1
C_3
C_2
C_4
C_1 C_3
C_3 C_2
C_2 C_1
C_1 C_2
C_2 C_3
C_3 C_4
C_4 C_1

Yes(2)

Click "Reduce" to reduce the input instance to the output instance.
 Click "Solve" to run a solver the output problem instance.
 If a solution is found, it is then decoded to the input problem as well.

Reduce

Solve

Shows how the reduction is done step by step. ☐ Show steps

Input HCYCLE Instance

Click on two nodes to add an edge between them.
 Click on an edge to remove it.

☐ Move enabled

Certificate

☐ Show as list

$$P = (C_1, C_2, C_3, C_4, C_1)$$

Output HCIRCUIT Instance

☐ Move enabled

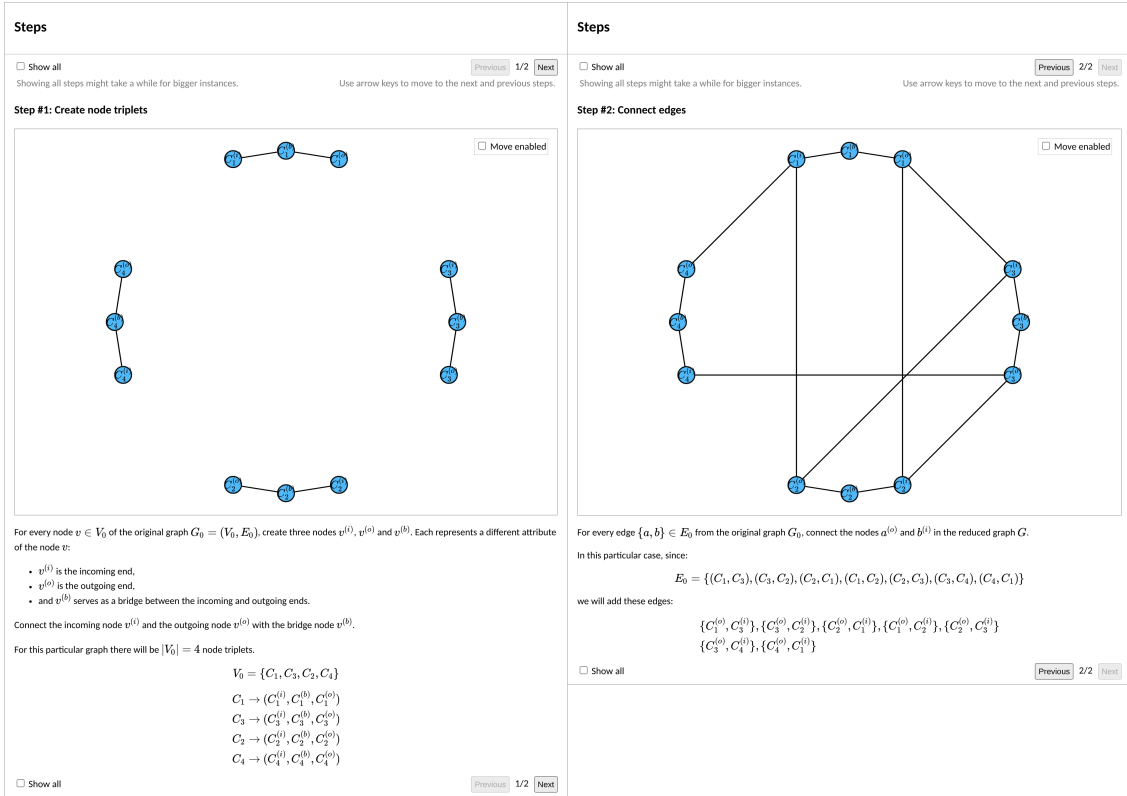
Certificate

☐ Show as list

$$P = (C_1^{(i)}, C_1^{(b)}, C_1^{(o)}, C_2^{(i)}, C_2^{(b)}, C_2^{(o)}, C_3^{(i)}, C_3^{(b)}, C_3^{(o)}, C_4^{(i)}, C_4^{(b)}, C_4^{(o)})$$

Obrázek 51: Vstupní instance HCYCLE a výsledný graf HCIRCUIT (kladná odpověď)

85



Obrázek 52: Krokové zobrazení redukce HCYCLE na HCIRCUIT (kladná odpověď)

6.4.2 Instance se zápornou odpovědí

Ukázkovou instancí je orientovaný acyklický graf:

$$V = \{C_1, C_2, C_3, C_4\}$$

$$E = \{(C_1, C_3), (C_2, C_3), (C_2, C_1), (C_2, C_3), (C_3, C_4), (C_4, C_1)\}$$

Demonstruje neexistenci hamiltonovského cyklu v acyklickém grafu a jak aplikace zpracovává zápornou odpověď.

Editor

Write each entry on a new line. Each entry defines either a node, or an edge.
 A node is a single word containing only letters (a-z, A-Z), digits (0-9) and the symbols $\square$, $\square$, $\square$, $\square$, $\square$, $\square$.
 The symbol $\square$ must be followed by letters (e.g. $\square$ alpha, $\square$ beta_5).
 Node names must not include spaces.
 To define an edge, write two node names separated by a space.
 Any duplicate entries will be removed automatically.

C_1
C_3
C_2
C_4
C_1 C_3
C_2 C_3
C_2 C_1
C_3 C_4
C_4 C_1

No(2)

Click "Reduce" to reduce the input instance to the output instance.
 Click "Solve" to run a solver the output problem instance.
 If a solution is found, it is then decoded to the input problem as well.

Reduce

Solve

Shows how the reduction is done step by step.

Show steps

Input HCYCLE Instance

Click on two nodes to add an edge between them.
 Click on an edge to remove it.

Move enabled

Certificate

The graph doesn't contain a Hamiltonian cycle.

Output HCIRCUIT Instance

Copy to clipboard

Move enabled

Certificate

The graph doesn't contain a Hamiltonian circuit.

Obrázek 53: Vstupní instance HCYCLE a výsledný graf HCIRCUIT (záporná odpověď)

6.5 Redukce HCIRCUIT na TSP

6.5.1 Instance s kladnou odpovědí

Ukázkovou instancí je neorientovaný graf obsahující hamiltonovskou kružnici:

$$V = \{x_0, x_1, x_2, x_3, x_4, x_5\}$$

$$E = \{\{x_0, x_1\}, \{x_4, x_2\}, \{x_5, x_3\}, \{x_3, x_2\}, \{x_5, x_0\}, \{x_2, x_1\}, \{x_4, x_5\}, \{x_0, x_2\}, \{x_4, x_0\}\}$$

Instance ukazuje přidání hran s vyšší vahou a nastavení cílové hodnoty $k = |V|$.

Editor

Write each entry on a new line. Each entry defines either a node, or an edge.
 A node is a single word containing only letters ($a-z$, $A-Z$), digits ($0-9$) and the symbols $_, \cdot, \backslash, \{, \}$.
 The symbol $\backslash$ must be followed by letters (e.g. $\backslash alpha$, $\backslash beta_{\gamma gamma_3}$).
 Node names must not include spaces.
 To define an edge, write two node names separated by a space.
 Any duplicate entries will be removed automatically.

```

x_0
x_1
x_2
x_3
x_4
x_5
x_0 x_1
x_4 x_2
x_5 x_3
x_3 x_2
x_5 x_0
x_2 x_1
x_4 x_5
x_0 x_2
x_4 x_0
      
```

Click "Reduce" to reduce the input instance to the output instance.
 Click "Solve" to run an solver the output problem instance.
 If a solution is found, it is then decoded to the input problem as well.

Shows how the reduction is done step by step.

☐ Show steps

Input HCIRCUIT Instance

Click on two nodes to add an edge between them.
 Click on an edge to remove it.

☐ Move enabled

Certificate

☐ Show as list

$$P = (x_0, x_4, x_5, x_3, x_2, x_1, x_0)$$

Output TSP Instance

$k = 6$

☐ Move enabled

Certificate

☐ Show as list

$$P = (x_0, x_4, x_5, x_3, x_2, x_1, x_0)$$

Obrázek 54: Vstupní instance HCIRCUIT a výsledná instance TSP (kladná odpověď)

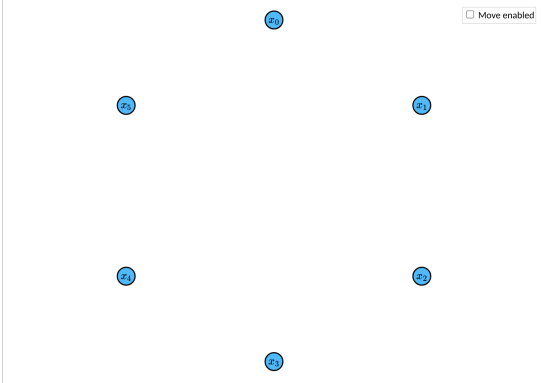
Steps

☐ Show all
Showing all steps might take a while for bigger instances. Previous 1/3 Next

Use arrow keys to move to the next and previous steps.

Step #1: Copy vertices

☐ Move enabled



To create a reduced graph instance $G = (V, E)$ for TSP from a graph instance $G_0 = (V_0, E_0)$ from HCIRCUIT, take all of the nodes from V_0 and add them to G .

$$V = V_0 = \{x_0, x_1, x_2, x_3, x_4, x_5\}$$

☐ Show all Previous 1/3 Next

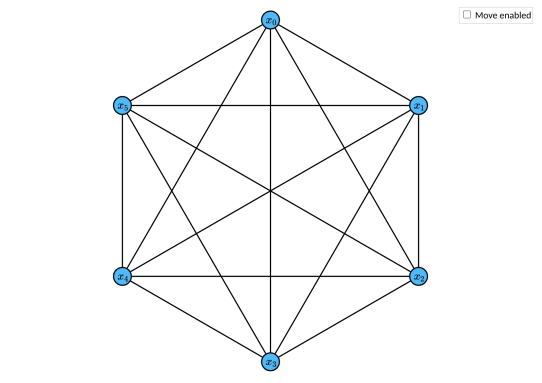
Steps

☐ Show all
Showing all steps might take a while for bigger instances. Previous 2/3 Next

Use arrow keys to move to the next and previous steps.

Step #2: Create edges

☐ Move enabled



Then, add an edge between each pair of vertices to create a complete graph.

$$E = \{\{v_1, v_2\} \mid v_1, v_2 \in V_0\}$$

The number of nodes in G is $|V| = 6$, which means there must be $|E| = (6 \cdot 5)/2 = 15$ edges, for the graph to be complete.

$$E = \{\{x_0, x_1\}, \{x_0, x_2\}, \{x_0, x_3\}, \{x_0, x_4\}, \{x_0, x_5\}, \{x_1, x_2\}, \{x_1, x_3\}, \{x_1, x_4\}, \{x_1, x_5\}, \{x_2, x_3\}, \{x_2, x_4\}, \{x_2, x_5\}, \{x_3, x_4\}, \{x_3, x_5\}, \{x_4, x_5\}\}$$

☐ Show all Previous 2/3 Next

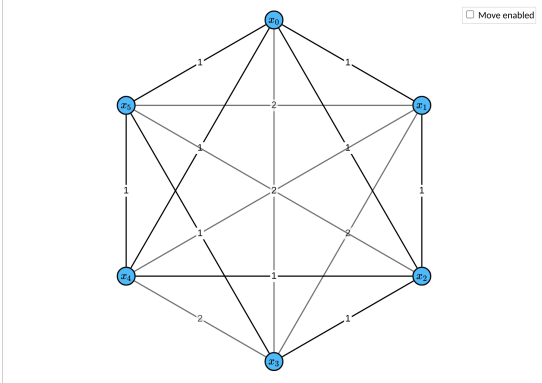
Steps

☐ Show all
Showing all steps might take a while for bigger instances. Previous 3/3 Next

Use arrow keys to move to the next and previous steps.

Step #3: Assign weights

☐ Move enabled



For every edge, that exists in the original instance G_0 , assign the weight to 1. Otherwise, assign some weight greater than 1. The function $w(e)$ for assigning weights to edges in E may be defined as:

$$w(e) = \begin{cases} 1 & \text{if } e \in E_0 \\ 2 & \text{otherwise} \end{cases}$$

Let k be the maximal cost of the hamiltonian path in the graph G . Set k to the number of nodes $|V|$.

$$k = |V| = 6$$

Now, the TSP problem can be expressed by a question: Does the graph $G = (V, E)$ contain a hamiltonian path P of length $|V| + 1$, such that the cost of the traversed path k_P is less than or equal to k ? In other words, does this statement hold?

$$k_P = \sum_{i=0}^{|V|} w(\{P_i, P_{i+1}\}) \leq k$$

If so, the original graph G_0 contains a hamiltonian circuit, otherwise it doesn't.

☐ Show all Previous 3/3 Next

Obrázek 55: Krokové zobrazení redukce HCIRCUIT na TSP (kladná odpověď)

6.5.2 Instance se zápornou odpovědí

Ukázkovou instancí je graf neobsahující hamiltonovskou kružnici:

$$V = \{0, 1, 2, 3, 4\}$$
$$E = \{\{0, 1\}, \{1, 2\}, \{2, 3\}, \{3, 4\}, \{0, 2\}, \{0, 3\}\}$$

Demonstruje případ, kdy neexistuje cesta s cenou $k = |V|$, protože graf neobsahuje hamiltonovskou kružnici.

Editor

Write each entry on a new line. Each entry defines either a node, or an edge.
 A node is a single word containing only letters (`a-z`, `A-Z`), digits (`0-9`) and the symbols `_`, `.`, `!`, `?`, `~`, `&`, `^`, `*`, `+`, `=`, `<`, `>`, `~`, `&`, `^`, `*`, `+`, `=`, `<`, `>`.
 The symbol `\` must be followed by letters (e.g. `\alpha`, `\beta`, `\gamma`, `\delta`, `\epsilon`, `\zeta`, `\eta`, `\theta`, `\iota`, `\kappa`, `\lambda`, `\mu`, `\nu`, `\xi`, `\o`, `\p`, `\q`, `\r`, `\s`, `\t`, `\u`, `\v`, `\w`, `\x`, `\y`, `\z`).
 Node names must not include spaces.
 To define an edge, write two node names separated by a space.
 Any duplicate entries will be removed automatically.

```
0
1
2
3
4
0 1
1 2
2 3
3 4
0 2
0 3
```

Click "Reduce" to reduce the input instance to the output instance.
 Click "Solve" to run a solver the output problem instance.
 If a solution is found, it is then decoded to the input problem as well.

Shows how the reduction is done step by step.

☐ Show steps

Input HCIRCUIT Instance

Click on two nodes to add an edge between them.
 Click on an edge to remove it.

☐ Move enabled

Certificate

The graph doesn't contain a Hamiltonian cycle.

Output TSP Instance

$k = 5$

☐ Move enabled

Certificate

The graph doesn't contain a Hamiltonian cycle with the given cost of 5.

Obrázek 56: Vstupní instance HCIRCUIT a výsledná instance TSP (záporná odpověď)

Kapitola 7

Zhodnocení

V této kapitole jsou shrnuty dosažené výsledky, diskutována omezení navrženého řešení a nastíněny možnosti budoucího rozšíření.

7.1 Shrnutí výsledků

Cílem této práce bylo navrhnout a implementovat interaktivní výukovou aplikaci pro vizualizaci polynomiálních redukcí mezi NP-úplnými problémy. V rámci práce byla tato aplikace realizována.

Konkrétně byla implementována webová aplikace umožňující:

- zadávání vlastních instancí vstupních problémů a automatické provádění redukcí,
- krokové zobrazení procesu redukce s vysvětlením jednotlivých operací,
- vizualizaci vstupních a výstupních instancí v podobě grafů, logických výrazů a tabulek,
- nalezení řešení pomocí implementovaných algoritmů,
- dekódování řešení zpět na řešení vstupní instance a
- zobrazení nalezených certifikátů řešení.

7.2 Výuková hodnota

Navržená aplikace přináší několik výhod oproti tradičním statickým výukovým materiálům:

1. Uživatel může pracovat s libovolnou vlastní instancí. Na rozdíl od předpřipravených animací tak může experimentovat a ověřovat si své pochopení na konkrétních příkladech.
2. Krokové zobrazení umožňuje sledovat transformaci instance. Každý krok je doplněn o textové vysvětlení.
3. Možnost dekódování řešení demonstruje, jak lze z řešení výstupní instance odvodit řešení instance vstupní. Toto propojení je klíčové pro pochopení korektnosti redukcí.

7.3 Omezení

Navržené řešení má několik omezení, která je třeba vzít v úvahu.

Rozsah podporovaných problémů

Aplikace podporuje pouze problémy a redukce definované v rámci této práce. Přidání nových problémů vyžaduje implementaci odpovídajících tříd a může si vyžádat úpravu stávající architektury.

Výkon řešících algoritmů

Řešící algoritmy běží na klientské straně v prostředí webového prohlížeče. Pro rozsáhlé instance může být výpočet časově náročný, což vede ke zpomalení odezvy aplikace.

Vizualizace grafů a srozumitelnost výstupu

Grafy jsou rozloženy automaticky, což nemusí vždy vést k optimálnímu výsledku. Ačkoliv byly pro některé redukce implementovány vlastní algoritmy pro umístování vrcholů, u složitějších instancí může být výsledný graf stále obtížně čitelný. Totéž platí pro vizualizace v TeX – text není vždy rozložen nejpřehledněji.

Uživatelské rozhraní

Rozhraní je navrženo primárně pro desktopové prostředí. Při vývoji nebylo řešeno responzivní chování pro zařízení s menším displejem.

7.4 Možnosti rozšíření

Navržené řešení lze rozšířit v několika směrech.

Rozšíření o další problémy

Přidání podpory pro další NP-úplné problémy a redukce, například 3-SAT na problém maximálního řezu (Max-Cut) nebo na problém nezávislé množiny (Independent Set), by umožnilo demonstraci širší škály převodů.

Export a ukládání dat

Aplikaci by bylo možné rozšířit o možnost exportu grafů jako obrázků či exportu výsledků do formátu TeX pro vložení do dokumentace. Dále by bylo užitečné umožnit ukládání vlastních instancí v prohlížeči, například s využitím `localStorage`.

Závěr

Tato práce se zabývala návrhem a implementací interaktivní webové aplikace pro vizualizaci polynomiálních redukcí mezi NP-úplnými problémy. Cílem bylo překlenout mezeru mezi abstraktní teoretickou definicí redukcí a jejich konkrétním pochopením studenty.

V teoretické části byly představeny základy výpočetní složitosti a šest vybraných NP-úplných problémů. Klíčovým přínosem této části je detailní popis pěti polynomiálních redukcí mezi těmito problémy, včetně analýzy konstrukčních prvků a důkazů korektnosti převodů.

V praktické části byla navržena a realizována klientská webová aplikace, která umožňuje uživatelům zadávat vlastní instance vstupních problémů, provádět redukce, sledovat jejich průběh krok za krokem a vizualizovat výsledné instance a jejich řešení.

Hlavním přínosem práce je vytvoření nástroje, který umožňuje experimentovat s libovolnými instancemi a interaktivně prozkoumávat chování konstrukčních prvků při převodu z jednoho problému na druhý. Na rozdíl od statických výukových materiálů tak aplikace poskytuje aktivní přístup ke studiu vztahů mezi NP-úplnými problémy.

Navržené řešení může sloužit jako doplňková výuková pomůcka v předmětech zaměřených na teorii výpočetní složitosti a poskytuje studentům prostor pro samostatné ověřování svého porozumění této problematice.

Zdroje

- [1] OPENDSA PROJECT. *Reduction of 3-SAT to Hamiltonian Cycle* [online]. 15. říjen 2025 [cit. 2026-04-24]. Dostupné z: https://opensa-server.cs.vt.edu/OpenDSA/Books/Everything/html/threeSAT_to_hamiltonianCycle.html
- [2] CHRISTINA ZHANG. *Proving the Subset Sum Problem is NP-Complete / 3-SAT to Subset Sum Reduction* [online]. 23. srpen 2025 [cit. 2026-04-24]. Dostupné z: <https://www.youtube.com/watch?v=mXunNPW1erU>
- [3] COMPUTER SCIENCE THEORY EXPLAINED. *3-Colorability* [online]. 27. únor 2021 [cit. 2026-04-24]. Dostupné z: <https://www.youtube.com/watch?v=fMh42fsIf0Q>
- [4] ZDENĚK SAWA. *Příklady důkazů NP-úplnosti* [online]. 27. říjen 2025 [cit. 2026-04-24]. Dostupné z: <https://www.cs.vsb.cz/sawa/ti/slides/ti-slides-06.pdf>

Příloha A

Obsah přílohy

```
1 ./
2 |-- README.md ; hlavní dokumentace projektu
3 |-- ThesisSpecification_TRA0163_vsboee25040F4E.pdf ; oficiální zadání práce
4 |
5 |-- impl/ ; implementace aplikace (zdrojový kód)
6 | |-- README.md ; dokumentace k implementaci
7 | |-- push_homel.sh* ; skript pro nasazení (na homel)
8 | |-- package.json ; konfigurace projektu a závislosti
9 | |-- package-lock.json
10 | |-- src/ ; hlavní zdrojové soubory aplikace
11 | | |-- app.d.ts ; globální TypeScript deklarace
12 | | |-- app.html ; hlavní HTML šablona
13 | | |-- lib/ ; interní knihovny a moduly
14 | | | |-- component/ ; znovupoužitelné UI komponenty
15 | | | |-- core/ ; jádro aplikace (hlavní logika)
16 | | | |-- decode/ ; dekodovací algoritmy / logika
17 | | | |-- demo/ ; ukázkové / testovací části
18 | | | |-- instance/ ; reprezentace instancí problémů
19 | | | |-- page/ ; logika jednotlivých stránek
20 | | | |-- reduction/ ; redukční algoritmy
21 | | | |-- solve/ ; řešící algoritmy
22 | | | |-- state/ ; správa stavu aplikace
23 | | | |-- workers/ ; web workeři (vlákna na pozadí)
24 | | |-- routes/ ; routování (stránky aplikace)
25 | | | |-- 3sat-3cg/
26 | | | |-- 3sat-hcycle/
27 | | | |-- 3sat-ssp/
28 | | | |-- hcircuit-tsp/
29 | | | |-- hcycle-hcircuit/
30 | | | |-- +layout.svelte ; layout aplikace
31 | | | |-- +page.svelte ; domovská stránka
32 | | |-- styles/ ; globální styly (CSS apod.)
33 | |-- static/ ; statické soubory (obrázky, fonty, asety)
34 | |-- svelte.config.js ; konfigurace Svelte
35 | |-- tsconfig.json ; konfigurace TypeScriptu
36 | |-- vite.config.ts ; konfigurace bundleru (Vite)
37 |
38 |-- paper/ ; zdrojové soubory bakalářské práce
39 | |-- README.md ; dokumentace k textu práce
40 | |-- main.pdf ; finální PDF práce
41 | |-- main.typ ; hlavní vstupní soubor (Typst)
42 | |-- bib.yml ; bibliografie (použité zdroje)
43 | |-- gen-diagram.sh* ; skript pro generování diagramů (PlantUML)
44 | |-- config.typ ; konfigurace sazby (Typst)
45 | |-- assets/ ; doplňkové materiály (asety práce)
46 | | |-- codes/ ; ukázky zdrojového kódu (textové soubory)
47 | | |-- drawio/ ; diagramy (draw.io)
48 | | |-- iso690-numeric-brackets-cs.csl ; citační styl (ISO 690)
49 | | |-- lang-database.toml ; jazyková / lokalizační data
50 | | |-- plantuml/ ; UML diagramy (PlantUML)
51 | | |-- screenshots/ ; snímky obrazovky
52 | | |-- thesis-assignment.pdf ; zadání práce
53 | |-- chapters/ ; kapitoly práce
54 | |-- lib/ ; pomocný kód pro sazbu
55 | |-- scraper/ ; nástroj pro pořizování screenshotů
56
57
58 34 directories, 221 files
```
